Lab3- Persky Idan 215548512 , Golan Amir 216264333

Lab 03 – Data Visualization: Seeing What the Numbers Hide

Data Science Lab 83-554 · Bar-Ilan University · Dr. Alexandra Litinsky Simanovsky

You already know how to plot. Today the question is different: **does the chart tell the truth?**

The story. You are the new data analyst of a ministry's climate unit. The Minister's office sends you a file of world energy and CO2 data compiled by a hurried colleague, and asks for a briefing. In seven blocks you will check whether the data can be trusted, draw who emits and how much, find out why countries differ, build charts for the public website, find the bugs behind four wrong numbers, write the one-page briefing for the Minister, and review what an AI assistant produced for the same task. About 40 graphs on the way. You will work with real world energy and climate data (1990–2023). The data contain realistic problems, and a good chart is often the fastest way to find them.

Session	Block	Topic	Exercises
1	0	Load, clean and look	1–6
	1	Matplotlib: who emits, and how much?	7–14
	2	Seaborn: why do countries differ?	15–22
	3	Plotly: interactive charts for the website	23–27
2	4	Find the bug	28–31
	5	Integrated challenge: the briefing	32–36
	6	Working with an LLM, critically	37–40
extra	7	More chart types ★	41–46

Plan: about 90 minutes in the first session and 45 in the second.

★ = extra. Exercises marked ★ (all of Block 7) are extra practice with more chart types: do them if you finish early. Whatever you do not finish in class, you finish on your own.

How to work

- Each exercise shows the **expected output** (a chart, a table, or a description of the result) under an empty cell. Write code that produces it, then run the cell.
- Hints are collapsed under each exercise. Try first, open them only if you are stuck.
- Where an exercise says *explain*, write your answer in a short comment. A number from your own output is part of a good answer.
- Exercises depend on each other and run **in order, once**: if a result looks strange, restart from the first cell instead of re-running a single cell (some cells change the data in place). Later exercises use `yearly`, `countries`, `aggregates`, `info`, `m`, `d19`, `ct` from earlier ones.

Data (folder `data/`)

- `countries_yearly.csv` – one row per nation and year: population, GDP, CO2 emissions, CO2 by fuel, energy use, and `renewables_share_pct` (renewables as a percent of electricity).
- `energy_mix.csv` – energy use by source in TWh (long format), 2000–2023.
- `country_info.csv` – continent and income tier per country code (`iso3`).

Source: Our World in Data (CO2 and energy datasets), licensed CC BY 4.0, adapted by the course staff for teaching.

Block 0 – Load, clean and look: charts that expose data problems

The file arrives without documentation. Before any conclusion, look at it with charts: they find in seconds what a table hides. **Toolbox for checking a table quickly:** `df.info()` and `df.dtypes` (types and missing values), `df[col].value_counts()` (every distinct value), `df.duplicated().sum()` (repeated rows), `df[col].unique()`, and above all: compare a total with a number you already trust.

Before any analysis, plot. In this block you will clean the data and, more importantly, notice **why** it needs cleaning by looking at charts.

In [2]:
```
import numpy as np
```

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
import plotly.graph_objects as go

sns.set_theme(style="whitegrid")
yearly = pd.read_csv('Data/countries_yearly.csv')
energy = pd.read_csv('Data/energy_mix.csv')
info = pd.read_csv('Data/country_info.csv')
print(yearly.shape, energy.shape, info.shape)
yearly.head()
```

```
(5568, 12) (14837, 4) (144, 3)
```

Out[2]:

	nation	year	iso3	population	gdp_usd	co2_mt	co2_per_person	coal_co2_mt	c
0	Kyrgyzstan	2017	KGZ	6220148.0	2.887968e+10	9.364	1.505	3.391	
1	Gambia	2015	GMB	2224530.0	3.514848e+09	0.583	0.262	NaN	
2	USA	2018	USA	335056492.0	1.812469e+13	5361.236	16.001	1224.380	
3	Indonesia	2008	IDN	240157902.0	1.809491e+12	365.384	1.521	121.845	
4	Mexico	1991	MEX	84430565.0	8.651487e+11	329.831	3.907	15.510	

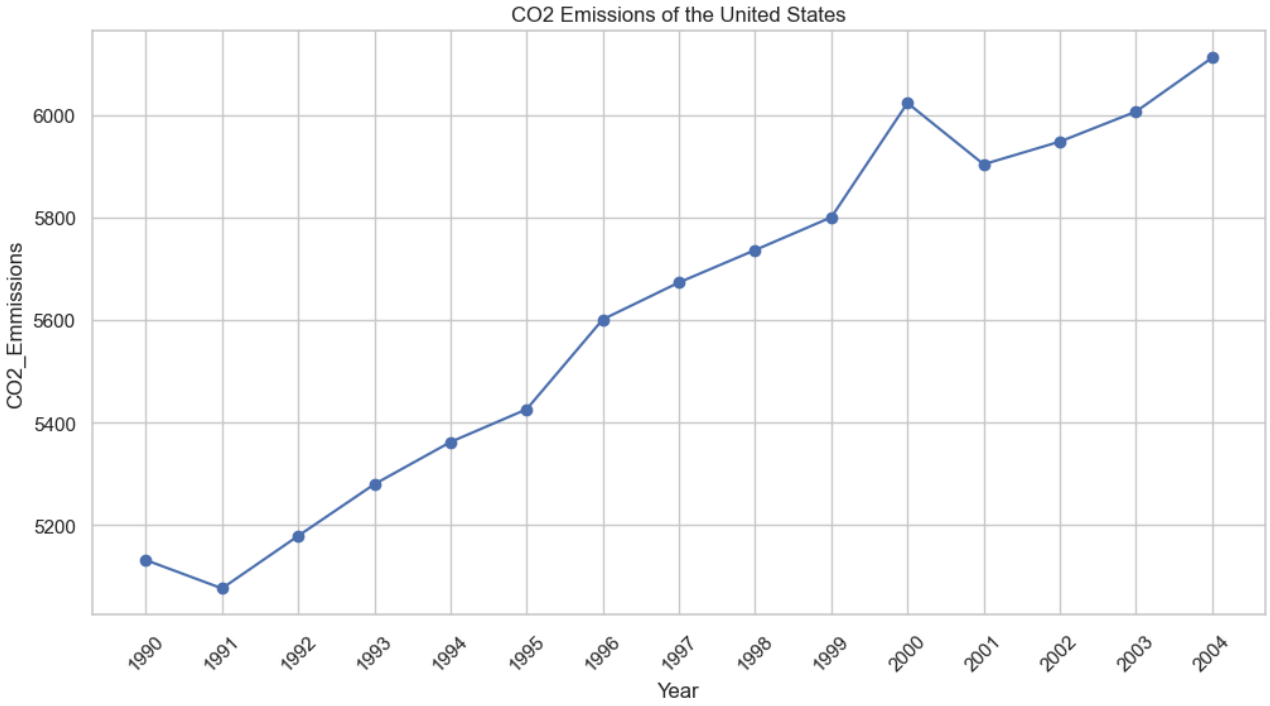
Exercise 1 – First chart

A line that stops. Plot CO2 emissions (`co2_mt` , megatonnes) of `nation == 'United States'` against `year` as a line, using `fig, ax = plt.subplots()` . Sort by year first, add a title, axis labels, and rotate the x tick labels by 45°. Use the data exactly as loaded (matplotlib draws text values as categories in the order they appear, so sort first). Then answer in a comment: **what is wrong with this chart?** Name one problem you can see in the chart, and one more that you can find with `yearly.dtypes` .

In [3]:

```
# Write your code here
fig, ax = plt.subplots(figsize=(12,6))
year_usa = yearly[yearly['nation']=='United States'].copy()
year_usa = year_usa.sort_values('year')
ax.plot(year_usa['year'],year_usa['co2_mt'],marker="o")
plt.title(label='CO2 Emissions of the United States')
plt.xlabel('Year')
plt.ylabel('CO2_Emmissions')
plt.xticks(rotation=45)
```

Out[3]:
```
([0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14],
 [Text(0, 0, '1990'),
  Text(1, 0, '1991'),
  Text(2, 0, '1992'),
  Text(3, 0, '1993'),
  Text(4, 0, '1994'),
  Text(5, 0, '1995'),
  Text(6, 0, '1996'),
  Text(7, 0, '1997'),
  Text(8, 0, '1998'),
  Text(9, 0, '1999'),
  Text(10, 0, '2000'),
  Text(11, 0, '2001'),
  Text(12, 0, '2002'),
  Text(13, 0, '2003'),
  Text(14, 0, '2004')])
```



Hint 1 Look at the first and the last x tick. Does the data end where you expect? **Hint 2** `yearly['nation'].unique()` – search for other spellings of the same country. **Hint 3** `yearly.dtypes` – is `year` a number?

Exercise 2 – Cleaning the year column

Make `year` numeric, then drop duplicates. The suffix `p` marks provisional 2023 values. In `yearly`, in this order: (1) add a boolean column `provisional` (True where the year text ends with `p`) **before** touching `year`, (2) turn `year` into an integer, (3) remove exact duplicate rows (reset the index). Print how many provisional rows remain **after** the duplicates are

removed, how many duplicates you removed, and the new number of rows.

In [4]:
```
yearly['year'] = yearly['year'].astype(str)
yearly['provisional'] = yearly['year'].str.contains('p', na=False)
yearly['year'] = yearly['year'].str.replace('p', '', regex=False).astype(
```

In [5]:
```
# Write your code here
count_before = len(yearly)
yearly.drop_duplicates(inplace=True)
count_after = len(yearly)
print(f"Provisional Rows {len(yearly[yearly['provisional']==True])}| Dupl
```

```
Provisional Rows 102| Duplicates removed 60| Rows now: 5508
```

Hint 1 `.str.endswith('p')` and `.str.rstrip('p')` work on a string column. **Hint 2** Create `provisional` **before** converting `year`, otherwise the marker is lost. **Hint 3** `duplicated()` then `drop_duplicates()`.

Exercise 3 – One name per country

One name per country. The line in Exercise 1 stopped because of how names are stored. Make every country have exactly one name: (1) for each `iso3` code, find its most frequent spelling (`name_by_iso`); (2) build a dictionary `rename_map` {other spelling: most frequent spelling} for every name that differs; (3) apply it to `yearly['nation']` **and** to `energy['nation']` (the energy table has no `iso3`). Print `rename_map` , and check that every name in `energy` now exists in `yearly` . Use the most frequent spelling exactly, even if it looks odd (for example `'Korea, South'`), because later exercises use these names (for example `'USA'`). This is a practical choice, not a rule: in real work you would ask the data owner for the official name.

In [6]:
```
# Write your code here
name_by_iso = yearly.groupby('iso3')['nation'].agg(lambda s: s.value_coun
unique_combos = yearly.dropna(subset=['iso3'])[['iso3', 'nation']].drop_d
unique_combos['freq_name'] = unique_combos['iso3'].map(name_by_iso)
differing = unique_combos[unique_combos['nation'] != unique_combos['freq_
rename_map = dict(zip(differing['nation'], differing['freq_name']))

# Strip potential trailing/leading whitespace from nation columns
yearly['nation'] = yearly['nation'].str.strip()
energy['nation'] = energy['nation'].str.strip()

# Apply renaming
yearly['nation'] = yearly['nation'].replace(rename_map)
energy['nation'] = energy['nation'].replace(rename_map)

print(rename_map)
```

```
print("Are all energy names in yearly?", energy['nation'].isin(yearly['na
```

```
{'United States': 'USA', 'South Korea': 'Korea, South', 'Czechia': 'Czech
Republic'}
Are all energy names in yearly? True
```

Hint 1 `groupby('iso3')['nation']` and `value_counts().index[0]` give the most frequent name. **Hint 2** Compare each row's name with `iso3.map(name_by_iso)` to find the rows that differ. **Hint 3** `Series.replace(dict)` applies a mapping and leaves other values untouched.

Exercise 4 – Splitting the table

Countries and aggregates. Some rows are not countries. Find out how to recognise them, then split `yearly` into `countries` (with `iso3`) and `aggregates` (without), both with `.copy()`. Print the sorted list of aggregate names, and how many distinct countries there are.

In [7]:
```
# Recognise incorrect countries
yearly[yearly["iso3"].isna()] # Wherever there is NaN value in iso3 the n
```

Out[7]:

	nation	year	iso3	population	gdp_usd	co2_mt	co2_per_person	coal_co2_mt
41	Oceania	2023	NaN	4.556314e+07	NaN	433.177	9.562	147.462
44	Europe	2023	NaN	7.469662e+08	NaN	4887.137	6.546	1054.397
47	Asia	2015	NaN	4.482088e+09	NaN	19070.406	4.255	10642.819
49	European Union (27)	1994	NaN	4.251890e+08	NaN	3589.454	8.442	1363.125
55	Lower-middle-income countries	2001	NaN	2.169846e+09	NaN	1848.146	0.852	671.004
...	...	...	...	...	...	...	...	...
5508	Asia	2007	NaN	4.090564e+09	NaN	14118.133	3.451	7664.360
5525	Lower-middle-income countries	2010	NaN	2.550520e+09	NaN	2838.991	1.113	1146.915
5540	High-income countries	2022	NaN	1.410112e+09	NaN	14274.797	10.153	3086.188
5554	Low-income countries	2019	NaN	6.598753e+08	NaN	214.494	0.325	71.232
5566	Oceania	2013	NaN	3.923394e+07	NaN	445.004	11.422	183.846

408 rows × 13 columns

In [8]:
```
# Copy data
countries = yearly[yearly["iso3"].notna()].copy()
aggregates = yearly[yearly["iso3"].isna()].copy()

aggregates_sorted = sorted(aggregates["nation"].unique())
print(aggregates_sorted)
print(f"Countries: {countries["nation"].nunique()} | aggregate series: {l
```

```
['Africa', 'Asia', 'Europe', 'European Union (27)', 'High-income countrie
s', 'Low-income countries', 'Lower-middle-income countries', 'North Ameri
ca', 'Oceania', 'South America', 'Upper-middle-income countries', 'Worl
d']
Countries: 150 | aggregate series: 12
```

Hint 1 Aggregates such as `World` or `Asia` have no ISO code. **Hint 2** `isna()` / `notna()` on the `iso3` column.

Exercise 5 – How complete is the data?

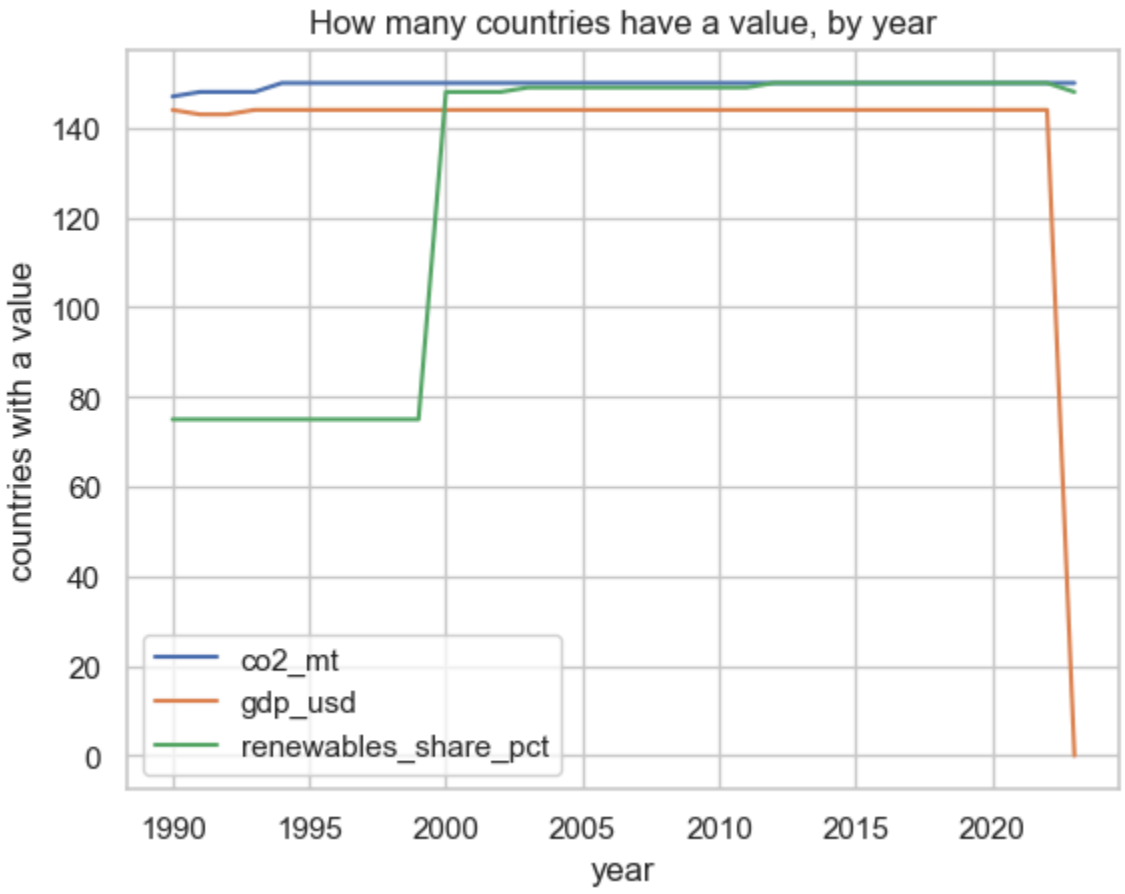
How complete is the data? Using `countries`, count for every year how

many countries have a value in `co2_mt`, `gdp_usd` and `renewables_share_pct`, and draw the three counts as three lines in one chart (legend, axis labels, title). Print the three counts for 1990 and for 2019. In a comment: which of the three variables would you trust least for a trend that starts in 1990, and why?

In [9]:
```
grouped_by_year = countries.groupby("year")[["co2_mt", "gdp_usd", "renewa

plt.plot(grouped_by_year.index, grouped_by_year)
plt.legend(grouped_by_year.columns)
plt.xlabel("year")
plt.ylabel("countries with a value")
plt.title("How many countries have a value, by year")
plt.show()


# I would trust renewables_share_pct the least since it starts very low a
```



Hint 1 `df.groupby('year')[[cols]].count()` counts the non-missing values per column and year. **Hint 2** `cov.plot(ax=ax)` draws one line per column. **Hint 3** Self-check: your chart has three lines; one of them starts clearly lower than the other two.

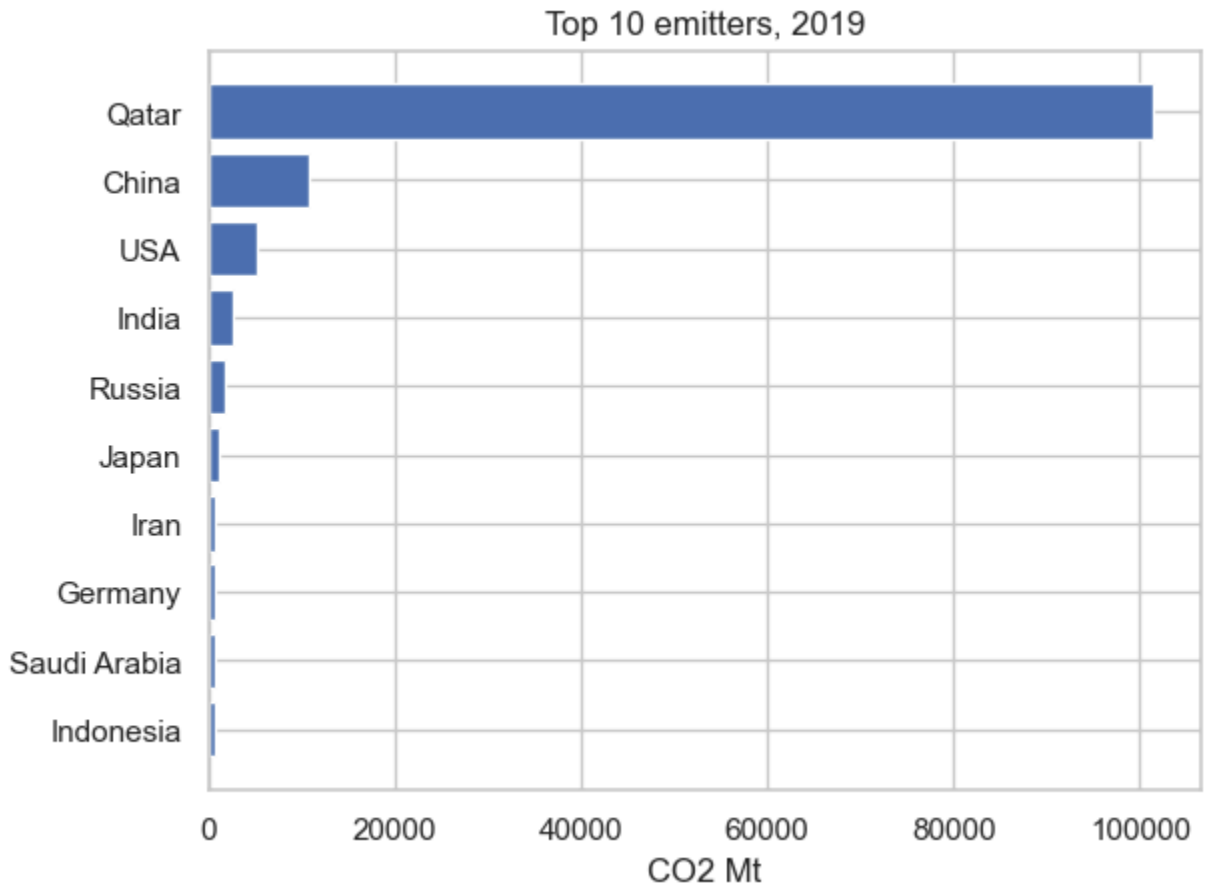
Exercise 6 – The ten largest CO2 emitters, 2019

A chart that finds a unit error. (a) Draw a horizontal bar chart (`ax.barh`) of the 10 countries with the highest `co2_mt` in 2019 (from `countries`). One bar is absurd. (b) Prove it with two printed checks: that country's `co2_mt` compared with the `World` row of `aggregates` , and the per-person value implied by `co2_mt * 1e6 / population` compared with the reported `co2_per_person` . (c) That country reports in a different unit from all the others (in the four columns `co2_mt` , `coal_co2_mt` , `oil_co2_mt` , `gas_co2_mt`). Correct them **in `countries`** and draw the chart again. Run this cell once: running it twice corrects the values twice.

In [10…

```
# (a)
# Filter top 10 co2 emitters of 2019
top_ten2019_co2 = (countries[countries["year"] == 2019]).sort_values(by="

# Print barplot
plt.barh(top_ten2019_co2["nation"], top_ten2019_co2["co2_mt"])
plt.title("Top 10 emitters, 2019")
plt.xlabel("CO2 Mt")
plt.gca().invert_yaxis()
```



In [11…

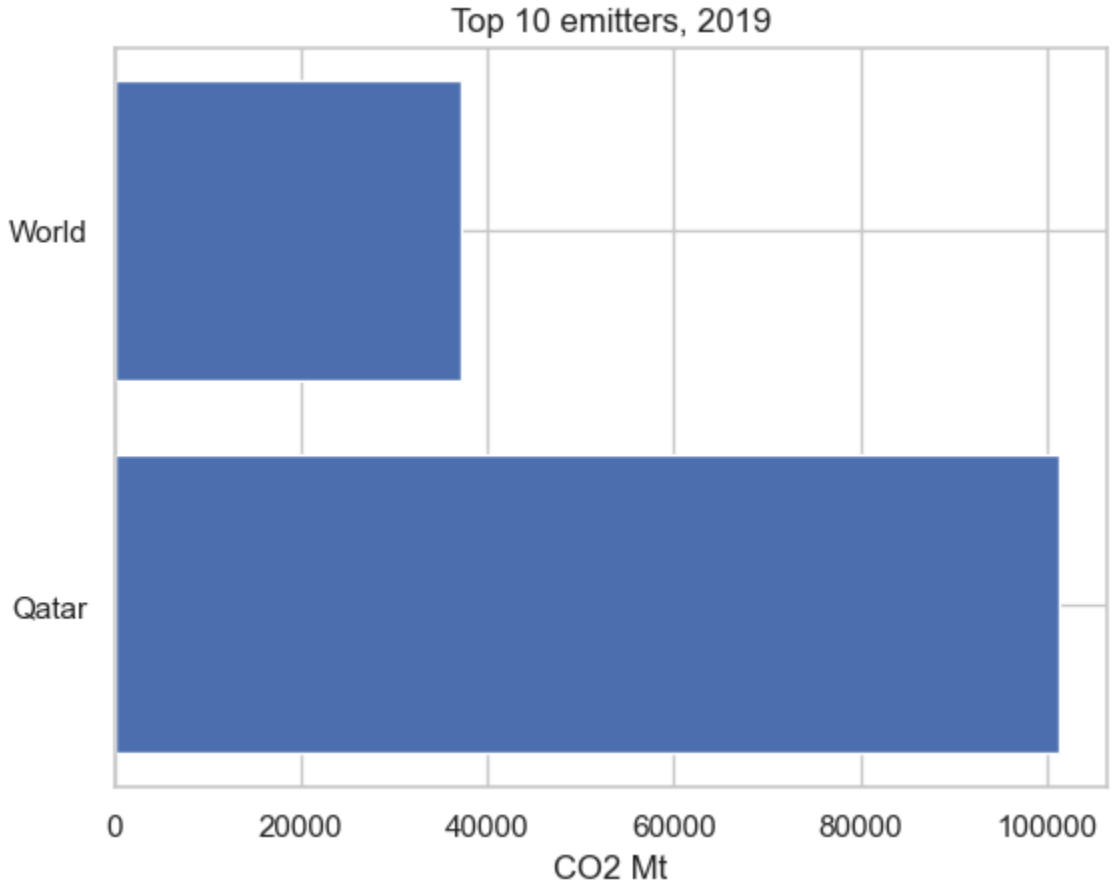
```
# (b)
# Filter only "World" row in 2019
aggregates_world = aggregates[(aggregates["nation"] == "World") & (aggreg

# Print barplot
plt.barh(["World", "Qatar"], [aggregates_world["co2_mt"].iloc[0], top_ten
plt.title("Top 10 emitters, 2019")
plt.xlabel("CO2 Mt")
plt.gca().invert_yaxis()

# PP value
print(f"Calculated per-person value: {top_ten2019_co2.iloc[0]["co2_mt"] *

# The calculated pp value is 1000 times bigger than the written pp value
```

```
Calculated per-person value: 36251.83930817351 | Written per-person valu
e: 36.252
```

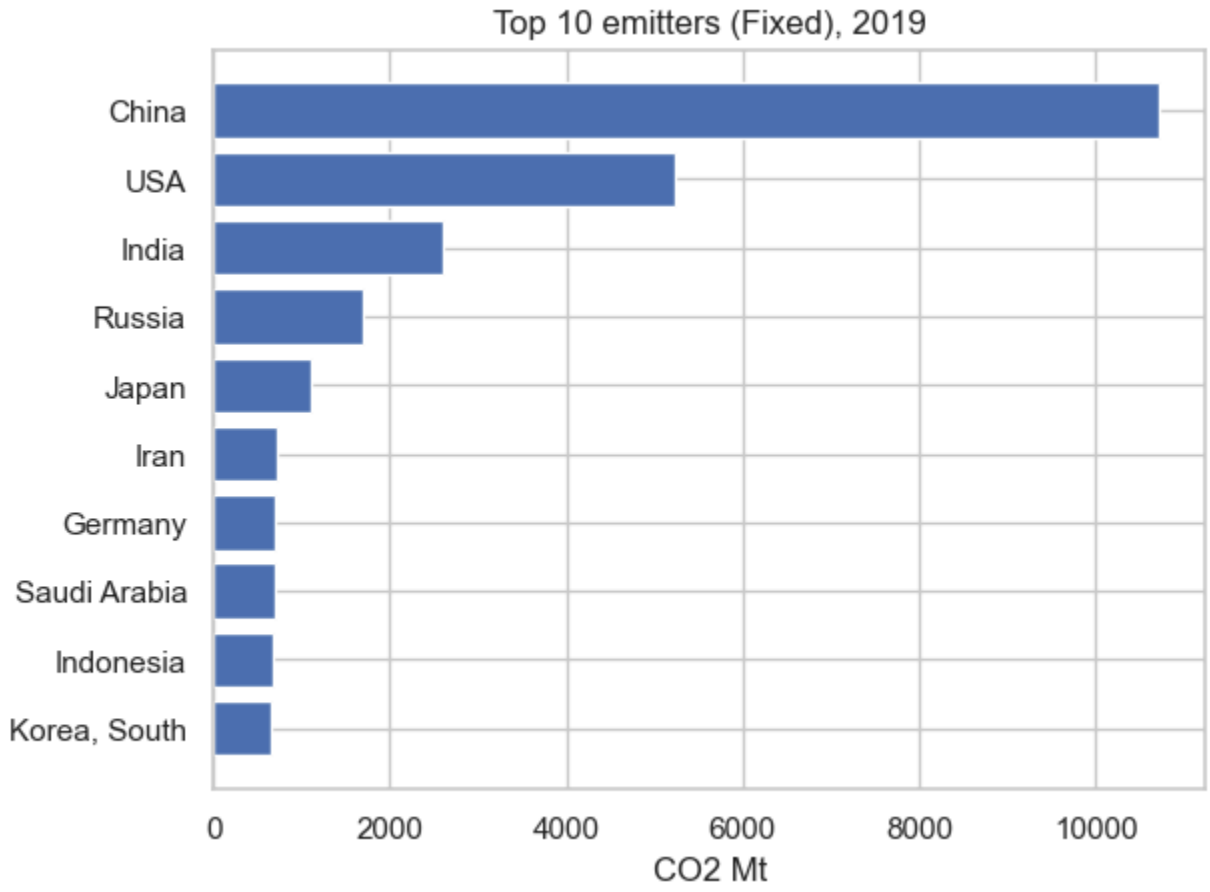


In [12…

```
# (c)
# Divide by 1000
countries.loc[countries["nation"] == "Qatar", ["co2_mt", "coal_co2_mt", "


# Filter top 10 co2 emitters of 2019
top_ten2019_co2 = (countries[countries["year"] == 2019]).sort_values(by="
```

```
# Print barplot
plt.barh(top_ten2019_co2["nation"], top_ten2019_co2["co2_mt"])
plt.title("Top 10 emitters (Fixed), 2019")
plt.xlabel("CO2 Mt")
plt.gca().invert_yaxis()
```



Hint 1 `nlargest(10, 'co2_mt')` then `sort_values` so the biggest bar ends up on top in `barh`. **Hint 2** A single country cannot emit more than the whole world. The implied per-person value should match the reported one – by what factor does it not? **Hint 3** Assignment on a subset of rows and columns: `df.loc[mask, columns] = ...`.

Block 1 – Matplotlib: who emits, and how much?

The Minister's first question: show me the emitters. Everything in matplotlib hangs on two objects: the **Figure** (the canvas) and the **Axes** (one plot on it). Work with `fig, ax = plt.subplots(...)` and call methods on `ax`; it scales from one chart to a dashboard.

In [13…
```
focus = ['USA', 'China', 'India', 'Germany', 'Israel']
palette = {'Africa': '#e15759', 'Americas': '#4e79a7', 'Asia': '#f28e2b',
tier_order = ['Low', 'Lower-middle', 'Upper-middle', 'High']
focus
```

Out[13]:
```
['USA', 'China', 'India', 'Germany', 'Israel']
```

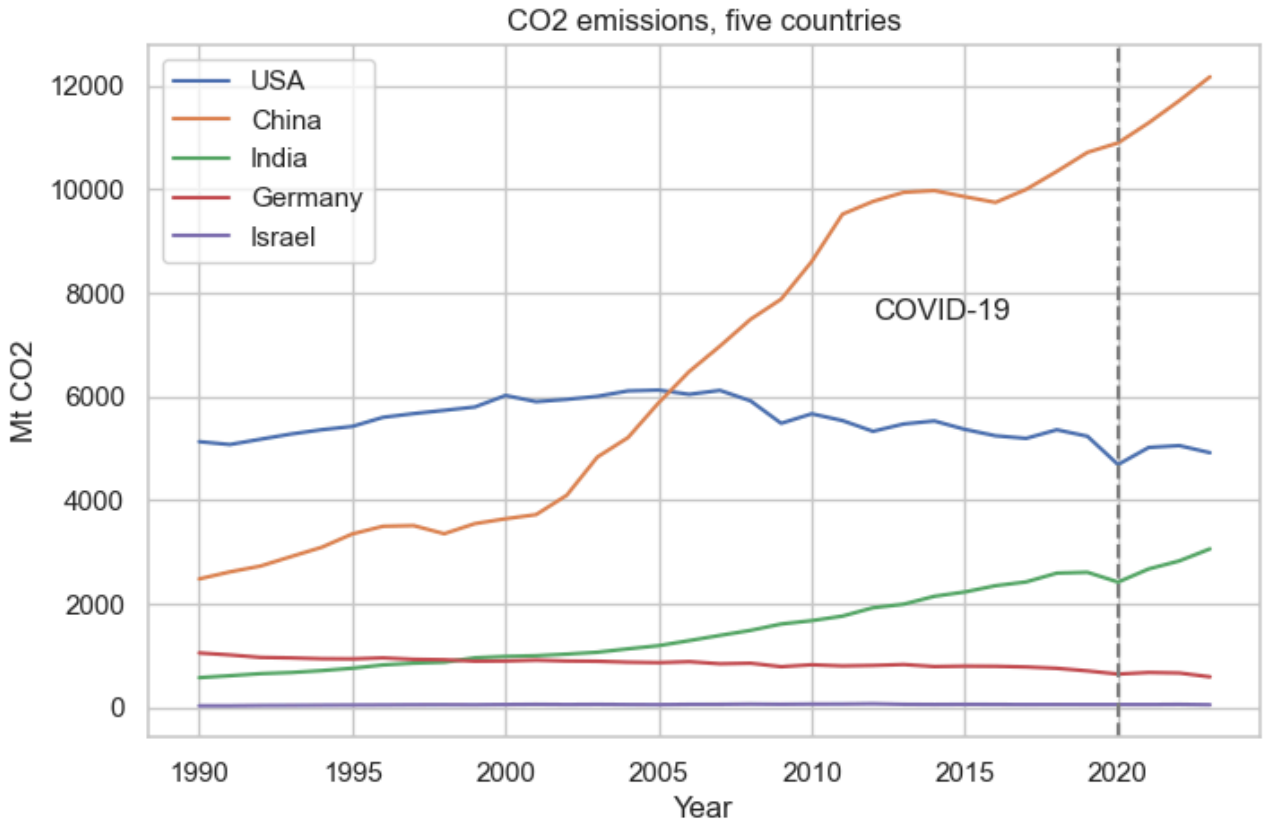
Exercise 7 – Five countries over time

Five countries over time. Pivot `countries` into a table `wide` (one column per country in `focus`, rows = year) and draw CO2 emissions (`co2_mt`) with one line per country, a legend, title and axis labels. Add a dashed vertical line at 2020 with the annotation `COVID-19`. Then print, for each country, the **percentage change between 2007 and 2020** (rounded to 1 decimal).

In [14…
```
# Creating pivot table
wide = countries.pivot(index="year", columns="nation", values="co2_mt")[f


fig, ax = plt.subplots(figsize=(8, 5))
ax.plot(wide.index, wide)
ax.set_title("CO2 emissions, five countries")
ax.legend(wide.columns)
ax.set_xlabel("Year")
ax.set_ylabel("Mt CO2")
ax.axvline(2020, linestyle="--", color="gray")
ax.annotate(
    "COVID-19",
    xy=(2020, 6000),
    xytext=(2012, 7500),
)
plt.show()

# Percentage change between 2007 and 2020
pct_change = ((wide.loc[2020] / wide.loc[2007] - 1) * 100).round(1)
print(pct_change)
```




```
nation
USA       -23.4
China      56.2
India      73.9
Germany   -23.5
Israel     -5.6
dtype: float64
```

Hint 1 `pivot(index='year', columns='nation', values='co2_mt')` gives one column per country. **Hint 2** `ax.axvline(x)` draws the vertical line; `ax.annotate(text, xy=..., xytext=..., arrowprops=...)` adds the label. **Hint 3** Percentage change: `(value_2020 / value_2007 - 1) * 100`.

Exercise 8 – Which scale?

Which scale? In one figure with two side-by-side Axes, draw the same five lines on a **linear** y axis (left) and a **logarithmic** y axis (right, `ax.set_yscale('log')`). Reuse `wide` from the previous exercise. Print the factor by which each country's CO2 grew between 1990 and 2019 (rounded to 1 decimal). In a comment: on which of the two scales can you see how fast Israel's emissions grew, and what does a straight line mean on a log axis?

Expected result (described, no picture or output is shown): One Figure

with two side-by-side Axes showing the same five lines, one with a linear y axis and one with a logarithmic y axis, titled accordingly; the table of 1990-to-2019 growth factors printed with one decimal; two comment lines answering the two questions.

In [15…

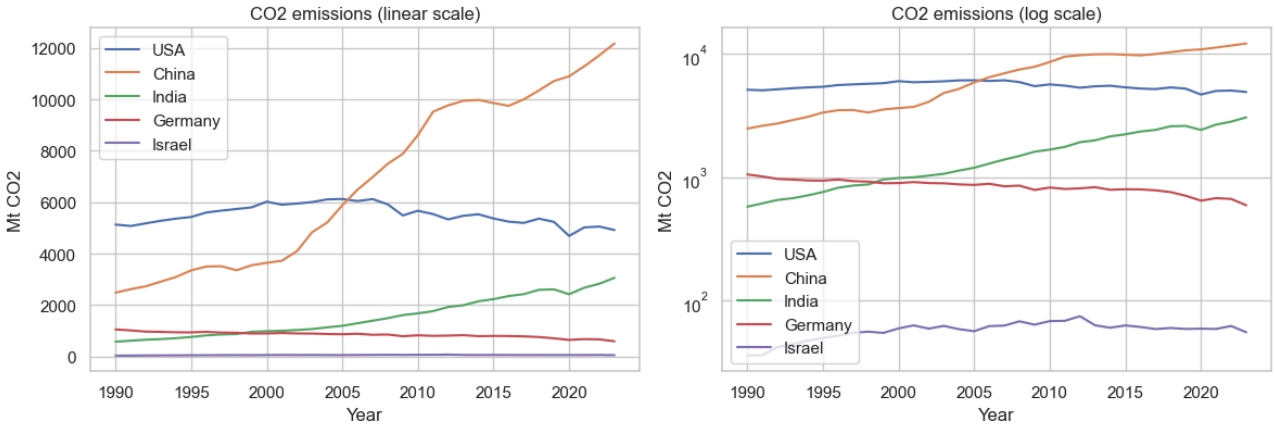
```python
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4.2))

for ax, scale in ((ax1, "linear"), (ax2, "log")):
    ax.plot(wide.index, wide)
    ax.set_yscale(scale)
    ax.set_title(f"CO2 emissions ({scale} scale)")
    ax.set_xlabel("Year")
    ax.set_ylabel("Mt CO2")
    ax.legend(wide.columns)

plt.tight_layout()
plt.show()

# Growth factor between 1990 and 2019
growth_factor = (wide.loc[2019] / wide.loc[1990]).round(1)
print(growth_factor)

# We can see how fast Israel's emissions grew on the logarithmic scale (o
# A straight line on a logarithmic axis means constant percentage (expone
```




```
nation
USA        1.0
China      4.3
India      4.5
Germany    0.7
Israel     1.6
dtype: float64
```

Hint 1 `plt.subplots(1, 2, ...)` returns two Axes. **Hint 2** `ax.set_yscale('log')`. **Hint 3** A straight line on a log axis means constant percentage growth. **Hint 4** Skeleton: `fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4.2))`, then `for ax, scale in ((ax1, 'linear'), (ax2, 'log')):` loop over `focus`, `ax.plot(wide.index, wide[n], label=n)` and `ax.set_yscale(scale)`. **Hint 5**

Self-check: your printed table has 5 numbers; one of them is below 1 (a country that shrank) and two are above 4.

Exercise 9 – Grouped bars and shares

Grouped bars and shares. For 2019 and the countries `['USA', 'China', 'India', 'Germany', 'Israel', 'Japan']`, draw CO2 from coal, oil and gas (`coal_co2_mt`, `oil_co2_mt`, `gas_co2_mt`) in **two panels** of one Figure: (1) **grouped** bars in Mt, (2) **stacked** bars as **percent of each country's coal + oil + gas total** (every bar reaches 100). Use `ax.bar` with `np.arange` offsets for the grouped version and `bottom=` for the stacked one. Print each country's coal share rounded to a whole number. In a comment: Israel is almost invisible in panel 1. Why, and which panel answers 'what is each country's fuel mix?'

In [16…

```
# Filtering needed countries in 2019
target_nations = ['USA', 'China', 'India', 'Germany', 'Israel', 'Japan']
fuels = ['coal_co2_mt', 'oil_co2_mt', 'gas_co2_mt']
UCIGIJ_countries_2019 = (countries[countries["year"] == 2019].set_index("

# Plotting graphs
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4.2))

x = np.arange(len(target_nations))
width = 0.25

# Panel 1: Grouped bars
for i, fuel in enumerate(fuels):
    ax1.bar(x + (i - 1) * width, UCIGIJ_countries_2019[fuel], width=width

ax1.set_xticks(x)
ax1.set_xticklabels(target_nations)
ax1.set_title("Grouped (Mt)")
ax1.set_ylabel("Mt CO2")
ax1.legend()


# Panel 2: Stacked bars
countries_shares = UCIGIJ_countries_2019.div(UCIGIJ_countries_2019.sum(ax
bottom = np.zeros(len(target_nations))

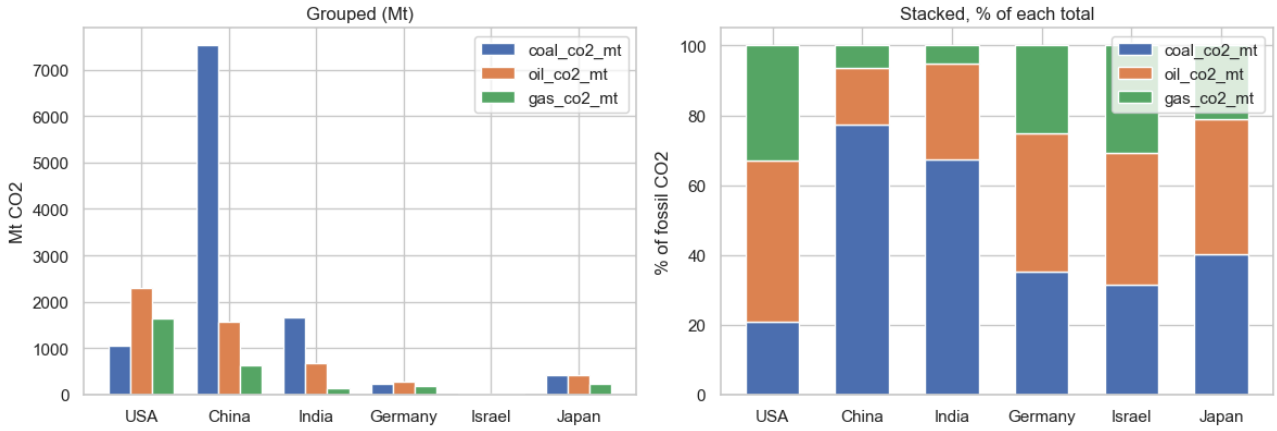
for i, fuel in enumerate(fuels):
    ax2.bar(x, countries_shares[fuel], width=0.6, bottom=bottom, label=fu
    bottom += countries_shares[fuel].values

ax2.set_xticks(x)
ax2.set_xticklabels(target_nations)
ax2.set_title("Stacked, % of each total")
ax2.set_ylabel("% of fossil CO2")
```

```
ax2.legend()


plt.tight_layout()
plt.show()


# Print each country's coal share rounded to a whole number
print(countries_shares["coal_co2_mt"].round(0))
```




```
nation
USA        21.0
China      77.0
India      67.0
Germany    35.0
Israel     32.0
Japan      40.0
Name: coal_co2_mt, dtype: float64
```

Hint 1 For grouped bars shift each series by `(i - 1) * width`; the three bars are centred on the tick. **Hint 2** For the stacked panel divide each row by its own total first (`df.div(df.sum(axis=1), axis=0) * 100`), then keep a running `bottom` array. **Hint 3** Round with `.round(0)`.

Exercise 10 – A bubble chart with four variables

A bubble chart with four variables. Build `m`: the 2019 rows of `countries` with non-missing `gdp_usd`, `population` and `co2_per_person`, joined to `info[['iso3', 'continent', 'gdp_tier']]` with a **left** join; fill missing `continent` with `'Unknown'`; add `gdp_pc = gdp_usd / population`. Draw a scatter (`ax.scatter`, one call per continent): x = `gdp_pc` on a log axis, y = `co2_per_person`, bubble size proportional to `population`, colour from `palette`. Annotate the three most populous countries and add a legend titled `Continent`. Rows with missing GDP, population or CO2 cannot be drawn, so fewer than 150 countries remain:

print how many are drawn and how many of them have no continent. (`info` does not list every country, which is why a continent can be missing; a left join keeps those countries.)

In [17…

```
# Creating m and merging with info
m = (
    countries[countries["year"] == 2019]
    .dropna(subset=["gdp_usd", "population", "co2_per_person"])
    .merge(info[["iso3", "continent", "gdp_tier"]], on="iso3", how="left"
)

# Filling continent nan values with 'Unknown'
m["continent"] = m["continent"].fillna('Unknown')

# Creating new column
m["gdp_pc"] = m["gdp_usd"] / m["population"]


# Plotting scatter plot
fig, ax = plt.subplots(figsize = (9, 5.5))

for cont, color in palette.items():
    sub = m[m["continent"] == cont]
    ax.scatter(
        sub["gdp_pc"],
        sub["co2_per_person"],
        s=sub["population"] / 2e6,  # proportional to population
        color=color,
        alpha=0.5,
        edgecolors="black",
        linewidth=0.5,
        label=cont,
    )

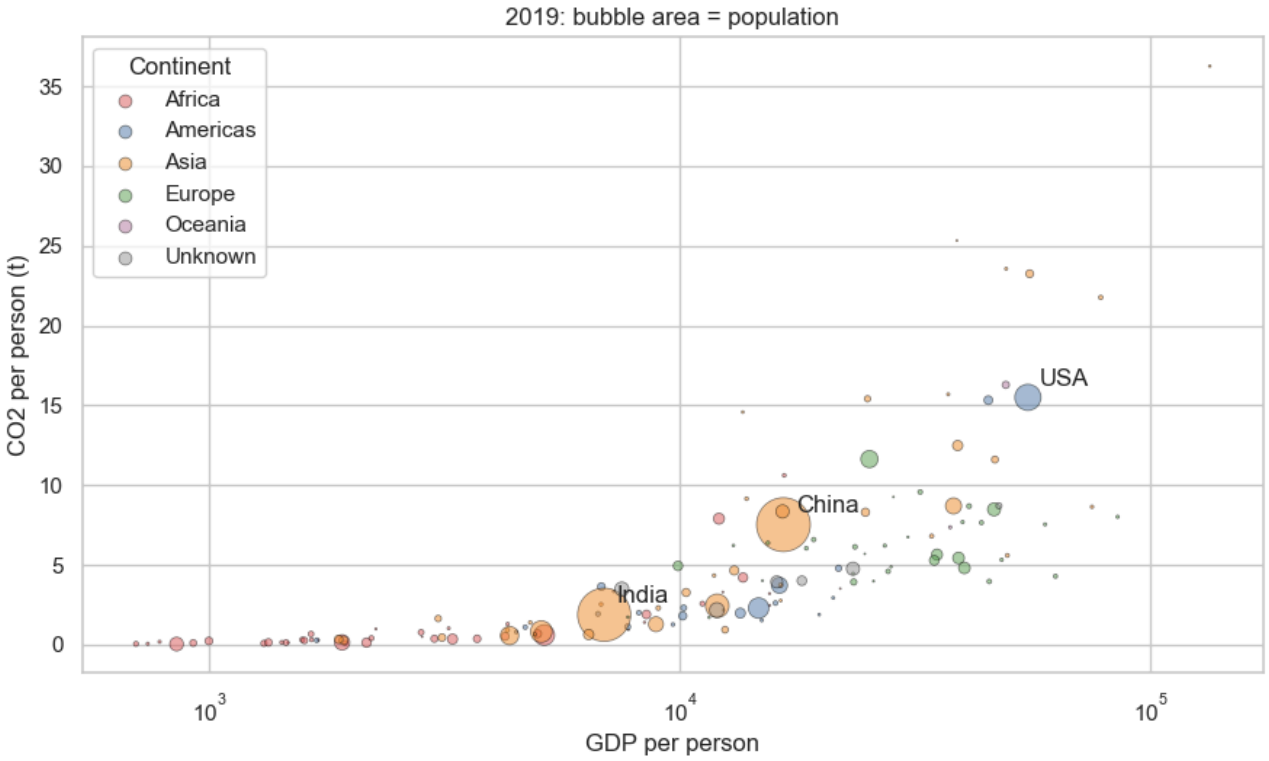
ax.set_xscale("log")
ax.set_title("2019: bubble area = population")
ax.set_xlabel("GDP per person")
ax.set_ylabel("CO2 per person (t)")

# Annotate the 3 most populous countries
for _, row in m.nlargest(3, "population").iterrows():
    ax.annotate(
        row["nation"],
        xy=(row["gdp_pc"], row["co2_per_person"]),
        xytext=(6, 6),
        textcoords="offset points",
    )

# Legend titled 'Continent' (with uniform marker sizes per Hint 3)
leg = ax.legend(title="Continent")
for h in leg.legend_handles:
    h.set_sizes([40])
```

```
plt.tight_layout()
plt.show()

# Print how many countries are drawn and how many have no continent
print(f"Countries drawn: {len(m)} | Unknown continent: {(m['continent'] =
```




```
Countries drawn: 144 | Unknown continent: 6
```

Hint 1 `dropna(subset=[...])`, then `merge(..., how='left')`. **Hint 2** `s=` in `ax.scatter` is the area of the marker: divide the population by a constant to get sensible sizes. **Hint 3** Legend marker sizes copy the bubble sizes: `leg.legend_handles` lets you reset them with `h.set_sizes([40])`.

Exercise 11 – A lollipop ranking

A lollipop ranking. From `m`, take the countries with a population above 5 million and draw the 15 with the highest `co2_per_person` as a lollipop chart: a thin horizontal line from 0 (`ax.hlines`) with a dot at the end (`ax.scatter`), sorted so the highest is on top, with the value written next to each dot. Print the top 3 with one decimal. In a comment: why filter by population?

In [18…

```
m_above_5m_largest = m[m["population"] > 5000000].nlargest(15, "co2_per_p

fig, ax = plt.subplots(figsize = (9, 6))
```

```python
# Create dots
ax.scatter(m_above_5m_largest["co2_per_person"], m_above_5m_largest["nati

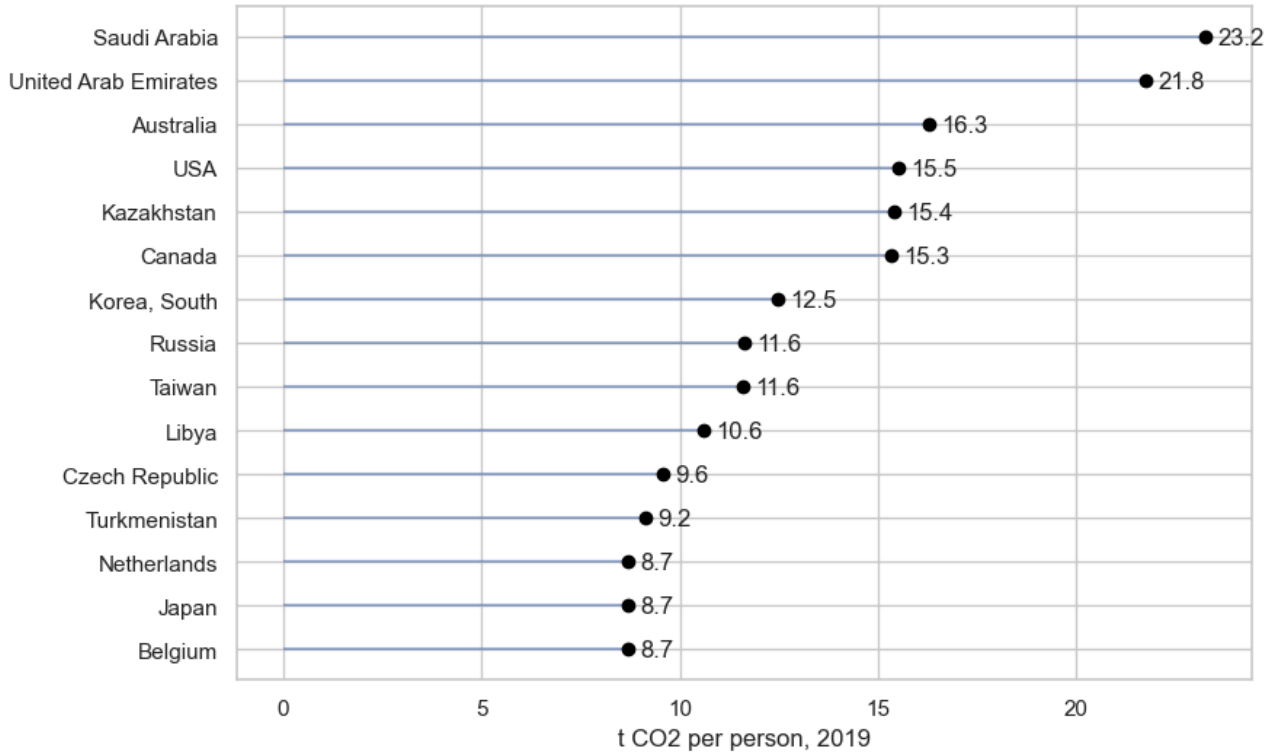
# Create lines to each dot
ax.hlines(m_above_5m_largest["nation"], 0, m_above_5m_largest["co2_per_pe

# Write the value next to each dot
for val, nation in zip(m_above_5m_largest["co2_per_person"], m_above_5m_l
    ax.text(val + 0.3, nation, f"{val:.1f}", va="center")

ax.set_xlabel("t CO2 per person, 2019")
plt.gca().invert_yaxis()
plt.show()


# Print the top 3 with one decimal
print(m_above_5m_largest.set_index("nation")["co2_per_person"].head(3).ro

# We filter by nation because very small states or island nations (e.g.,
# due to large oil/gas/industrial output divided by a tiny population, -
# dominating the ranking even though their total global impact is small.
```




```
nation
Saudi Arabia            23.2
United Arab Emirates    21.8
Australia               16.3
Name: co2_per_person, dtype: float64
```

Hint 1 `ax.hlines(y_labels, 0, values)` and `ax.scatter(values, y_labels)` share the same y labels. **Hint 2** Sort ascending before plotting: the first row is

drawn at the bottom.

Exercise 12 – Two axes, one story

Two axes, one story. For Israel (`countries` , sorted by year) draw total `co2_mt` on the left axis and `co2_per_person` on the right axis (`ax.twinx()`) with axis labels coloured like their lines. Print the percentage change between 1990 and 2022 of the total, of the per-person value and of the population. In a comment: how can both the total go up and the per-person value go down, and what is the danger of a chart with two y axes?

In [19…

```
# Write your code here
israel_masked = countries[countries["nation"] == "Israel"].sort_values(by

fig, ax1 = plt.subplots()

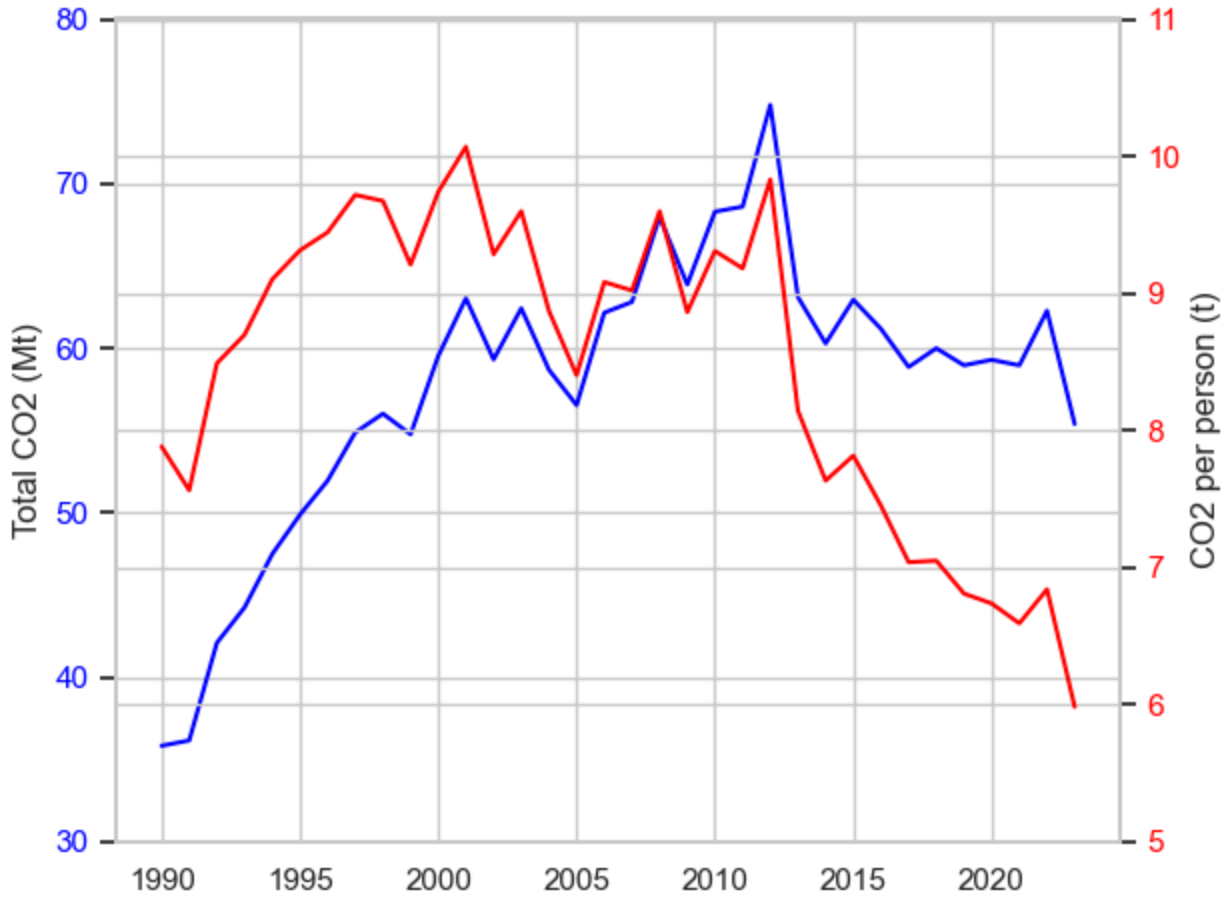
# Co2 mt plot
ax1.plot(israel_masked["year"], israel_masked["co2_mt"], color = "blue")
ax1.set_ylabel("Total CO2 (Mt)")
ax1.tick_params(axis='y', labelcolor = "blue")
ax1.set_ylim(30, 80)

# Co2 pp plot
ax2 = ax1.twinx()
ax2.plot(israel_masked["year"], israel_masked["co2_per_person"], color =
ax2.set_ylabel("CO2 per person (t)")
ax2.tick_params(axis='y', labelcolor="red")
ax2.set_ylim(5.0, 11.0)

fig.tight_layout()
plt.show()


# Percentage change between 1990 and 2022
isr_idx = israel_masked.set_index("year")
cols = ["co2_mt", "co2_per_person", "population"]
print(((isr_idx.loc[2022, cols] / isr_idx.loc[1990, cols] - 1) * 100).rou

# Israel's population grew faster than its total CO2 emissions thus effec
# The danger of two y axes is that each axis has its own scale, which can
```




```
co2_mt             73.813644
co2_per_person    -13.219995
population        100.289395
dtype: object
```

Hint 1 `ax2 = ax1.twinx()` shares the x axis. **Hint 2** Percentage change: `value_2022 / value_1990 - 1`.

Exercise 13 – Stacked areas over time

Stacked areas over time. For the USA draw CO2 from coal, oil and gas (`coal_co2_mt` , `oil_co2_mt` , `gas_co2_mt`) as a stacked area chart over the years (`ax.stackplot`) with a legend. Print each fuel's share (%, 1 decimal) in 1990 and in 2022.

In [20…

```
# Stacked areas
usa_masked = countries[countries["nation"] == "USA"].sort_values(by="year
cols = ["Coal", "Oil", "Gas"]

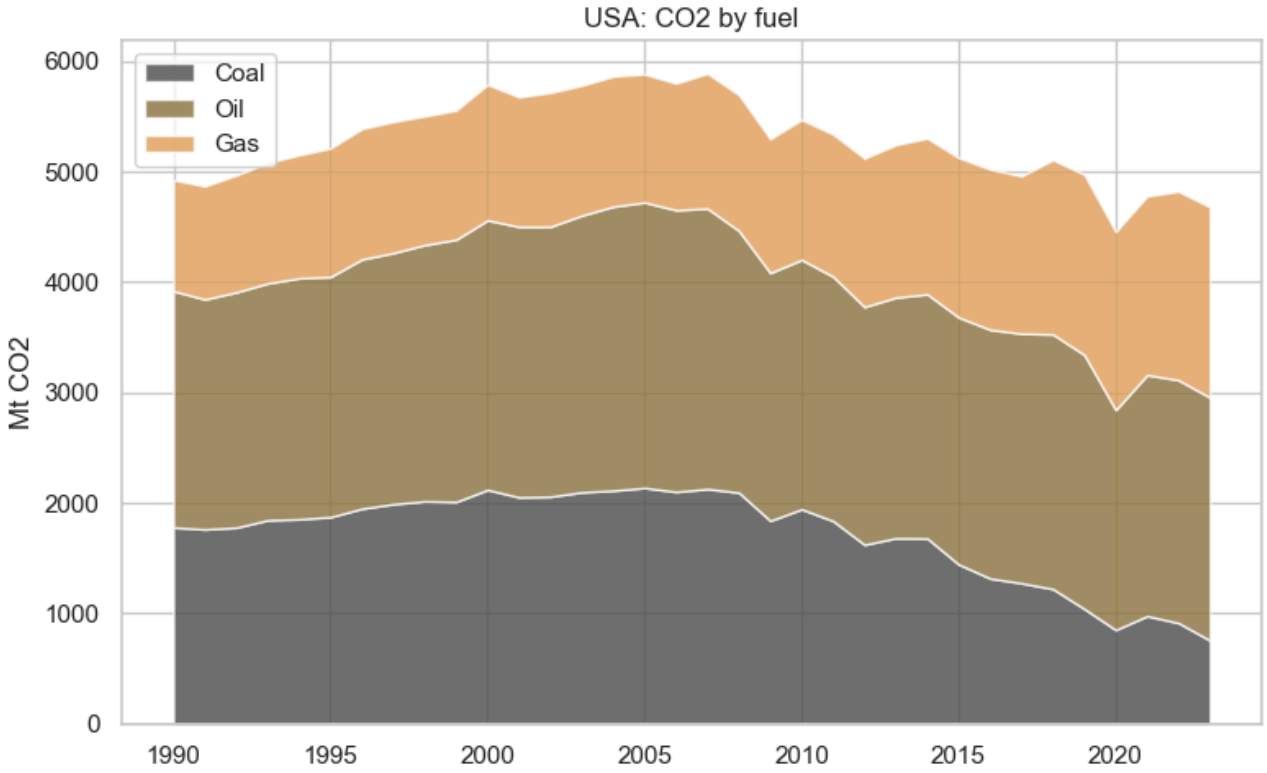
fig, ax = plt.subplots(figsize=(8, 5))

ax.stackplot(usa_masked["year"], usa_masked["coal_co2_mt"], usa_masked["o
```

```
ax.set_title("USA: CO2 by fuel")
ax.set_ylabel("Mt CO2")
ax.legend(loc="upper left")

plt.tight_layout()
plt.show()

# Fuel share table
fuel_share = usa_masked[usa_masked["year"].isin([1990, 2022])].set_index(
fuel_share = (fuel_share.div(fuel_share.sum(axis=1), axis=0) * 100).round
fuel_share
```



Out[20]:

year	coal_co2_mt	oil_co2_mt	gas_co2_mt
1990	36.2	43.4	20.4
2022	19.0	45.6	35.4

Hint 1 `ax.stackplot(x, y1, y2, y3, labels=[...])`. **Hint 2** Shares: divide each row by its own sum.

Exercise 14 – A small dashboard

A small dashboard. For Germany (from `countries`, sorted by year) build a 2×2 grid of line charts that share the x axis (`sharex=True`): `co2_mt`, `co2_per_person`, `energy_twh`, `renewables_share_pct`. Give each panel

its own title, add `fig.suptitle`, and finish with `fig.tight_layout()`. Mark provisional values (column `provisional`) with a hollow red circle. Print Germany's change in `co2_mt` between 1990 and 2022 as a percentage.

In [21…

```
germany_masked = countries[countries["nation"] == "Germany"].sort_values(

# Provisional year masked
prov = germany_masked[germany_masked["provisional"]]

# Used columns and titles
cols = ["co2_mt", "co2_per_person", "energy_twh", "renewables_share_pct"]
titles = ["CO2 (Mt)", "CO2 per person (t)", "Primary energy (TWh)", "Rene

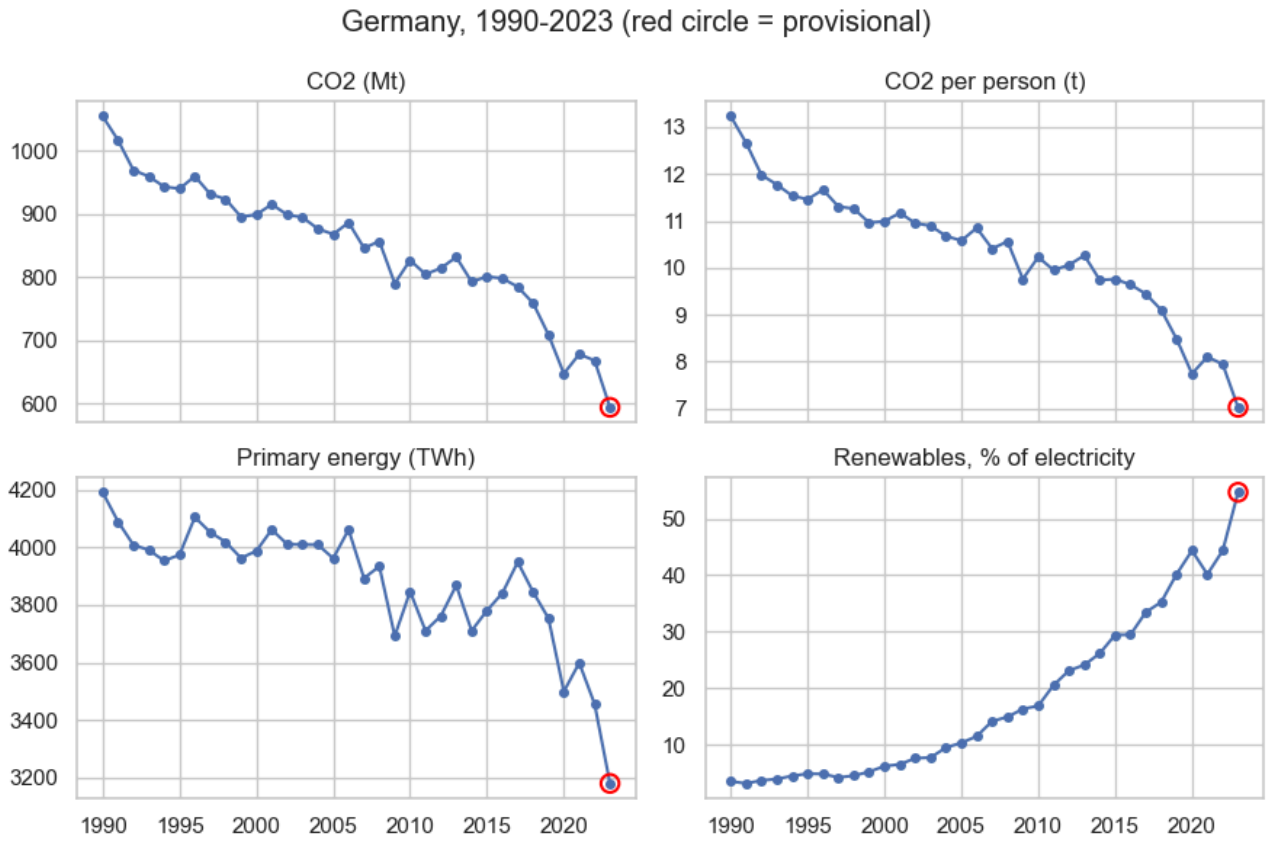
# Plotting
fig, axes = plt.subplots(2, 2, sharex = True, figsize = (9,6))

for ax, col, title in zip(axes.ravel(), cols, titles):
    # Plotting the main graphs
    ax.plot(germany_masked["year"], germany_masked[col], marker="o", mark

    # Printing the red circle around the provisional year
    ax.scatter(prov["year"], prov[col], facecolors="none", edgecolors="re
    ax.set_title(title)


fig.suptitle("Germany, 1990-2023 (red circle = provisional)")
fig.tight_layout()
plt.show()

germany_masked = germany_masked.set_index("year")
print("Germany CO2 change from 1990 to 2022: {}%".format(((germany_masked
```




```
Germany CO2 change from 1990 to 2022: -36.7%
```

Hint 1 `plt.subplots(2, 2, sharex=True)` returns a 2×2 array of Axes; `axes.ravel()` flattens it. **Hint 2** Loop over a list of `(column, title)` pairs.

Block 2 – Seaborn: why do countries differ?

Now the analysis: groups, distributions, relationships. Seaborn works on whole tables, and you must know what it computes for you. **Seaborn in four lines:** functions take `data=` and column names (`x=` , `y=` , `hue=`); `hue` splits by colour; many functions **aggregate for you** (for example `lineplot` draws a mean with a confidence band) unless you pass `estimator=None` ; `relplot` / `catplot` can make one panel per group (`col=`).

Seaborn works on whole DataFrames: you say *which columns*, it handles grouping, colours and statistics. The price: you must know what it computes for you.

Exercise 15 – Box plus points

Box plus points. Build `d19` : the 2019 rows of `countries` joined (left) to

`info[['iso3', 'continent', 'gdp_tier']]`, with missing `continent` set to `'Unknown'` and a column `gdp_pc = gdp_usd / population`. Draw `co2_per_person` by continent: a box plot (`showfliers=False`, ordered by the median) with a strip plot on top showing every country. Print the median and the inter-quartile range (IQR) per continent.

Expected result (described, no picture or output is shown): A box plot per continent ordered by median, with every country drawn on top as a dot, plus a table with the median and the IQR per continent, sorted by median.

In [22…

```
# Merged data
d19 = pd.merge(countries[countries["year"] == 2019], info[['iso3', 'conti

# Filling continent nan values with 'Unknown'
d19["continent"] = d19["continent"].fillna('Unknown')

# Creating new column
d19["gdp_pc"] = d19["gdp_usd"] / d19["population"]

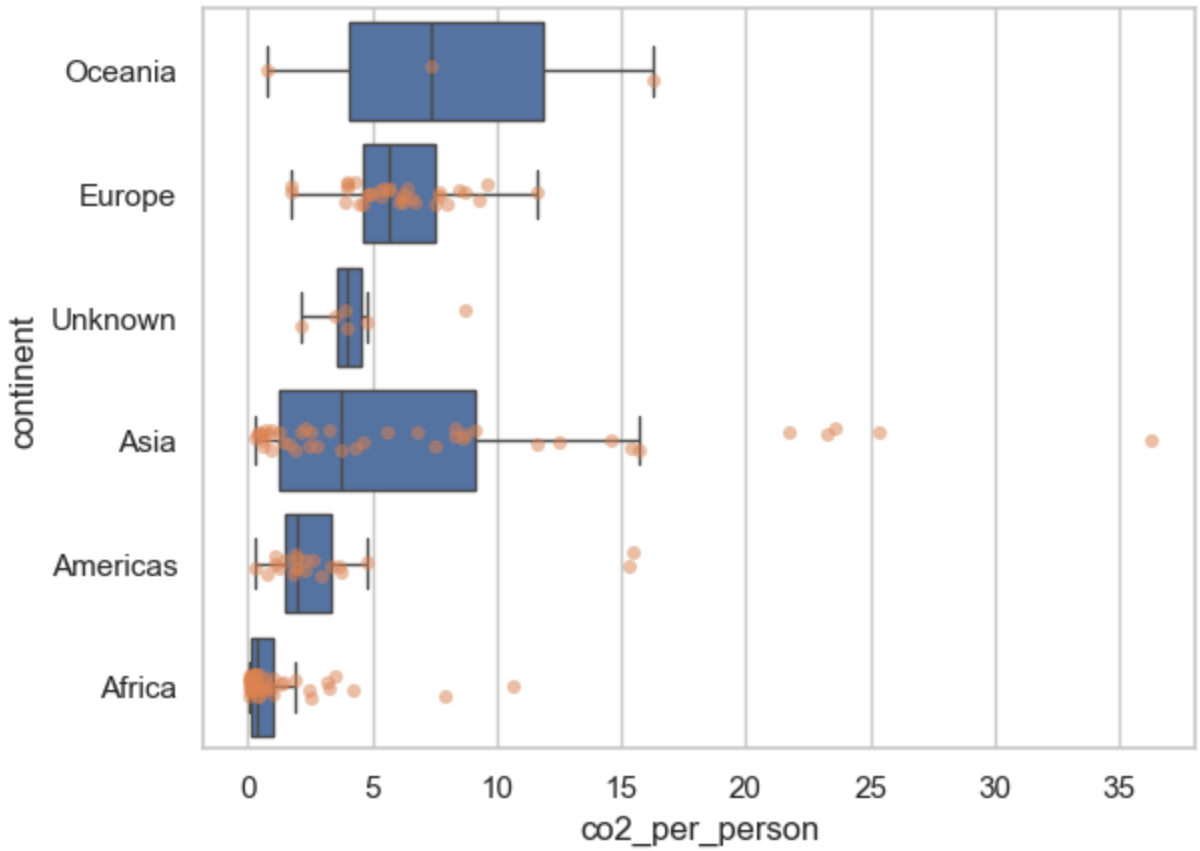
# Plotting
fig, ax = plt.subplots()

order_ = d19.groupby("continent")["co2_per_person"].median().sort_values(
sns.boxplot(data=d19, x='co2_per_person', y='continent' , showfliers=Fals
sns.stripplot(data=d19, x='co2_per_person', y='continent', order=order_,
plt.gca().invert_yaxis()

# Table
d19.groupby("continent")["co2_per_person"].agg(median = "median", iqr =la
```

Out[22]:

continent	median	iqr
Africa	0.3755	0.875
Americas	2.0070	1.811
Asia	3.7340	7.863
Unknown	3.9765	0.971
Europe	5.6930	2.940
Oceania	7.3580	7.747



Hint 1 `order=` takes the category order; `groupby(...).median().sort_values().index` produces it. **Hint 2** Boxes hide the individual points: draw them again with `stripplot` on the same Axes. **Hint 3** `s.quantile(.75) - s.quantile(.25)` is the IQR. **Hint 4** Skeleton: `fig, ax = plt.subplots()`; `sns.boxplot(data=d19, x=..., y=..., order=order, showfliers=False, ax=ax)`; `sns.stripplot(data=d19, x=..., y=..., order=order, ax=ax)`; the table with `groupby('continent')[col].agg(median='median', iqr=...)`. **Hint 5** Self-check: your table has 6 rows (five continents and `Unknown`), sorted by median, and Africa has the lowest median.

Exercise 16 – Distribution by group

Distribution by group. Using `d19`, two panels in one Figure (sharing nothing): (left) a stacked histogram of `co2_per_person` by continent on a **log x axis** (`sns.histplot`, `hue='continent'`, `multiple='stack'`, `log_scale=True`, 25 bins, `palette=palette`); (right) the empirical cumulative distribution (`sns.ecdfplot`, **linear** x axis) by continent with a dashed horizontal line at 0.5. Print, for each continent, the share of its countries that emit **less than 2 tonnes per person**.

In [23…

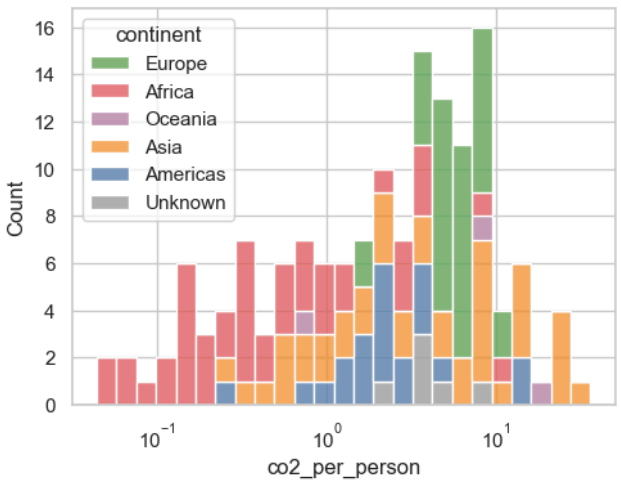
```
# Plotting

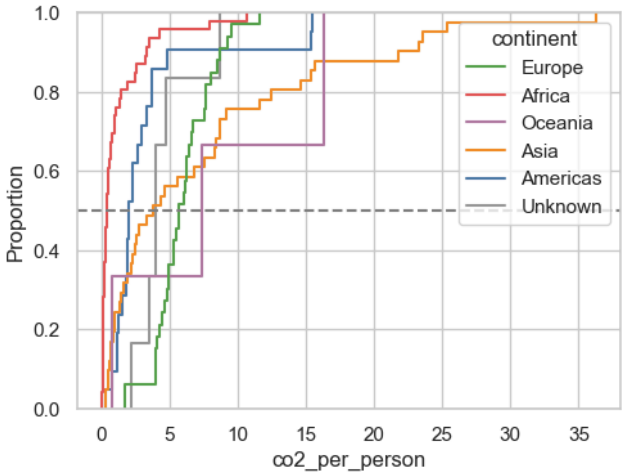
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12,4))

sns.histplot(ax=ax1, data=d19, x="co2_per_person", log_scale=True, multip
sns.ecdfplot(ax=ax2, data=d19, x="co2_per_person", hue="continent", palet
ax2.axhline(0.5, linestyle="--", color="gray")

plt.show()

# Share of countries that emit less than 2 tons pp
d19.assign(under_2=d19["co2_per_person"] < 2).groupby("continent")["under
```





Out[23]:

```
continent
Africa      0.83
Americas    0.48
Asia        0.34
Europe      0.06
Oceania     0.33
Unknown     0.00
Name: under_2, dtype: float64
```

Hint 1 `log_scale=True` spreads a right-skewed variable so the shape is visible. **Hint 2** In an ECDF the height at x is the share of observations ≤ x; where a curve crosses 0.5 is its median. **Hint 3** A boolean column's mean is the share of True.

Exercise 17 – Seaborn lineplot and renewables

Seaborn lineplot and renewables. Build `ct`: all rows of `countries` joined (left) to `info[['iso3', 'continent', 'gdp_tier']]`, `continent` filled with `'Unknown'`, and `gdp_pc` added. Draw `sns.lineplot` of `renewables_share_pct` over `year` with `hue='gdp_tier'` (use `tier_order`). Then print the number of countries with a **non-missing**

`renewables_share_pct` in 1990, 1999, 2000 and 2019, per tier. In a comment: (1) what does the shaded band mean, and what can a wide band mean (countries that differ a lot, or only a few countries)? Why is there no band for `Low` before 2000? (2) Look at the `Low` line around the year 2000: describe what you see and explain it with a number from your table. Also print how many countries have no `gdp_tier` in 2019; the code below leaves them out of the chart (`dropna`): say why that is the right call here.

Expected result (described, no picture or output is shown): A line chart of mean renewables share per year with one coloured line per income tier (and bands where seaborn can draw them); a table with four rows (years) and four columns (tiers) of counts; two short answers in comments, each backed by a number from the table.

In [24…

```
ct = pd.merge(countries, info[['iso3', 'continent', 'gdp_tier']], how='le

# Filling continent nan values with 'Unknown'
ct["continent"] = ct["continent"].fillna('Unknown')

# Creating new column
ct["gdp_pc"] = ct["gdp_usd"] / ct["population"]

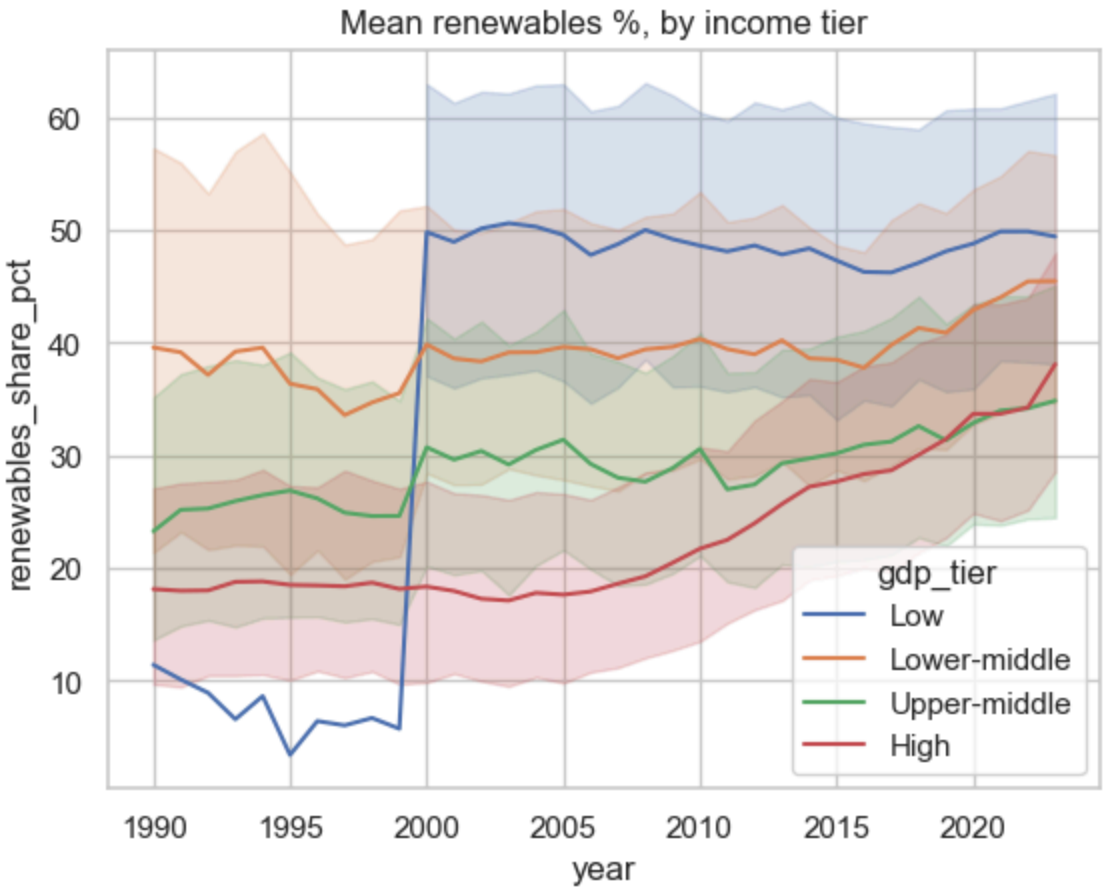
# Plotting
sns.lineplot(data=ct.dropna(subset=["gdp_tier"]), y="renewables_share_pct
plt.title("Mean renewables %, by income tier")
plt.show()


# Countries with non missing
ct[ct["year"].isin([1990, 1999, 2000, 2019])].groupby(["year", "gdp_tier"

# (1)
# The shaded band is the 95% confidence interval around the mean. A wide
# few observations (small sample size). There is no band for 'Low' before
# (a band needs at least 2 observations).

# (2)
# Around 2000, the 'Low' line jumps and a wide confidence band appears be
# 21 countries in 2000.

# Leaving out missing gdp_tier (dropna) is the right call because only a
# meaningful economic interpretation.
```



Out[24]:

gdp_tier	Low	Lower-middle	Upper-middle	High
year				
1990	1	12	22	34
1999	1	12	22	34
2000	36	34	33	35
2019	36	34	33	35

Hint 1 `sns.lineplot` with several rows per x value computes the mean and a confidence interval automatically. **Hint 2** `groupby(['year', 'gdp_tier'])[col].count().unstack()` counts non-missing values. **Hint 3** A band needs at least two observations per year. Compare the counts table across years. **Hint 4** Skeleton: `sns.lineplot(data=ct.dropna(subset=['gdp_tier']), x=..., y=..., hue='gdp_tier', hue_order=tier_order)`; counts: `ct[ct['year'].isin([...])].groupby(['year', 'gdp_tier'])[col].count().unstack()`. **Hint 5** Self-check: the counts table has 4 rows (the years) and 4 columns (the tiers). Read the `Low` column slowly.

Exercise 18 – A correlation heatmap

A correlation heatmap. On `d19`, compute the correlation matrix of `co2_mt`, `coal_co2_mt`, `oil_co2_mt`, `gas_co2_mt`, `population`, `gdp_usd`, `energy_twh`. Draw a heatmap with only the lower triangle (`mask`), annotations with 2 decimals, a diverging colour map centred on 0 (`vmin=-1, vmax=1`) and square cells. Print the two strongest correlations between *different* variables (take each pair once, without the diagonal; one line per pair). In a comment: is the strongest one a finding? Why do all of them look so strong?

In [25…

```
# Creating correlation matrix
cols = ["co2_mt", "coal_co2_mt", "oil_co2_mt", "gas_co2_mt", "population"
corr_d19 = d19[cols].corr()

# Mask only the lower triangle
mask = np.triu(np.ones_like(corr_d19, dtype=bool), k=1)

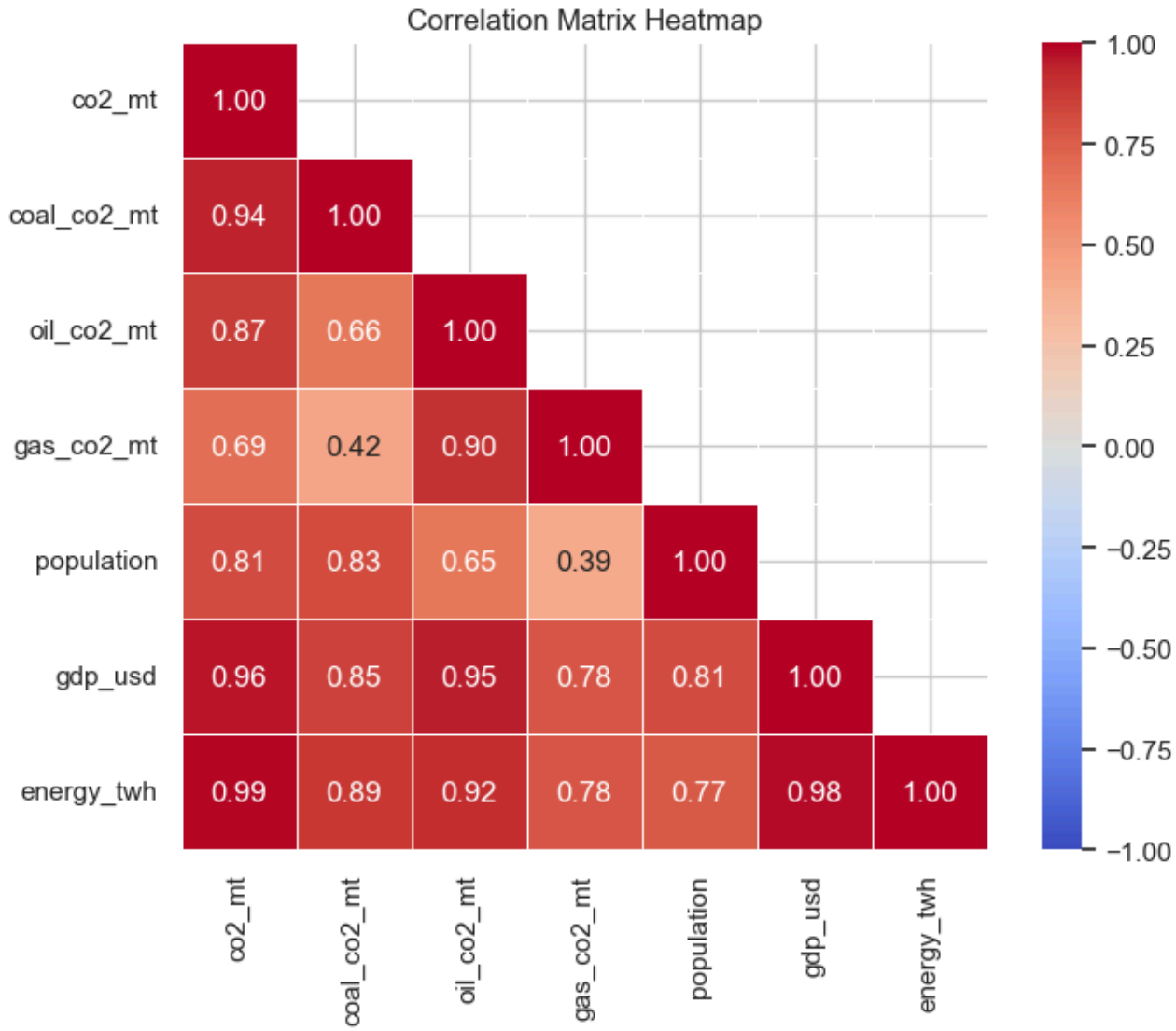
# Plotting
plt.figure(figsize=(8, 6))

sns.heatmap(
    corr_d19,
    mask = mask,      # Show only lower triangle
    annot=True,       # Display the correlation values inside the cells
    center = 0,
    cmap='coolwarm',
    fmt='.2f',
    linewidths=0.5,   # Add thin white lines between grid cells
    square = True,
    vmin=-1, vmax=1   # Anchor the color scale from -1 to 1
)

plt.title('Correlation Matrix Heatmap')
plt.show()

# Two strongest correlations
lower_pairs = corr_d19.where(np.tril(np.ones_like(corr_d19, dtype = bool)
for (v1, v2), val in lower_pairs.nlargest(2).items(): # Iterating over th
    print(f"{v1} & {v2}: {val:.3f}")

# The strongest correlation didn't teach us anything new. Extracting ener
# The correlations look strong in general because the data is very relate
```




```
energy_twh & co2_mt: 0.989
energy_twh & gdp_usd: 0.981
```

Hint 1 `np.triu(np.ones_like(corr, dtype=bool), k=1)` is the upper triangle above the diagonal: pass it as `mask`. **Hint 2** `center=0` with `cmap='coolwarm'` makes zero white. **Hint 3** Think about what these columns have in common.

Exercise 19 – Countries by time

Countries by time. Take the 12 countries with the highest `co2_mt` in 2019 (from `d19`) and the years `1990, 1995, ..., 2020`. Build a pivot table of `co2_per_person` (rows = nation in order of total emissions, columns = year) and draw it as an annotated heatmap (`fmt='.1f'`, `YlOrRd`, colour bar label). Print the three countries whose per-person emissions fell most between

1990 and 2020, in tonnes per person.

In [26…

```
# Years from 1990 in jumps of 5
year_list = [i for i in range(1990, 2021, 5)]


top_12_d19 = d19.nlargest(12, "co2_mt")["nation"]
top_12_global = countries[countries["year"].isin(year_list)]

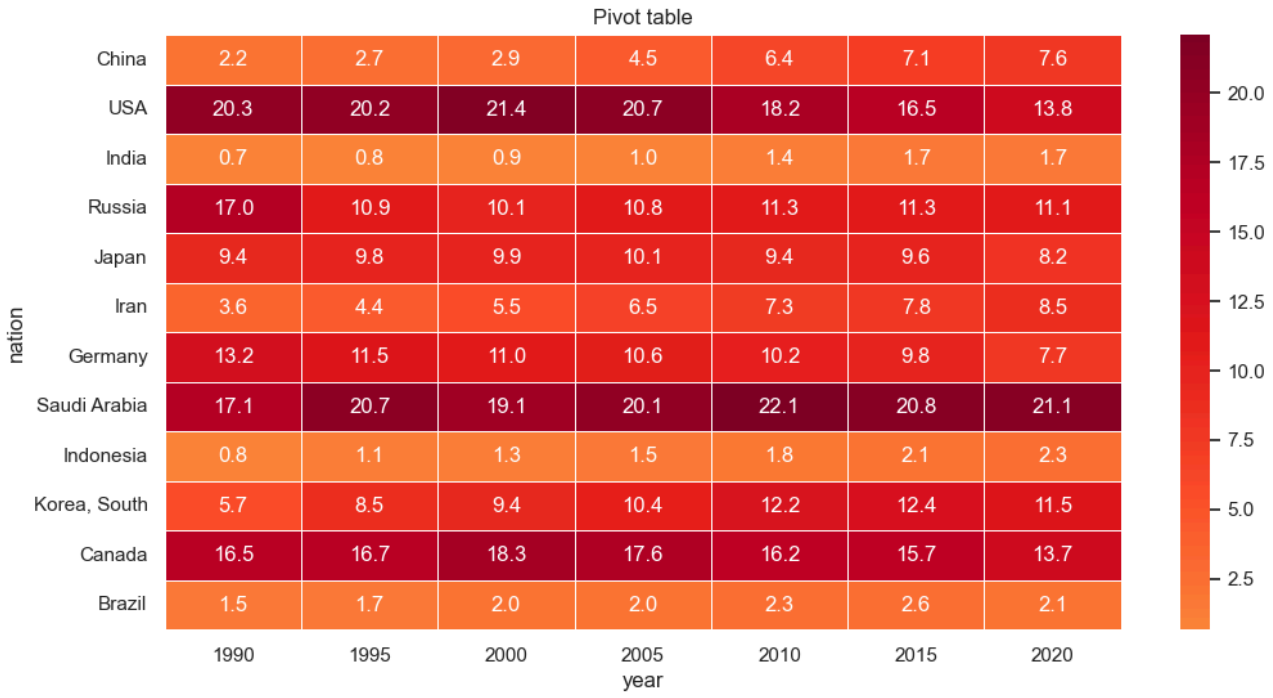

# Pivot table creation
piv_table = (
    top_12_global.pivot_table(index="nation", columns="year", values="co2
)

# Plotting
plt.figure(figsize=(12, 6))

sns.heatmap(
    piv_table,
    annot=True,       # Display the correlation values inside the cells
    center = 0,
    cmap='YlOrRd',
    fmt='.1f',
    linewidths=0.5,   # Add thin white lines between grid cells
)

plt.title('Pivot table')
plt.show()


# 3 countries which emissions fell most
(piv_table[2020] - piv_table[1990]).nsmallest(3).round(1)
```



Out[26]:
```
nation
USA       -6.4
Russia    -5.9
Germany   -5.5
dtype: float64
```

Hint 1 `pivot_table(index=..., columns='year', values=...)` then `.loc[top12]` to keep the emission order. **Hint 2** `sns.heatmap(..., annot=True, fmt='.1f')`.

Exercise 20 – Small multiples

Small multiples. With `sns.relplot` (`kind='line'`), draw one line per country of `co2_per_person` over `year`, in one panel per continent (`col='continent'`, `col_wrap=3`, grey thin lines; use `ct` without the `Unknown` group, `units='nation'` and `estimator=None`). Set panel titles to the continent name. Draw it first without any limit: it is unreadable because **one value** stretches the shared y axis. Find the row with the maximum `co2_per_person` in the whole table and print its country, year and value; then limit the y axis to 0–80 (`g.set(ylim=(0, 80))`) and say in a comment how many points the cap hides and why a cap is a decision that you must state on the chart. (If you know or can find out what happened to that country in that year, add it.) Print, per continent, the share of its countries whose per-person emissions in 2019 were **lower** than in 1990 (a country with no 1990 value counts as not lower).

Expected result (described, no picture or output is shown): One panel

per continent, a thin grey line per country, y axis limited to 0-80; the country, year and value of the maximum printed; one table with the share of countries per continent whose 2019 value is lower than their 1990 value; a comment on the extreme value.

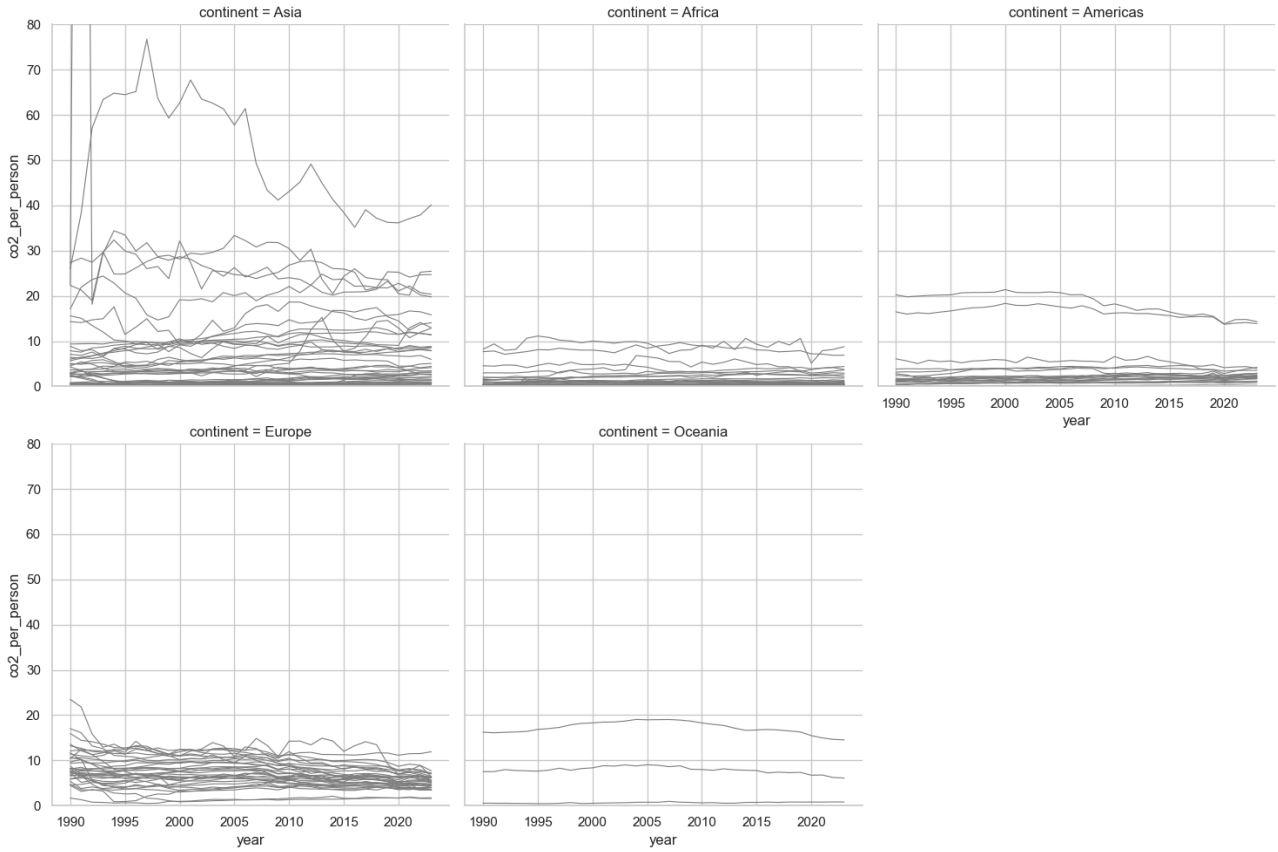
In [27…

```
ct_no_unknown = ct[ct["continent"] != "Unknown"]

# Plotting
g = sns.relplot(
    data=ct_no_unknown,
    x="year",
    y="co2_per_person",
    kind="line",
    col="continent",
    col_wrap=3,
    units='nation',
    estimator=None,
    color = "grey",
    linewidth = 0.8
    )
g.set(ylim=(0, 80)) # Only one point is hidden since its the one that cau
plt.show()

# Finding max co2 pp
max_co2 = ct.nlargest(1, columns="co2_per_person").reset_index(drop=True)
print(f"Max co2 pp country is {max_co2.loc[0, "nation"]} in the year {max

# Printing the share of countries with co2 pp in 2019 lower than 1990
piv = ct_no_unknown.pivot_table(
    index=["continent", "nation"],
    columns="year",
    values="co2_per_person",
)
lower_share = (piv[2019] < piv[1990]).groupby("continent").mean().round(2
print(lower_share)
```




```
Max co2 pp country is Kuwait in the year 1991 which has pp value of 364.7
91
continent
Africa      0.20
Americas    0.24
Asia        0.32
Europe      0.91
Oceania     0.33
dtype: float64
```

Hint 1 `units='nation'` with `estimator=None` draws every country separately instead of one mean line. **Hint 2** `col_wrap` limits the number of panels per row. **Hint 3** `ct['co2_per_person'].idxmax()` finds the extreme row; then `g.set(ylim=...)`. **Hint 4** Pivot with a two-level index (`continent`, `nation`) and compare two year columns. **Hint 5** Skeleton: `g = sns.relplot(data=..., x='year', y='co2_per_person', col='continent', col_wrap=3, kind='line', units='nation', estimator=None, color='grey')`, then `g.set(ylim=(0, 80))`. **Hint 6** Self-check: five panels (`Unknown` excluded), and the extreme value belongs to a single country in a single year.

Exercise 21 – Pairplot

Pairplot. On `d19` rows with no missing value in `population`, `gdp_pc`,

`co2_per_person`, `renewables_share_pct`, `gdp_tier`: (a) draw `sns.pairplot` of these four variables coloured by `gdp_tier` (`corner=True`); (b) explain in a comment why most points are squeezed against the left edge of the panels, then draw it again with `population`, `gdp_pc` and `co2_per_person` replaced by their **log10** versions.

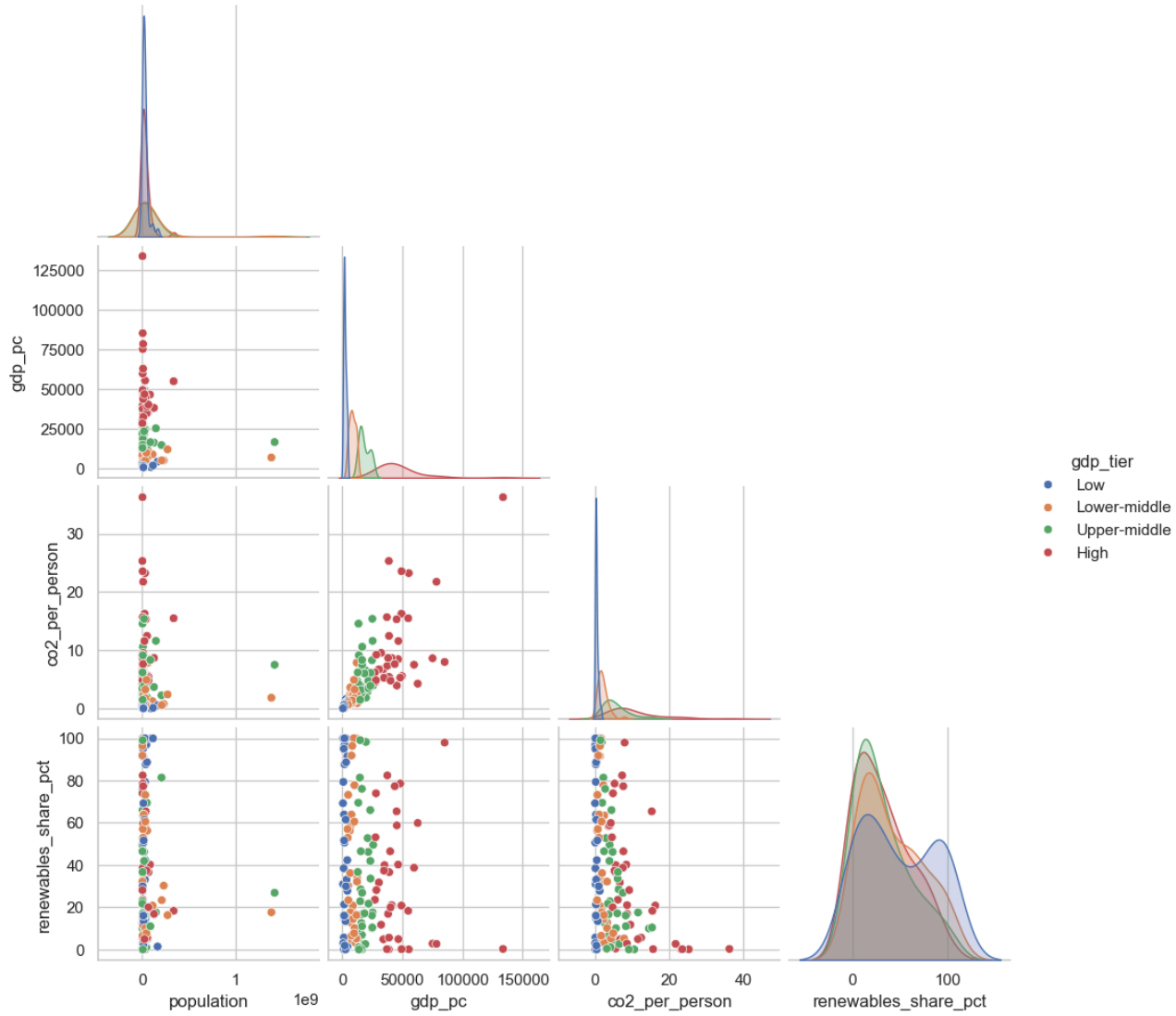
In [28…

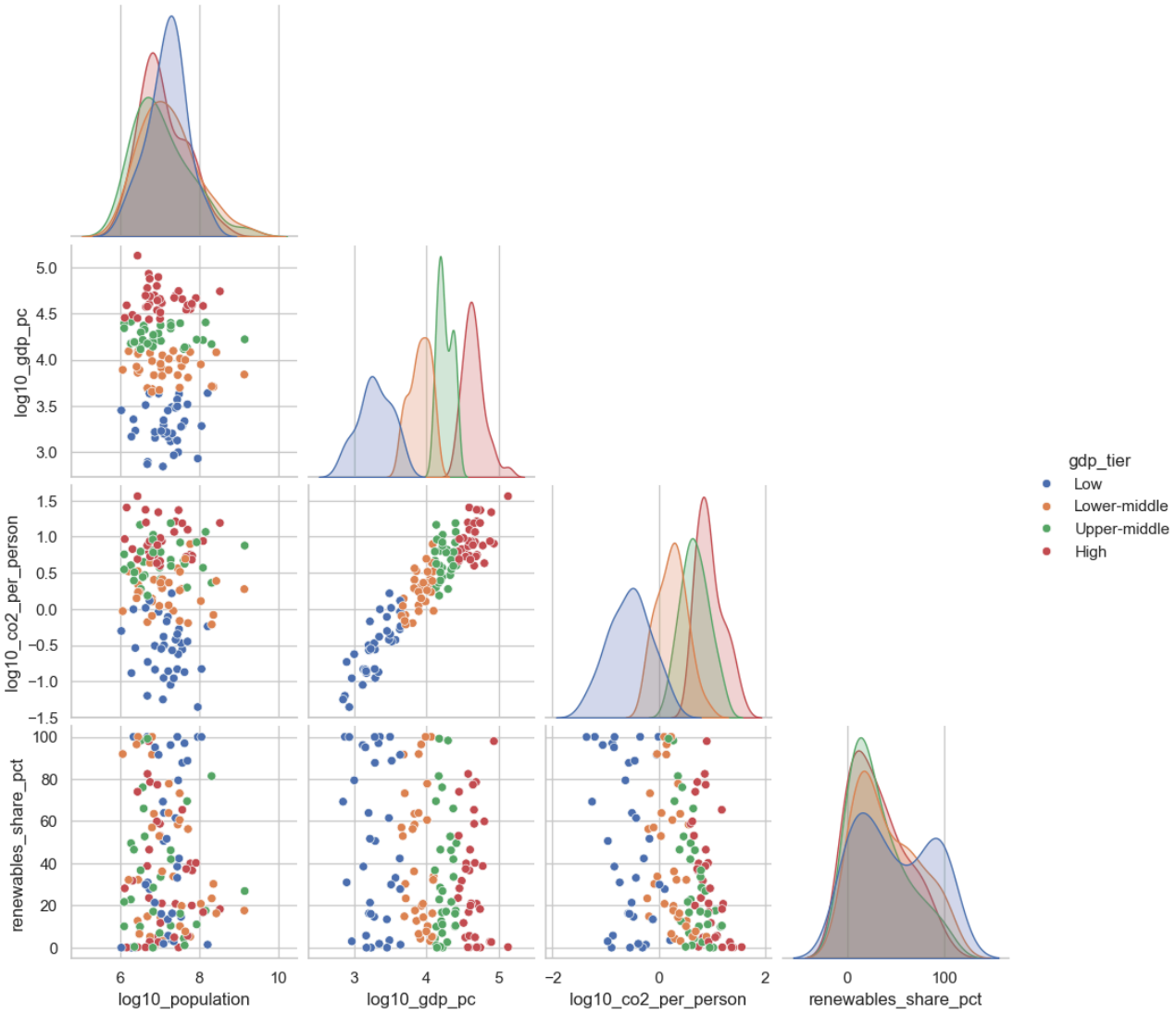
```
# Removing missing values
cols = ["population", "gdp_pc", "co2_per_person", "renewables_share_pct",
d19_no_missing = d19.dropna(subset=cols).copy()

# (a)
sns.pairplot(data=d19_no_missing[cols], corner=True, hue="gdp_tier", hue_
plt.show()

# (b) Most data is squeezed to the left since we have some edge points th
for c in ["population", "gdp_pc", "co2_per_person"]: # Applying log10
    d19_no_missing[f"log10_{c}"] = np.log10(d19_no_missing[c])

# Defining new column names
log_cols = ["log10_population", "log10_gdp_pc", "log10_co2_per_person", "
sns.pairplot(data=d19_no_missing, vars=log_cols, corner=True, hue="gdp_ti
plt.show()
```


gdp_pc
co2_per_person
renewables_share_pct
population
1e9
gdp_pc
co2_per_person
renewables_share_pct
gdp_tier
Low
Lower-middle
Upper-middle
High



Hint 1 `corner=True` draws only the lower triangle. **Hint 2** `np.log10` on the skewed columns, then pass the new column names to `vars=`.

Exercise 22 – A regression line and its outliers

A regression line and its outliers. Using `m` (from the bubble chart): draw `sns.regplot` of `log10(co2_per_person)` against `log10(gdp_pc)`. Compute the slope with `np.polyfit`, the correlation on the log scale and on the raw scale, and the three countries **furthest above** and **furthest below** the line. In a comment: what does the slope mean in plain words, and is it a causal effect?

Expected result (described, no picture or output is shown): A scatter of the two log10 variables with a fitted line and its confidence band; one printed line with the slope and the two correlations; one printed line with the

three countries above and the three below the line; a comment interpreting the slope.

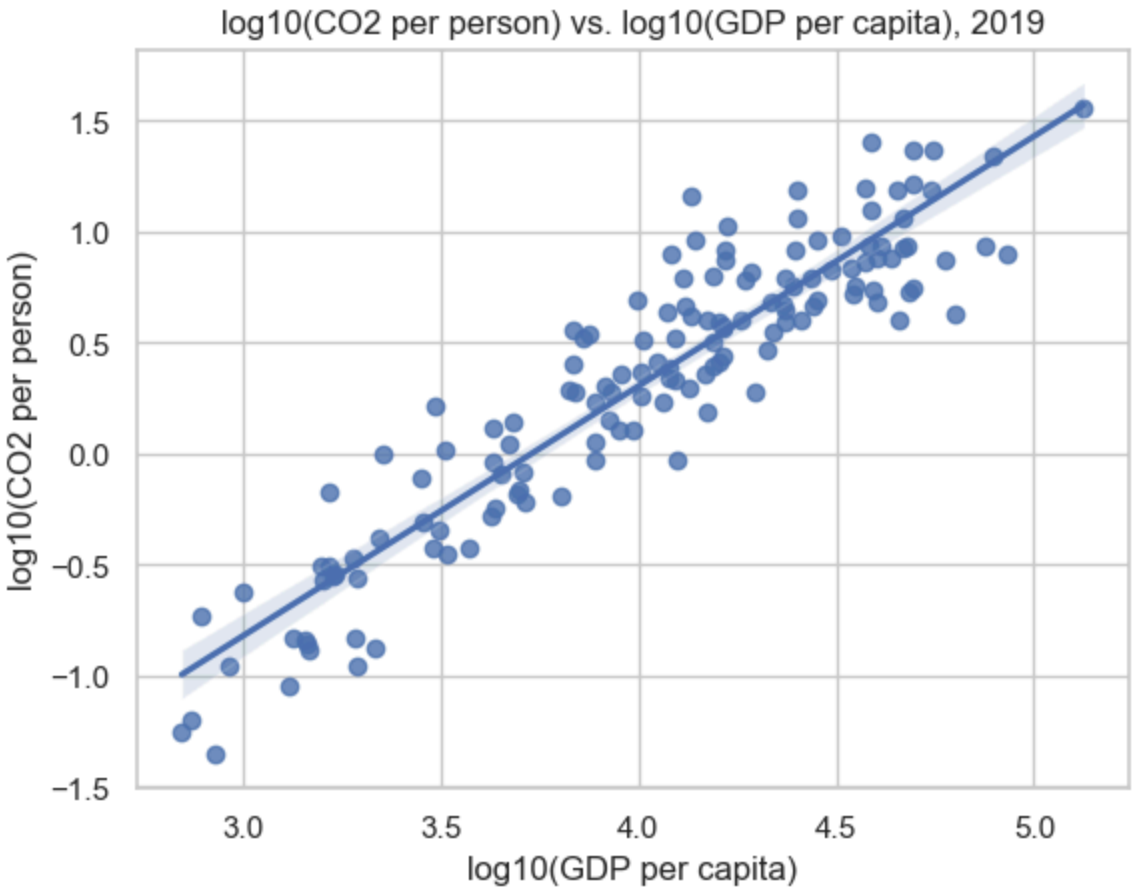
In [29…

```
# Applying log10
x_ = np.log10(m["gdp_pc"])
y_ = np.log10(m["co2_per_person"])

# Plotting
sns.regplot(x=x_, y=y_)
plt.xlabel("log10(GDP per capita)")
plt.ylabel("log10(CO2 per person)")
plt.title("log10(CO2 per person) vs. log10(GDP per capita), 2019")
plt.show()

# Unpack slope and intercept values
slope, intercept = np.polyfit(x_, y_, 1)
corr_log = x_.corr(y_)
corr_raw = m["gdp_pc"].corr(m["co2_per_person"])
print(f"Slope: {slope:.2f} | Corr (log): {corr_log:.2f} | Corr (raw): {co


# Residuals = observed y - predicted y
m_res = m.assign(residual=y_ - (slope * x_ + intercept))
above3 = m_res.nlargest(3, "residual")["nation"].tolist()
below3 = m_res.nsmallest(3, "residual")["nation"].tolist()
print(f"Furthest above: {above3} | Furthest below: {below3}")
```




```
Slope: 1.13 | Corr (log): 0.93 | Corr (raw): 0.79
Furthest above: ['Mongolia', 'South Africa', 'Turkmenistan'] | Furthest b
elow: ['Switzerland', 'Rwanda', 'Democratic Republic of Congo']
```

Hint 1 `np.polyfit(x, y, 1)` returns slope and intercept. **Hint 2** Residual = observed y minus the line's prediction. **Hint 3** On a log-log chart the slope is an elasticity: % change in y per % change in x. **Hint 4** Skeleton: `x = np.log10(m['gdp_pc'])`, `y = np.log10(m['co2_per_person'])`, `sns.regplot(x=x, y=y)`, `slope, intercept = np.polyfit(x, y, 1)`, residual `= y - (slope * x + intercept)`, then `m.assign(residual=...)` and `nlargest(3, 'residual')`. **Hint 5** Self-check: the slope is a little above 1, and the log-log correlation is higher than the raw one.

Block 3 – Plotly: interactive charts for the public website

The ministry wants interactive charts for its website. Plotly makes them with one call; the question is when interactivity helps the reader. **Plotly in four lines:** `px.scatter(df, x=, y=, color=, size=, hover_name=)` draws a chart; `animation_frame=` adds a time slider;

`fig.update_layout(updatemenus=[...])` adds buttons or dropdowns; in a notebook, put `fig` as the **last** line of the cell so that it is displayed.

Plotly Express draws interactive charts with one call: hover, zoom, animation. The question is when interactivity helps the reader and when it only decorates.

Exercise 23 – Interactive bubbles through time

An interactive bubble chart through time. Build `anim` from `ct`: rows with non-missing `gdp_pc`, `co2_per_person` and `population`, sorted by `year`. Draw the interactive version of the bubble chart of Block 1 with `px.scatter` (log x, size = `population`, colour = `continent` using `palette`, `hover_name='nation'`, `size_max=45`) and `animation_frame='year'`. Fix the axes so the frames are comparable: `range_x=[300, 250000]` (log) and `range_y=[0, 80]` (the same cap as in the small multiples). Print which rows fall outside `range_y`. In a comment: what does hovering give the reader that the static chart could not, what does a fixed axis protect the reader from, and what did the cap hide?

In [30…

```python
# Write your code here
# We keep only rows with all the non-missing values and we sort by year
anim = (
    ct.dropna(subset=["gdp_pc", "co2_per_person", "population"])
      .sort_values("year")
      .copy()
)

# Printing the rows outside the co2-per-person axis range after fix
outside_range = anim[
    (anim["co2_per_person"] < 0) |
    (anim["co2_per_person"] > 80)
]

print("Rows outside range_y [0, 80]:")
print(outside_range[["nation", "year", "co2_per_person"]])

# Creating the interactive animated bubble chart
fig = px.scatter(
    anim,
    x="gdp_pc",
    y="co2_per_person",
    size="population",
    color="continent",
    color_discrete_map=palette,
    hover_name="nation",
    size_max=45,
```

```
    animation_frame="year",
    log_x=True,
    range_x=[300, 250000],
    range_y=[0, 80]
)

fig.show()

# Hovering shows us each country's identity and more details,
# it also shows infromation that a static chart cannot show interactively
# by making our axes fixed we prevent changes in scale from year-to-year
# differences alkso appear larger or smaller than they really are.
# The y-axis cap hides values above 80 (and below 0), so the printed rows
```

```
Rows outside range_y [0, 80]:
      nation  year  co2_per_person
4832  Kuwait  1991         364.791
```

Hint 1 `animation_frame='year'` adds a slider and a play button. **Hint 2** `range_x`, `range_y` fix the axes; for a log axis give the data values.

Exercise 24 – A world map

A world map. Draw a choropleth of `co2_per_person` in 2019 from `d19` with `px.choropleth` (`locations='iso3'` , `hover_name='nation'` , a sequential colour scale, `projection='natural earth'`). Cap the colour range at 20 tonnes (`range_color=(0, 20)`). In a comment: why cap the range, and why are some countries grey?

In [31…

```
# Write your code here
fig = px.choropleth( # The function for a world map
    d19,
    locations="iso3",
    color="co2_per_person",
    color_continuous_scale="YlOrRd", # Color scheme
    projection="natural earth", # Type of world map
    range_color=(0, 20), # The range of coolrs
    hover_name="nation" # What we show when we hover
)

fig.show()
# The cap range is to prevent a case where countries with outlier values
# from to dominate the color scale, letting an easier way to difference b
# Grey countries are the places where the co2 per person is Nan, with no
```

Hint 1 `px.choropleth(df, locations='iso3', color=..., hover_name=...)`. **Hint 2** A colour scale is a visual axis: extreme values stretch it exactly like they stretch a y axis.

Exercise 25 – Size and colour in one chart

Size and colour in one chart. Draw a treemap with `px.treemap`: path = `continent` then `nation`, area = `co2_mt`, colour = `co2_per_person` (`RdYlGn_r`), on `d19` rows with both values. In a comment: which country is largest by **area** and which by **colour**, and why are they not the same?

In [32…
```
# Write your code here
px.treemap(
    d19,
    path=['continent','nation'],
    hover_data='nation',
    values='co2_mt',
    color='co2_per_person',
    color_continuous_scale="YlOrRd",
    range_color=(0, 25)
)

# The largest country by area is China, while by color is saudi Arabia, T
# While china has a bigger population but emits less co2 per person than
```

Hint 1 `px.treemap(df, path=[...], values=..., color=...)`. **Hint 2** What does the area of a rectangle measure, and what does its colour measure?

Exercise 26 – A dropdown

A dropdown. The energy table has inconsistent source labels. The table `energy` has the columns `nation`, `year`, `source`, `twh`. First print how many distinct labels `energy['source']` has, then clean them (strip spaces, capitalise) and print again. Aggregate to `mix` = TWh summed by `nation`, `year`, `source`. Then draw a stacked area chart (`go.Scatter(stackgroup=...)`) of the energy mix for `['Germany', 'Israel', 'Norway', 'China']`, with a dropdown (`updatemenus`) that shows one country at a time.

In [33…
```
# Write your code here
import plotly.graph_objects as go

# We print distinct labels before cleaning
print("Distinct sources before:", energy['source'].nunique())

# Cleaning the 'source' column by strip spaces and capitalize
energy['source'] = energy['source'].str.strip().str.capitalize()
print("Distinct sources after:", energy['source'].nunique())
```

```python
# Aggregate TWh summed by nation, year, source
mix = energy.groupby(['nation', 'year', 'source'], as_index=False)['twh']

nations = ['Germany', 'Israel', 'Norway', 'China']
dff = mix[mix['nation'].isin(nations)]
sources = dff['source'].unique()

fig = go.Figure()

# Adding traces for all nations
for nation in nations:
    ndf = dff[dff['nation'] == nation]
    for src in sources:
        sub = ndf[ndf['source'] == src]
        fig.add_trace(go.Scatter(
            x=sub['year'], y=sub['twh'], name=src,
            stackgroup='one', visible=(nation == nations[0])
        ))

# Create dropdown buttons using a list comprehension to access each count
fig.update_layout(
    updatemenus=[dict(
        buttons=[
            dict(
                label=nat,
                method="update",
                args=[{"visible": [n == nat for n in nations for _ in sou
                       {"title": f"Energy Mix for {nat}"}]
            ) for nat in nations
        ],
        x=0.1, y=1.15, xanchor='left', yanchor='top'
    )],
    title=f"Energy Mix for {nations[0]}",
    xaxis_title="Year",
    yaxis_title="TWh"
)

fig.show()
```

```
Distinct sources before: 21
Distinct sources after: 7
```

Hint 1 `str.strip().str.capitalize()` fixes both problems at once; `nunique()` shows the effect. **Hint 2** One trace per (country, source); `visible=True` only for the first country. **Hint 3** Each button sets the `visible` list: a boolean per trace, `True` for the traces of its country. **Hint 4** Skeleton: one `go.Scatter(..., visible=(i == 0))` per (country, source) pair; each button has `args=[{'visible': [...]}]` with one True or False per trace, True only for the traces of its country.

Exercise 27 – An animated bar race

An animated bar race. For every year take the 10 countries with the highest `co2_mt` (`countries`) and draw an animated horizontal bar chart with `px.bar` (`animation_frame='year'`, fixed `range_x=[0, 13000]`, bars sorted within each frame). Print how many different countries appear in any top 10.

In [34…

```
import plotly.express as px

# Identify the country column name to save them for later
country_col = 'countries' if 'countries' in yearly.columns else 'nation'

# Aggregating countries that are not countries.
aggregates_to_exclude = [
    'world',
    'upper-middle-income countries',
    'Lower-middle-income countries',
    'high-income countries',
]

# Filtering out the non-countries from before
filtered_yearly = yearly[
    ~yearly[country_col].str.lower().isin([a.lower() for a in aggregates_
].copy()

# Getting the top 10 countries for each year using the filtered data
top10_per_year = (
    filtered_yearly.groupby('year', group_keys=False)
       .apply(lambda x: x.nlargest(10, 'co2_mt'))
       .reset_index()
)

# Printing how many different countries are in any top 10
unique_countries_count = top10_per_year[country_col].nunique()
print(f"Number of unique countries appearing in top 10: {unique_countries

# Convert year to string so Plotly handles animation frames correctly
top10_per_year['year'] = top10_per_year['year'].astype(str)

# Sorting values so bars appear sorted in each frame
top10_per_year = top10_per_year.sort_values(by=['year', 'co2_mt'], ascend

# Creating hte animated horizontal bar chart
fig = px.bar(
    top10_per_year,
    x='co2_mt',
    y=country_col,
    animation_frame='year',
    animation_group=country_col,
    orientation='h',
```

```
    range_x=[0, 13000],
    title="Top 10 CO2 Emitting Countries Over Time"
)

# Keep category order tidy per frame
fig.update_layout(yaxis={'categoryorder': 'total ascending'})

fig.show()
```

```
Number of unique countries appearing in top 10: 12
```

```
C:\Users\Orange\AppData\Local\Temp\ipykernel_10608\2666229630.py:22: Depr
ecationWarning: DataFrameGroupBy.apply operated on the grouping columns.
This behavior is deprecated, and in a future version of pandas the groupi
ng columns will be excluded from the operation. Either pass `include_grou
ps=False` to exclude the groupings or explicitly select the grouping colu
mns after groupby to silence this warning.
  .apply(lambda x: x.nlargest(10, 'co2_mt'))
```

Hint 1 Sort by year ascending and `co2_mt` descending (`ascending=[True, False]`); then `groupby('year').head(10)` keeps the 10 largest per year. **Hint 2** `fig.update_yaxes(categoryorder='total ascending')` sorts the bars inside each frame.

Block 4 – Find the bug: fact-check the headlines

Four checks. In Exercises 28–30 a headline or a report quotes a number from this data; in Exercise 31 a summary table silently loses rows. In all four the code **runs without an error** and the result is still wrong. **How to hunt a silent bug:** (1) compare the result with a number you trust (a total, a count, an earlier result); (2) print the number of rows before and after every merge and filter; (3) ask what each row of the table represents, and whether every row belongs in the calculation.

Use the cleaned tables `countries`, `aggregates` and `info` from Block 0 (not `yearly`) in all four exercises. For each exercise: (1) find the problem **using a number from the data**, (2) explain it in a comment, (3) write the fixed code in the empty cell below the buggy code, so that your output matches the expected result.

Exercise 28 – Bug 1: the CO2 total looks too high

The total CO2 of 2019 looks far too high (`info` is not needed in this exercise). Find out why, fix it, and compare the corrected total with the

`World` row: the two will not match exactly, because the `World` row also counts emissions that belong to no country (such as international transport) and our sample has only 150 countries.

Expected result (described, no picture or output is shown): Your fixed code prints one line with the total for 2019 (in Mt), the World value, and the total as a percentage of the World value.

In [35…

```
# This code runs without an error but gives a wrong result. Find the bug,

# Before:
both = pd.concat([countries, aggregates])
total_2019 = both.loc[both['year'] == 2019, 'co2_mt'].sum()
print(f'Total CO2 2019: {total_2019:,.0f} Mt')

# After:
both = pd.concat([countries, aggregates]) # Just like before
# We calculate the true total for 2019 using only individual countries
total_2019 = countries.loc[countries['year'] == 2019, 'co2_mt'].sum()

# Then we get the world value for 2019 from aggregates
world_2019 = aggregates.loc[(aggregates['year'] == 2019) & (aggregates['n

# We calculate percentage of the World value and now we can compare the d
percentage = (total_2019 / world_2019) * 100

# Print the result
print(f"Total CO2 2019: {total_2019:,.0f} Mt | World Value: {world_2019:,
```

```
Total CO2 2019: 146,927 Mt
Total CO2 2019: 35,242 Mt | World Value: 37,087 Mt | Percentage: 95.0%
```

Hint 1 Compare the printed total with a number you trust from earlier exercises. **Hint 2** Which rows of `both` are not countries? **Hint 3** Self-check: the correct total is a little below the World value, not several times larger.

Exercise 29 – Bug 2: the headline about a jump in emissions

A newspaper says world emissions 'jumped' in 2019 and shows a bar chart whose y axis starts at 36,500. The buggy cell computes correctly how much taller the 2019 bar looks on that chart. Add (1) the real change in percent between 2018 and 2019, and (2) how much taller the 2019 bar would look if the axis started at zero. In a comment, say what is wrong with the headline.

Expected result (described, no picture or output is shown): Your fixed code prints the real percentage change and how many times taller the second bar looks on an axis that starts at 36,500.

In [36…

```
# This code runs without an error but gives a wrong result. Find the bug,
# Before:
w = aggregates[(aggregates['nation'] == 'World') & aggregates['year'].isi
looks = (w['co2_mt'].iloc[1] - 36500) / (w['co2_mt'].iloc[0] - 36500)
print("Before: \n")
print(f'On the chart the 2019 bar is {looks:.1f} times taller than the 20
# After:
# we need to get the world emissions for 2018 and 2019
country_col = 'countries' if 'countries' in aggregates.columns else 'nati
w = aggregates[(aggregates[country_col] == 'World') & (aggregates['year']

val_2018 = w.loc[w['year'] == 2018, 'co2_mt'].values[0]
val_2019 = w.loc[w['year'] == 2019, 'co2_mt'].values[0]

# change in percent between 2018 and 2019
real_pct_change = ((val_2019 - val_2018) / val_2018) * 100

# How much taller the 2019 bar looks on an axis starting at 36,500 vs sta
looks_truncated = (val_2019 - 36500) / (val_2018 - 36500)
looks_zero = val_2019 / val_2018
print("\n After: \n")
print(f"Real percentage change (2018 to 2019): {real_pct_change:.2f}%")
print(f"On the chart, the 2019 bar looks {looks_truncated:.1f} times tall
print(f"If the axis started at zero, it would look {looks_zero:.2f} times

# The headline is misleading because the y-axis starts at 36,500
# That leads to a artificial jump from real world percentage to the size.
# Menaing the jump just looks larging which is not the right scale
```

```
Before:

On the chart the 2019 bar is 2.5 times taller than the 2018 bar: emission
s jumped

 After:

Real percentage change (2018 to 2019): 0.96%
On the chart, the 2019 bar looks 2.5 times taller than the 2018 bar.
If the axis started at zero, it would look 1.01 times taller.
```

Hint 1 Compute the percentage change between the two values first. **Hint 2** What does the length of a bar encode, and where must a bar chart start? **Hint 3** Self-check: the real change is small, about one percent, and the bar looked more than twice as tall.

Exercise 30 – Bug 3: average CO2 per person by continent

A report says 'the average person in Asia emits 7.4 tonnes of CO2 per year'. Check it using the columns `co2_mt`, `co2_per_person` and `population` of

2019. Build a table `res` with one row per continent (the 6 countries without a continent get the label 'Unknown') and two columns: the plain mean of the countries' values, and the population-weighted value.

Expected result (described, no picture or output is shown): A table with one row per continent (and one for countries without a continent) and two columns: the plain mean of the countries' values and the population-weighted value.

In [37…

```python
# This code runs without an error but gives a wrong result. Find the bug,

# Before:
print("Before: \n")
c19 = countries[countries['year'] == 2019].merge(info[['iso3', 'continent
print(c19.groupby('continent')['co2_per_person'].mean().round(2))

# After:
print("\n After: \n")
# Filter for 2019 and merge with info to get continents
c19 = countries[countries['year'] == 2019].merge(info[['iso3', 'continent

# Fill missing continent values with 'Unknown'
c19['continent'] = c19['continent'].fillna('Unknown')

# Creating a function for the population-weighted mean: sum(co2 * pop) /
def weighted_mean(group):
    total_co2 = (group['co2_per_person'] * group['population']).sum()
    total_pop = group['population'].sum()
    return total_co2 / total_pop if total_pop > 0 else 0

# Build the res table
res = pd.DataFrame({
    'plain_mean': c19.groupby('continent')['co2_per_person'].mean(),
    'population_weighted': c19.groupby('continent').apply(weighted_mean,
})

# Round for cleanliness
res = res.round(2)

print(res)
```

```
Before:

continent
Africa      1.18
Americas    3.44
Asia        7.40
Europe      5.99
Oceania     8.14
Name: co2_per_person, dtype: float64

 After:

           plain_mean  population_weighted
continent
Africa           1.18                 1.05
Americas         3.44                 7.64
Asia             7.40                 4.55
Europe           5.99                 7.18
Oceania          8.14                11.46
Unknown          4.52                 3.76
```

Hint 1 Does every country count equally in this average? **Hint 2** How would you compute the real per-person average of a whole region from totals? Remember: `co2_mt` is in Mt (10^6 tonnes) and `population` is in persons. **Hint 3** 6 countries have no continent. What does `groupby` do with missing keys? **Hint 4** Self-check: the two columns differ a lot; for the Americas by more than a factor of two.

Exercise 31 – Bug 4: CO2 totals by continent

The continent totals look plausible, but something is missing. Compare the sum of the continent totals with the total of the data and fix it.

Expected result (described, no picture or output is shown): A table of CO2 by continent that includes a row for countries without a continent, and one printed line comparing the sum of the table with the total in the data (they must be equal).

In [ ]:
```python
# This code runs without an error but gives a wrong result. Find the bug,
# Before:
print('Before: ')
c19 = countries[countries['year'] == 2019].merge(info[['iso3', 'continent
by_cont = c19.groupby('continent')['co2_mt'].sum()
print(by_cont.round(0))

# After:
print("\n After: \n")
# 1. Filter countries for 2019 and use a left join with info
c19 = countries[countries['year'] == 2019].merge(info[['iso3', 'continent
```

```python
# 2. Fill missing continent values with 'Unknown' so they are not dropped
c19['continent'] = c19['continent'].fillna('Unknown')

# 3. Group by continent and sum CO2 emissions
by_cont = c19.groupby('continent')['co2_mt'].sum().round(0)
print(by_cont)

# 4. Compare the sum of the continent totals with the total of the raw co
print("Sum of table:", by_cont.sum())
print("Total in countries 2019:", countries.loc[countries['year'] == 2019
```

```
Before:
continent
Africa       1190.0
Americas     7311.0
Asia        19796.0
Europe       4891.0
Oceania       459.0
Name: co2_mt, dtype: float64

 After:

continent
Africa       1190.0
Americas     7311.0
Asia        19796.0
Europe       4891.0
Oceania       459.0
Unknown      1595.0
Name: co2_mt, dtype: float64
Sum of table: 35242.0
Total in countries 2019: 35242.259000000005
```

Hint 1 Print the sum of the continent totals and compare it with the sum of `countries` in 2019. **Hint 2** Look at how many rows the merge returns. **Hint 3** Even after you fix the merge, check that `groupby` keeps the countries without a continent (`fillna('Unknown')` or `dropna=False`). **Hint 4** Self-check: after the fix the sum of the table equals the total of `countries` in 2019, to the last digit.

Block 5 – Integrated challenge: the briefing for the Minister

The task. The Ministry of Energy asks for **one figure** that answers: *How does Israel's carbon footprint compare with countries of similar wealth, how is its energy mix changing, and where does it stand in the world?*

Roadmap (each step says what you should get, not how):

1. `peers`: the four countries (not Israel) whose 2019 GDP per person is closest to Israel's. Use `m`.
2. `trend`: store the five names as `group = ['Israel'] + peers`; `trend` has one column for each of the five countries and one row per year (1990–2023) of CO2 per person. Check its shape.
3. `share`: Israel's energy mix 2000–2023 as **percent** of the yearly total, one column per source. First make sure the `source` labels of `energy` are clean (strip and capitalise; skip this if you did Exercise 26) and check that each row of `share` adds up to 100.
4. One figure: top left the trend (Israel emphasised), top right Israel's mix as a stacked area, bottom the world scatter of GDP vs CO2 per person (log x) with the five countries highlighted and labelled.
5. Two sentences for the minister with numbers, and one caveat that is specific to *these* data: a limit of this figure that you can state with a number, for example about who the 'similar' countries are, how many countries stand behind a panel, or which year the data are from.

Exercise 32 – Challenge step 1: the four countries closest to Israel in wealth

Step 1 – the four peers (`peers`), with their wealth gap to Israel in percent.

Expected result (described, no picture or output is shown): A list `peers` with four country names (not Israel), and a table with each peer's GDP per person and its gap to Israel in percent (smallest absolute gaps first), plus one printed line with Israel's GDP per person.

In [78…

```
# Write your code here

# Firstly lets organize the countries by their statistics in 2019
c19 = countries[countries['year'] == 2019].merge(info[['iso3', 'continent

# Secondly we sort by GDP per person
gdp_country = c19.groupby('nation')['gdp_usd'].sum()
israel_gdp = gdp_country['Israel']
gdp_country_without_israel = gdp_country.drop('Israel', errors='ignore')

# Now we calculate the percentage gap for every country relative to Israe
gdp_pct_diff = ((gdp_country_without_israel - israel_gdp) / israel_gdp) *

# We use nsmallest to find the 4 peers closest to israel by percents
peer_indices = (gdp_country_without_israel - israel_gdp).abs().nsmallest(
peers = gdp_pct_diff.loc[peer_indices]

print(f"Israel's GDP: {israel_gdp}")
```

```
print("\nPeers GDP gap to Israel (%):")
print(peers)
```

```
Israel's GDP: 296799271432.0

Peers GDP gap to Israel (%):
nation
Ireland    -0.773444
Portugal   -3.468208
Morocco     3.647482
Denmark    -5.438998
Name: gdp_usd, dtype: float64
```

Hint 1 Compute the gap relative to Israel's value, take the absolute value, and keep the four smallest. **Hint 2** `nsmallest` needs a column: create `gap_pct` first.

Exercise 33 – Challenge step 2: CO2 per person over time (trend table)

Step 2 – `trend` (one column per country in `['Israel'] + peers`, rows = years). Print its shape and Israel's change in CO2 per person between 1990 and 2022 (%; 2023 is provisional, so we stop at 2022).

Expected result (described, no picture or output is shown): A table `trend` with one row per year (1990 to 2023) and five columns (Israel first), its printed shape, and one printed line with Israel's CO2 per person in 1990 and in 2022 and the change in percent.

In [12…
```
# Write your code here
countries_list = ['Israel', 'Ireland','Portugal','Morocco','Denmark']
pivot_df = countries.pivot(index='year', columns=country_col, values='co2
trend = pivot_df[countries_list].copy()
print(f"The shape of trend is: {trend.shape}")
print(trend['Israel'],"\n")
print("*"*40)
print(f"The Co2 Per person in 1990: {trend['Israel'].iloc[0]:.2f}\n The C
```

```
The shape of trend is: (34, 5)
year
1990     7.882
1991     7.563
1992     8.489
1993     8.697
1994     9.101
1995     9.312
1996     9.444
1997     9.717
1998     9.673
1999     9.209
2000     9.739
2001    10.068
2002     9.284
2003     9.598
2004     8.871
2005     8.402
2006     9.081
2007     9.020
2008     9.598
2009     8.862
2010     9.308
2011     9.181
2012     9.830
2013     8.142
2014     7.633
2015     7.815
2016     7.450
2017     7.038
2018     7.049
2019     6.809
2020     6.737
2021     6.592
2022     6.840
2023     5.983
Name: Israel, dtype: float64

**************************************
The Co2 Per person in 1990: 7.88
 The Co2 per person in 2022: 5.98
 The percent change is 131.74
```

Hint 1 Same `pivot` pattern as in Block 1. **Hint 2** Keep the column order with `[group]`.

Exercise 34 – Challenge step 3: Israel's energy mix in percent (share table)

Step 3 – `share`: Israel's energy mix 2000–2023 in percent of the yearly total, columns in the order Coal, Oil, Gas, Nuclear, Hydro, Wind, Solar. Print the

share of Coal, Gas and Solar in 2000 and in 2023.

Expected result (described, no picture or output is shown): A table `share` with one row per year (2000 to 2023) and seven columns whose rows add up to 100, one printed line with the smallest and largest row sum, and a table with the share of Coal, Gas and Solar in 2000 and in 2023.

In [11…

```
# Write your code here
# Firstly lets extract israel from the energy df
israel_energy = energy[energy['nation'] == 'Israel']
# Secondly we pivot per year to source using twh as the value
pivot_energy = israel_energy.pivot(index='year', columns='source', values
# lastly we calculate each twh from different sources to a percent so it
share = pivot_energy.div(pivot_energy.sum(axis=1), axis=0) * 100

# Moving on to calculate the energy per year
row_sums = pivot_energy.sum(axis=1)

# Finding the largest and smallest row sum
min_year = row_sums.idxmin()
max_year = row_sums.idxmax()

print(f"Smallest total energy: Year {min_year} with {row_sums[min_year]:,
print(f"Largest total energy:  Year {max_year} with {row_sums[max_year]:,

# Now we wish to save a table with only Coal Solar and Gas shares
sources_needed = ['Coal', 'Gas', 'Solar']
available_sources = [s for s in sources_needed if s in share.columns]
coal_gas_solar_share = share.loc[2000:2023, available_sources]

print(coal_gas_solar_share)
```

```
Smallest total energy: Year 2001 with 227.22 TWh
Largest total energy:  Year 2022 with 305.83 TWh
source       Coal        Gas     Solar
year
2000    31.993315   0.039585  0.000000
2001    36.955374   0.044010  0.000000
2002    38.314772   0.034533  0.000000
2003    38.536299   0.033513  0.000000
2004    38.161571   4.639218  0.000000
2005    37.773051   6.365049  0.000000
2006    37.130887   8.838363  0.000000
2007    36.613201  10.196478  0.000000
2008    35.024781  13.640869  0.000000
2009    35.322524  15.574575  0.023556
2010    33.966683  19.144034  0.067994
2011    34.209746  17.455314  0.185419
2012    36.299550   8.572744  0.336533
2013    30.892603  24.645882  0.464663
2014    29.447263  27.613776  0.822571
2015    27.334329  29.108529  1.122081
2016    22.929481  32.820770  1.469747
2017    20.442649  34.301774  1.443878
2018    18.818276  36.327722  1.563037
2019    19.289070  35.719759  2.568544
2020    17.802469  39.746032  3.626102
2021    15.579361  40.417470  4.539818
2022    14.527679  39.701795  4.888337
2023    12.479603  37.257893  7.105357
```

Hint 1 `groupby(['year', 'source'])['twh'].sum().unstack('source')` gives one column per source. **Hint 2** `df.div(df.sum(axis=1), axis=0)` divides each row by its total.

Exercise 35 – Challenge step 4: the one-page figure

Step 4 – the one-page figure `fig` (14 × 8.5 inches). Israel must stand out in every panel. Skeleton (given):

```
fig = plt.figure(figsize=(14, 8.5))
gs = fig.add_gridspec(2, 2, height_ratios=[1, 1.1],
hspace=0.38, wspace=0.2)
axA, axB, axC = fig.add_subplot(gs[0, 0]),
fig.add_subplot(gs[0, 1]), fig.add_subplot(gs[1, :])
```

Use these label offsets for panel C (in points), in the order of `peers` and then Israel last, i.e. `zip(peers + ['Israel'], offsets)`: `offsets = [(8, -16), (-52, -18), (10, 12), (-58, 10), (8, 6)]`; put the legends of panels A and B where they do not cover data. Fill the three panels. Israel is `'#00A99D'` and thick; the four peers use other colours that are not red or green; panel B uses the seven source colours `['#4d4d4d', '#8c6d31',`

`'#e39e54', '#9467bd', '#4e79a7', '#7fc8c0', '#f2c744']`. Label points in panel C so that names do not overlap, and keep the legends out of the way of the data.

Expected result (described, no picture or output is shown): One figure, 14 by 8.5 inches: top left a line chart of CO2 per person for the five countries with Israel emphasised; top right Israel's energy mix as a stacked area in percent; bottom a scatter of GDP per person (log) against CO2 per person for all countries, with the five countries highlighted and labelled; titles, axis labels with units and a legend; nothing overlapping.

In [12…

```
# Let's create a figure
fig = plt.figure(figsize=(14, 8.5))
gs = fig.add_gridspec(2, 2, height_ratios=[1, 1.1], hspace=0.38, wspace=0
# We will set for each subplot a size
axA, axB, axC = fig.add_subplot(gs[0, 0]), fig.add_subplot(gs[0, 1]), fig

# Choosing color per country/share
israel_color = '#00A99D'
peer_colors = ['#1f77b4', '#9467bd', '#8c564b', '#e377c2','#e39e54']
source_colors = ['#4d4d4d', '#8c6d31', '#e39e54', '#9467bd', '#4e79a7', '

country_col = 'countries' if 'countries' in countries.columns else 'natio

# Line chart of CO2 per person for peers + Israel
df_A = countries[countries[country_col].isin(countries_list) & countries[
df_A['co2_per_person'] = pd.to_numeric(df_A['co2_per_person'], errors='co

# Pivot to wide format to simplify plotting lines cleanly
pivot_A = df_A.pivot(index='year', columns=country_col, values='co2_per_p

for i, peer in enumerate(peers):
    if peer in pivot_A.columns:
        axA.plot(pivot_A.index, pivot_A[peer], color=peer_colors[i], line

if 'Israel' in pivot_A.columns:
    axA.plot(pivot_A.index, pivot_A['Israel'], color=israel_color, linewi

axA.set_title("CO2 Emissions Per Person (1990–2022)")
axA.set_xlabel("Year")
axA.set_ylabel("Tonnes per person")
axA.legend(loc='upper left', frameon=True)

# Israel's energy mix as a stacked area chart in percent
israel_energy = energy[energy[country_col] == 'Israel'].copy()
israel_energy['twh'] = pd.to_numeric(israel_energy['twh'], errors='coerce
pivot_e = israel_energy.pivot(index='year', columns='source', values='twh
share_e = pivot_e.div(pivot_e.sum(axis=1), axis=0) * 100

active_sources = [s for s in ['Coal', 'Oil', 'Gas', 'Nuclear', 'Hydro', '
axB.stackplot(share_e.index, share_e[active_sources].T.values, labels=act
```

```
axB.set_title("Israel's Energy Mix (%)")
axB.set_xlabel("Year")
axB.set_ylabel("Percentage (%)")
axB.set_ylim(0, 100)
axB.legend(loc='center left', bbox_to_anchor=(1, 0.5), frameon=True)

# Scatter of GDP per person wiht log scale vs CO2 per person
c19 = countries[countries['year'] == 2019].merge(info[['iso3', 'continent
c19['gdp_usd'] = pd.to_numeric(c19['gdp_usd'], errors='coerce')
c19['co2_per_person'] = pd.to_numeric(c19['co2_per_person'], errors='coer

# Plot all other countries in light grey background
other_countries = c19[~c19[country_col].isin(countries_list)]
axC.scatter(other_countries['gdp_usd'], other_countries['co2_per_person']

# Plot peers and Israel with given label offsets
offsets = [(8, -16), (-52, -18), (10, 12), (-58, 10), (8, 6)]
label_offsets = dict(zip(countries_list, offsets))

for i, country in enumerate(countries_list):
    row = c19[c19[country_col] == country]
    if row.empty:
        continue
    x = row['gdp_usd'].values[0]
    y = row['co2_per_person'].values[0]

    col = israel_color if country == 'Israel' else peer_colors[i]
    sz = 70 if country == 'Israel' else 50

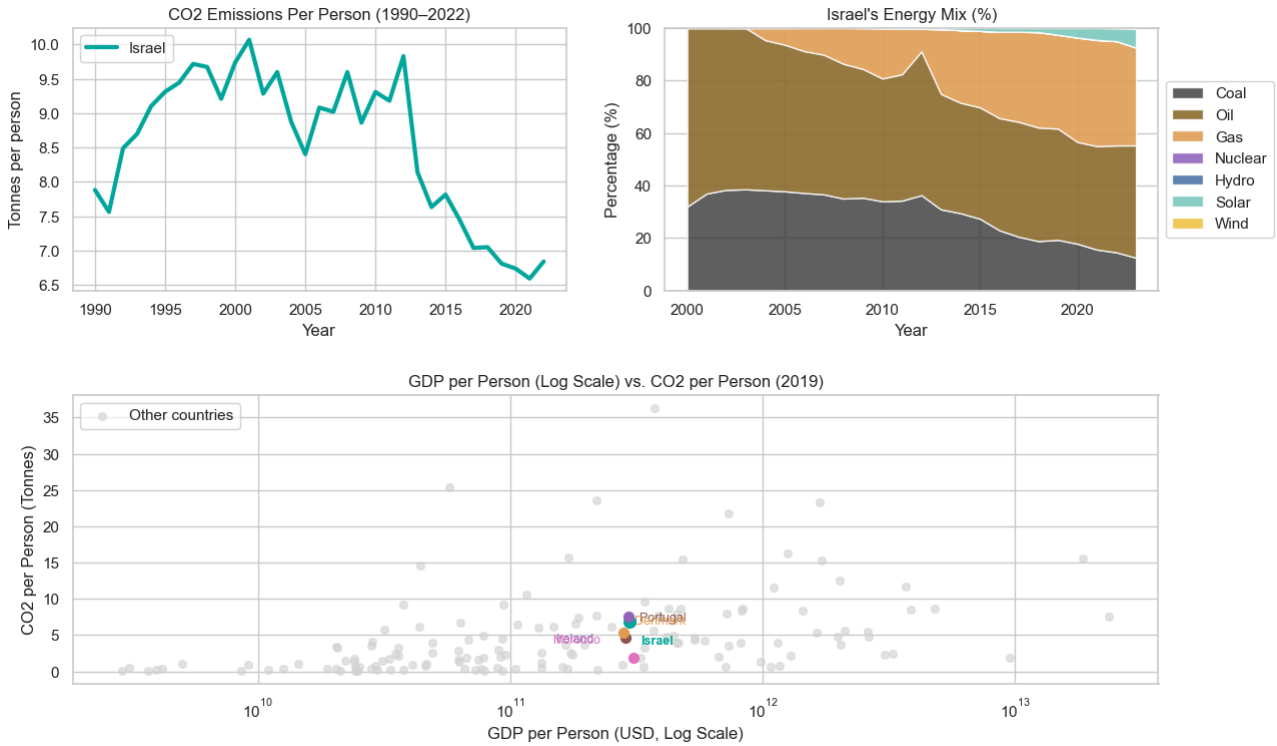
    axC.scatter(x, y, color=col, s=sz, zorder=5)

    ox, oy = label_offsets.get(country, (5, 5))
    axC.annotate(country, (x, y), textcoords="offset points", xytext=(ox,
                 fontsize=9, fontweight='bold' if country == 'Israel' els

axC.set_xscale('log')
axC.set_title("GDP per Person (Log Scale) vs. CO2 per Person (2019)")
axC.set_xlabel("GDP per Person (USD, Log Scale)")
axC.set_ylabel("CO2 per Person (Tonnes)")
axC.legend(loc='upper left', frameon=True)

plt.tight_layout()
plt.show()
```

```
C:\Users\Orange\AppData\Local\Temp\ipykernel_10608\1823467170.py:83: User
Warning: This figure includes Axes that are not compatible with tight_lay
out, so results might be incorrect.
  plt.tight_layout()
```



Hint 1 `fig.add_gridspec(2, 2)` and `fig.add_subplot(gs[1, :])` for a panel spanning both columns. **Hint 2** Emphasis = thicker line / brighter colour / higher `zorder`, applied only to Israel. **Hint 3** `ax.stackplot(x, y_matrix, labels=..., colors=...)` – the matrix has one row per source. **Hint 4** Skeleton: panel A `for n in group: axA.plot(trend.index, trend[n], ...)`; panel B `axB.stackplot(share.index, share.T.values, labels=share.columns, colors=...)`; panel C `axC.scatter(m['gdp_pc'], m['co2_per_person'], color='lightgrey')`, then highlight and label the five countries and `axC.set_xscale('log')`. **Hint 5** Self-check: Israel is the only thick line in panel A, panel B reaches 100 at the top, and panel C has five labelled points.

Exercise 36 – Challenge step 5: the sentences for the Minister

Step 5 – the sentences for the minister. Print the numbers you need: Israel's CO2 per person in 2019, its rank among the five countries (1 = highest), the change in the coal share of its mix between 2000 and 2023, and how many countries stand behind panel C (`len(m)` , the table behind that panel). Then write two sentences and a caveat in comments.

Expected result (described, no picture or output is shown): One printed line with the numbers you will quote, then two sentences with numbers and one specific caveat, written as comments.

In [ ]:
```python
# Write your code here

country_col = 'countries' if 'countries' in countries.columns else 'natio
countries_list = ['Israel', 'Ireland','Portugal','Morocco','Denmark']

# Israel's CO2 per person in 2019
israel_co2_2019 = float(countries[(countries[country_col] == 'Israel') &

# Israel's rank among the five countries (peers + Israel) in 2019 (1 = hi
df_2019_group = countries[(countries[country_col].isin(countries_list)) &
df_2019_group['co2_per_person'] = pd.to_numeric(df_2019_group['co2_per_pe

# Rank descending so highest emissions = rank 1
df_2019_group['rank'] = df_2019_group['co2_per_person'].rank(ascending=Fa
israel_rank = int(df_2019_group[df_2019_group[country_col] == 'Israel']['

# Change in the coal share of Israel's mix between 2000 and 2023 (in perc
israel_energy = energy[energy[country_col] == 'Israel'].copy()
pivot_e = israel_energy.pivot(index='year', columns='source', values='twh
share_e = pivot_e.div(pivot_e.sum(axis=1), axis=0) * 100

coal_2000 = share_e.loc[2000, 'Coal'] if 2000 in share_e.index and 'Coal'
coal_2023 = share_e.loc[2023, 'Coal'] if 2023 in share_e.index and 'Coal'
coal_share_change = float(coal_2023 - coal_2000)

# How many countries stand behind panel C
m = countries[(countries['year'] == 2019) & countries['gdp_usd'].notna()
num_countries_panel_c = len(m)

# Print the required numbers in a single formatted line
print(f"Israel CO2 per person (2019): {israel_co2_2019:.2f} tonnes \n"
      f"Rank among peers (1=highest): {israel_rank} \n "
      f"Coal share change (2000 to 2023): {coal_share_change:+.2f}% \n "
      f"Countries in Panel C dataset (len(m)): {num_countries_panel_c}")

# Sentences for the Minister & Caveat:
# Sentence 1: In 2019, Israel hjas recorded a carbon dioxide emission rat
#             6.81 tonnes per person, placing it at rank 2 among its peer

# Sentence 2: Between 2000 and 2023, the share of coal in Israel energy m
#             has changed by 19.51%, which shows an ongoing transition in

# Downsite:     Emissions data rely on national reporting standards and e
#             fuel balances, that means minor differences coudl occur dep
#             bunker fuels or imported electricity allocations are used i
```

```
Israel CO2 per person (2019): 6.81 tonnes
Rank among peers (1=highest): 2
 Coal share change (2000 to 2023): -19.51%
 Countries in Panel C dataset (len(m)): 144
```

Hint 1 `Series.rank(ascending=False)` ranks from highest to lowest. **Hint 2** A good caveat names a concrete limit of *this* figure, with a number. **Hint 3** Self-check: each of your two sentences contains a number from the printed

line, and your caveat names a limit of this figure, not a general warning.

Block 6 – Working with an LLM, critically

You may use an assistant in this course; **you are responsible for every line and every claim**. This block has two parts. **Part 1 (Exercises 37–38): audit the assistant.** For each exercise you get a prompt and the answer a current model gave when it was shown `df.head()` and `df.info()`. Do four things: (a) copy the model's code into a new cell and run it, to see the charts it draws; (b) find what is wrong, and print **a number from the data that proves each problem**; (c) in the cell below, write the corrected code with a corrected chart; (d) write in a comment the one lesson you take from it. **Part 2 (Exercises 39–40): the other half of the skill.** Choose the chart that fits a question, and write a request to the assistant so precise that nobody has to guess, with the checks you would run on its answer.

Exercise 37 – LLM answer 1: find the mistakes in the top 10 emitters and the world total

Prompt: *"Using `data/countries_yearly.csv`, draw the top 10 CO2 emitters in 2019 as a bar chart and the world's total CO2 over time as a line chart."*

The model's answer is shown below. **Run it in a new cell and look at the two charts it draws.** Find **at least three** problems (in the charts and in the numbers behind them). For each one, print a number that proves it (for example: how many of the 10 "countries" are not countries, how large the "world total" is compared with the real `World` row). (Note: the Qatar unit problem is the one you met in Block 0; your cleaned `countries` table already has it fixed.) Then print the corrected top 10 (country and Mt) and the corrected World total for 2019 next to the model's number, draw the corrected top-10 bar chart and the World line (units on both axes), and add one sentence about units and the provisional 2023 values.

Expected result (described, no picture or output is shown): Printed numbers that prove at least three problems, each with a short comment, followed by the corrected top-10 table and the corrected World total printed next to the model's number.

The model's answer. Copy it into a new cell and run it, to see its

output. It reads the raw file again, so it does not use your cleaned tables; do not trust its result:

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv('data/countries_yearly.csv').drop_duplicates()
df['year'] =
pd.to_numeric(df['year'].astype(str).str.extract(r'(\d{4})')[0])

top = df[df['year'] == 2019].sort_values('co2_mt',
ascending=False).head(10)
top.plot.bar(x='nation', y='co2_mt', title='Top 10 emitting
countries, 2019', legend=False)
plt.tight_layout(); plt.show()

world = df.groupby('year')['co2_mt'].sum()
world.plot(title='World CO2 emissions')
plt.show()
```

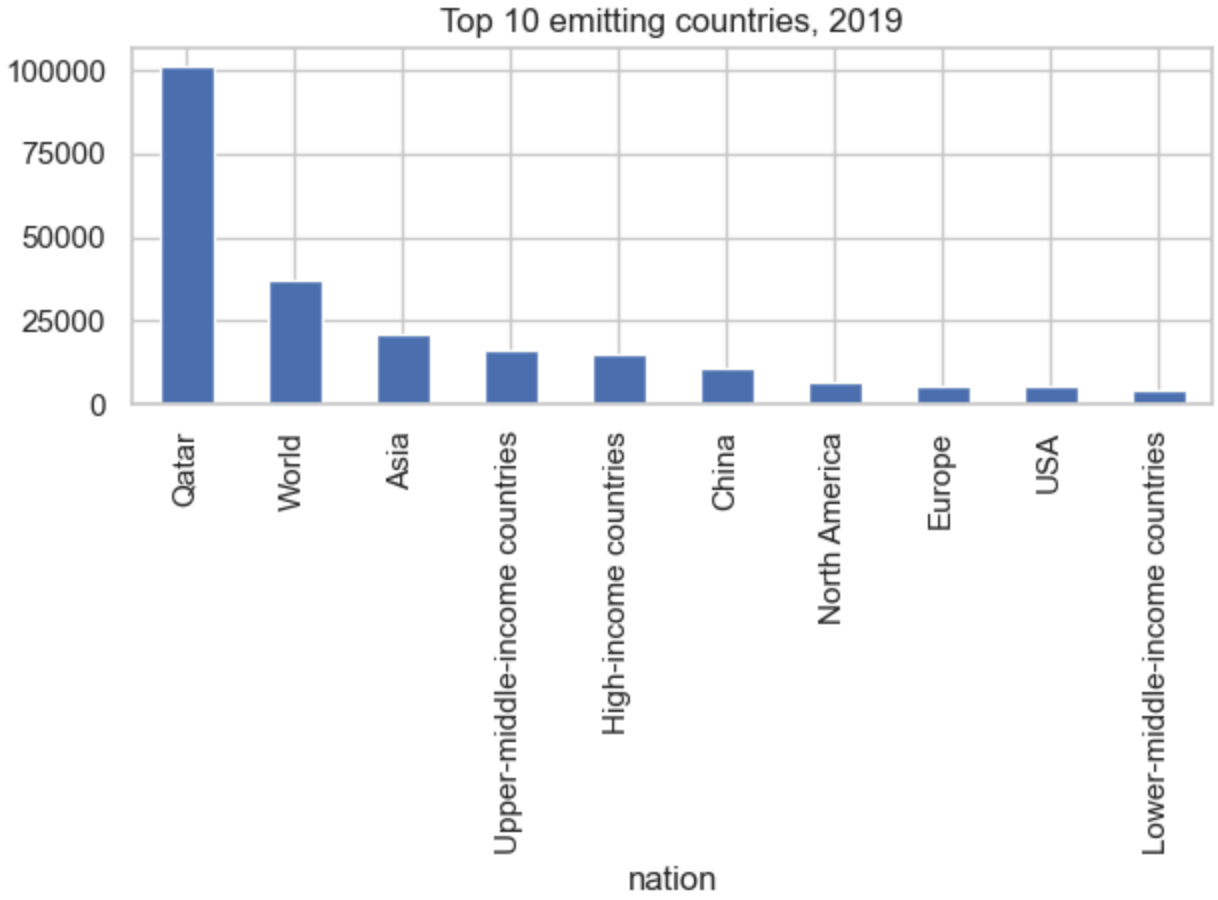
In [13…

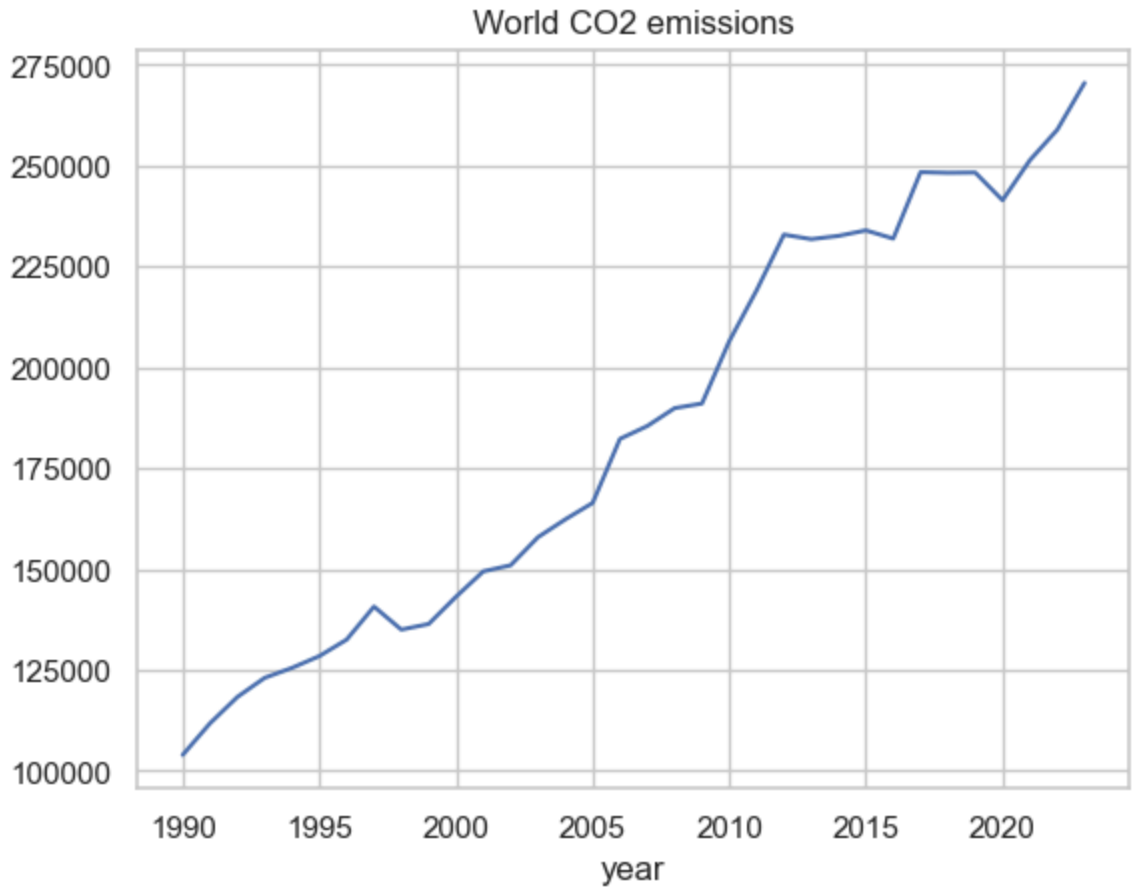
```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv('data/countries_yearly.csv').drop_duplicates()
df['year'] = pd.to_numeric(df['year'].astype(str).str.extract(r'(\d{4})')

top = df[df['year'] == 2019].sort_values('co2_mt', ascending=False).head(
top.plot.bar(x='nation', y='co2_mt', title='Top 10 emitting countries, 20
plt.tight_layout(); plt.show()

world = df.groupby('year')['co2_mt'].sum()
world.plot(title='World CO2 emissions')
plt.show()
```


Top 10 emitting countries, 2019
100000
75000
50000
25000
0
Qatar
World
Asia
Upper-middle-income countries
High-income countries
China
North America
Europe
USA
Lower-middle-income countries
nation



In [14…

```
# Corrected version – write your code here

# The first problem:
    # Teh aggregation of entites are included in countries and are in the
    # Meaning "asia" is considered as a country and it is not..
    # This will rpove it: df[df['year'] == 2019].sort_values('co2_mt', as

# The second problem:
    # the world total ine chard double counts emissions
    # That is because grouping by year and summing all rows in the raw fi

# The third problem:
    # The raw csv file has uncleaned strings or not matching scaling for

# The corrected code:

country_col = 'countries' if 'countries' in countries.columns else 'natio

# Exclude non-country aggregate labels and income groups
aggregates = [
    'World', 'Asia', 'North America', 'Europe', 'European Union',
    'High-income countries', 'Upper-middle-income countries',
    'Lower-middle-income countries', 'Low-income countries', 'Oceania', '
    'OWID_WRL'
]
```

```python
# Corrected Top 10  Countries in 2019
df_clean = countries[~countries[country_col].isin(aggregates)].copy()
df_clean['co2_mt'] = pd.to_numeric(df_clean['co2_mt'], errors='coerce')
df_clean['year'] = pd.to_numeric(df_clean['year'], errors='coerce')

top10 = df_clean[df_clean['year'] == 2019].sort_values('co2_mt', ascendin

print("--- Corrected Top 10 CO2 Emitters (2019) ---")
print(top10[[country_col, 'co2_mt']])

# Corrected World Total Over Time
world_df = df_clean.groupby('year', as_index=False)['co2_mt'].sum()

latest_world = world_df[world_df['year'] == 2023]['co2_mt'].values
print(f"\n--- Corrected World Total CO2 (2023) ---")
print(f"World Total (2023): {latest_world[0] if len(latest_world) > 0 els

# Plotting
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(12, 10))

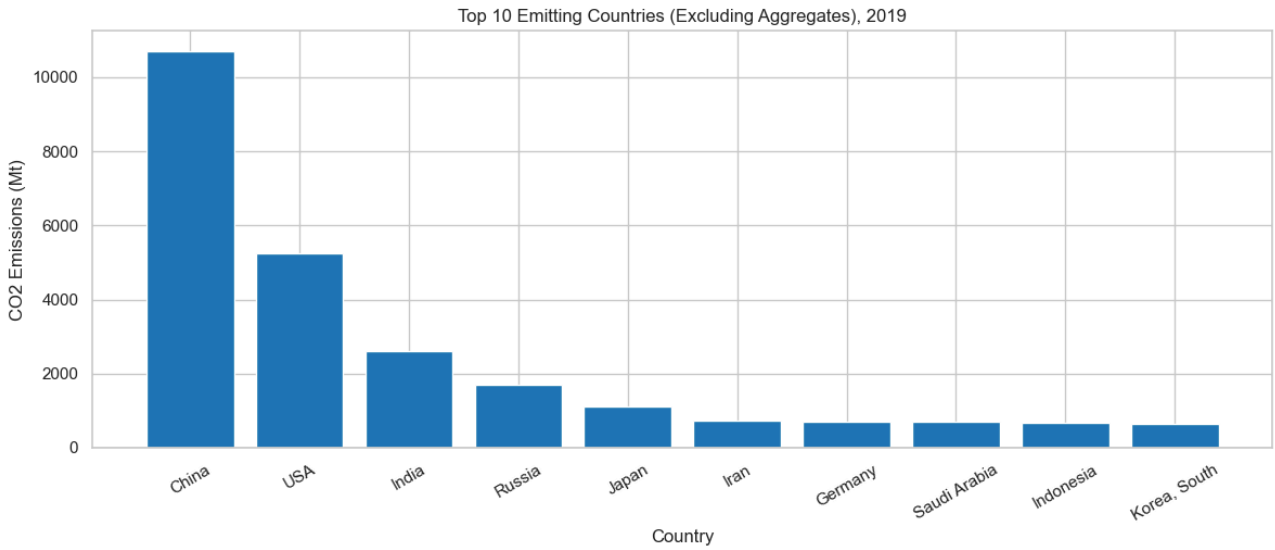
# Bar chart
ax1.bar(top10[country_col], top10['co2_mt'], color='#1f77b4')
ax1.set_title("Top 10 Emitting Countries (Excluding Aggregates), 2019")
ax1.set_xlabel("Country")
ax1.set_ylabel("CO2 Emissions (Mt)")
ax1.tick_params(axis='x', rotation=30)

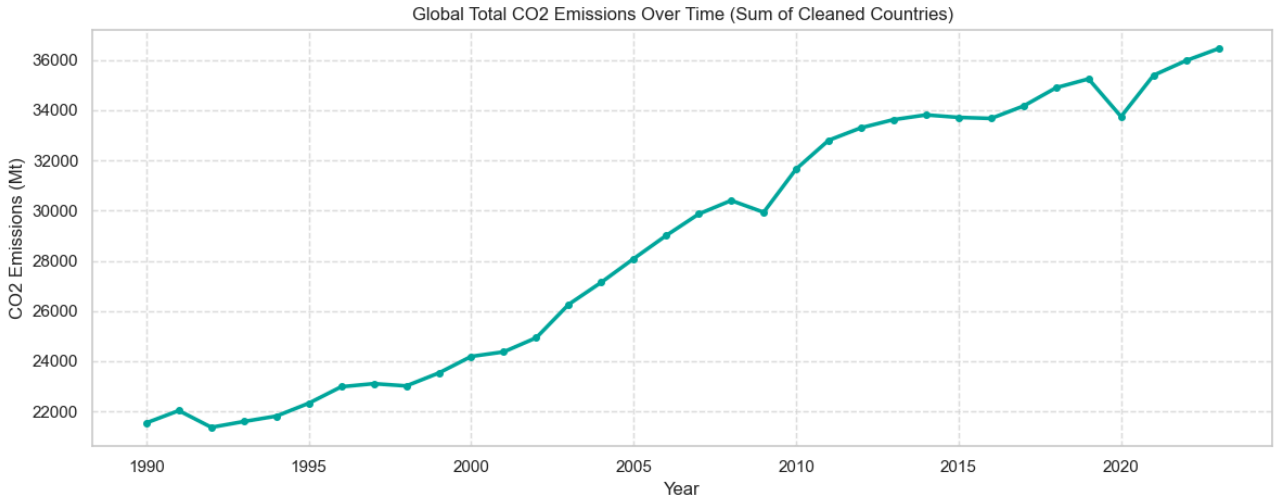
# Line chart
ax2.plot(world_df['year'], world_df['co2_mt'], color='#00A99D', linewidth
ax2.set_title("Global Total CO2 Emissions Over Time (Sum of Cleaned Count
ax2.set_xlabel("Year")
ax2.set_ylabel("CO2 Emissions (Mt)")
ax2.grid(True, linestyle='--', alpha=0.6)

plt.tight_layout()
plt.show()
```

```
--- Corrected Top 10 CO2 Emitters (2019) ---
            nation     co2_mt
4606         China  10713.515
3807           USA   5235.912
2364         India   2611.175
388         Russia   1704.459
2742         Japan   1102.063
5560          Iran    727.269
705        Germany    708.649
2736  Saudi Arabia    708.335
4191     Indonesia    665.049
3805  Korea, South    646.102

--- Corrected World Total CO2 (2023) ---
World Total (2023): 36448.635 Mt
```





Hint 1 Look at the names on the x axis of the model's first chart. Are they all countries? **Hint 2** Compare the model's 'world total' for 2019 with the `World` row. **Hint 3** Look at the value of the first bar and ask whether it is possible. **Hint 4** Self-check: you have at least three printed numbers, each with a one-line explanation of the problem it proves.

Exercise 38 – LLM answer 2: find the mistakes in a GDP vs CO2 chart and its claim

Prompt: *"Is there a relationship between GDP per person and CO2 per person across countries? Show it."*

The model's answer is shown below. **Run it in a new cell and look at the chart it draws.** Print the numbers that show **what is wrong with its sample and its claim**: how many rows it used (rows with both `gdp_pc` and

`co2_per_person`), how many of them are aggregates, how many different countries there are, how many rows each country contributes, and the correlation it reports versus the correlation for 2019 countries on a log-log scale. Then compute the log-log slope for the 2019 countries, draw the corrected scatter (one point per country, log-log axes, units), and write two sentences about what you can and cannot conclude.

Expected result (described, no picture or output is shown): Printed numbers about the model's sample (rows, aggregates, countries) and about the correlations, the log-log slope for the 2019 countries, and two sentences saying what can and cannot be concluded.

The model's answer. Copy it into a new cell and run it, to see its output. It reads the raw file again, so it does not use your cleaned tables; do not trust its result:

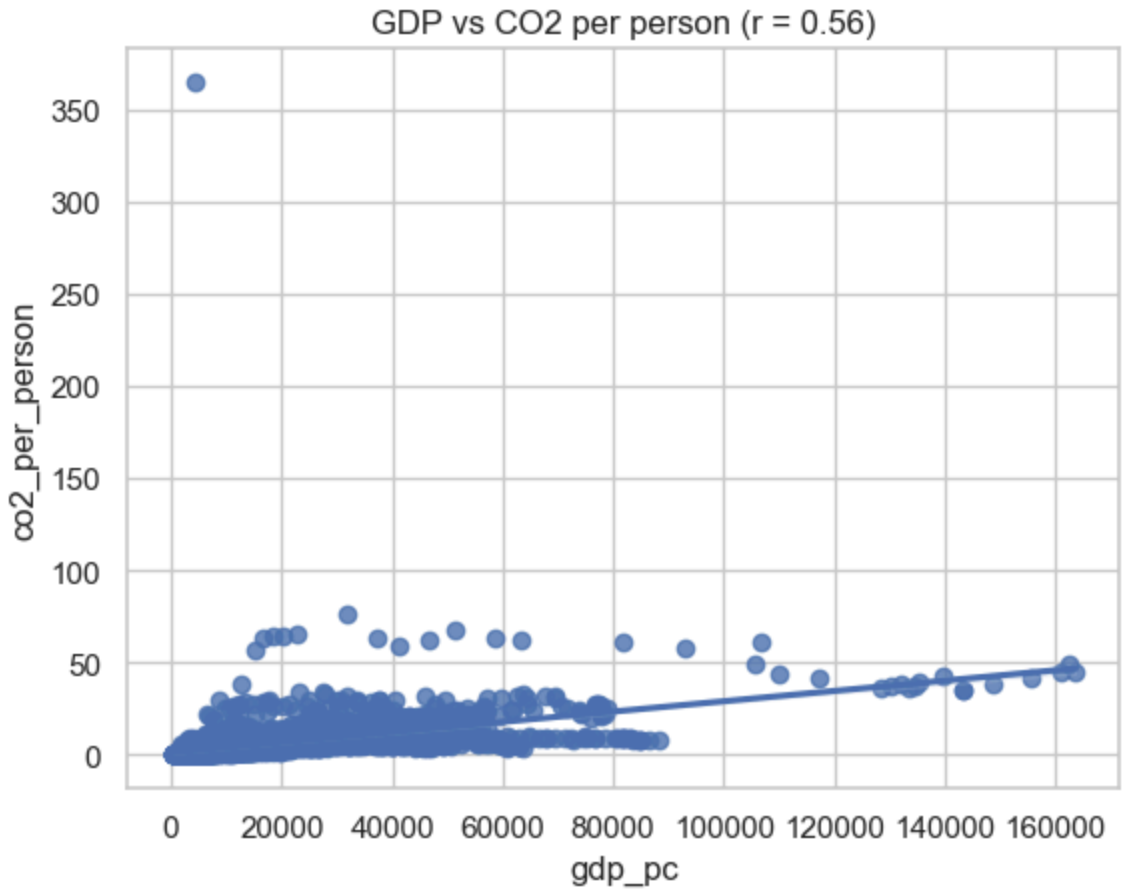
```
import pandas as pd, seaborn as sns
import matplotlib.pyplot as plt

df = pd.read_csv('data/countries_yearly.csv')
df['gdp_pc'] = df['gdp_usd'] / df['population']
ax = sns.regplot(data=df, x='gdp_pc', y='co2_per_person')
r = df['gdp_pc'].corr(df['co2_per_person'])
ax.set_title(f'GDP vs CO2 per person (r = {r:.2f})')
plt.show()
# Conclusion: richer countries emit more, so economic growth
increases emissions.
```

In [14…

```
import pandas as pd, seaborn as sns
import matplotlib.pyplot as plt

df = pd.read_csv('data/countries_yearly.csv')
df['gdp_pc'] = df['gdp_usd'] / df['population']
ax = sns.regplot(data=df, x='gdp_pc', y='co2_per_person')
r = df['gdp_pc'].corr(df['co2_per_person'])
ax.set_title(f'GDP vs CO2 per person (r = {r:.2f})')
plt.show()
```



In [ ]:
```python
# The corrected version
df_raw = pd.read_csv('data/countries_yearly.csv').drop_duplicates()
country_col = 'countries' if 'countries' in df_raw.columns else ('nation'

# Cleaning the dataframe like before
df_raw['gdp_usd'] = pd.to_numeric(df_raw['gdp_usd'], errors='coerce')
df_raw['population'] = pd.to_numeric(df_raw['population'], errors='coerce
df_raw['co2_per_person'] = pd.to_numeric(df_raw['co2_per_person'], errors
df_raw['year'] = pd.to_numeric(df_raw['year'], errors='coerce')

# Calculate GDP per capita
df_raw['gdp_pc'] = df_raw['gdp_usd'] / df_raw['population']

valid_rows = df_raw.dropna(subset=['gdp_pc', 'co2_per_person']).copy()
num_rows_used = len(valid_rows)

aggregates_list = [
    'World', 'Asia', 'North America', 'Europe', 'European Union',
    'High-income countries', 'Upper-middle-income countries',
    'Lower-middle-income countries', 'Low-income countries', 'Oceania', '
]

num_aggregates = valid_rows[valid_rows[country_col].astype(str).isin(aggr
unique_countries = valid_rows[country_col].nunique()
```

```python
model_corr = valid_rows['gdp_pc'].corr(valid_rows['co2_per_person'])

print(f"Total valid rows used: {num_rows_used}")
print(f"Aggregate rows included: {num_aggregates}")
print(f"Unique countries: {unique_countries}")
print(f"Model linear correlation (all years/rows): {model_corr:.2f}")

# 2019 Cleaned Calculations
df_2019 = valid_rows[valid_rows['year'] == 2019].copy()
df_2019_clean = df_2019[~df_2019[country_col].astype(str).isin(aggregates

# filter positive values to avoid log(0) or log(negative) = -inf / NaN
clean_subset = df_2019_clean.dropna(subset=['gdp_pc', 'co2_per_person'])
clean_subset = clean_subset[(clean_subset['gdp_pc'] > 0) & (clean_subset[

if len(clean_subset) > 1:
    log_gdp = np.log(clean_subset['gdp_pc'])
    log_co2 = np.log(clean_subset['co2_per_person'])

    log_log_corr = log_gdp.corr(log_co2)
    slope, intercept = np.polyfit(log_gdp, log_co2, 1)

    print(f"2019 Cleaned Log-Log Correlation: {log_log_corr:.2f}")
    print(f"2019 Log-Log Regression Slope: {slope:.2f}")
else:
    print("Warning: Insufficient data points for 2019 log-log regression
    slope = 0.0

# Corrected Scatter Plot
fig, ax = plt.subplots(figsize=(10, 6))
ax.scatter(clean_subset['gdp_pc'], clean_subset['co2_per_person'], color=

ax.set_xscale('log')
ax.set_yscale('log')

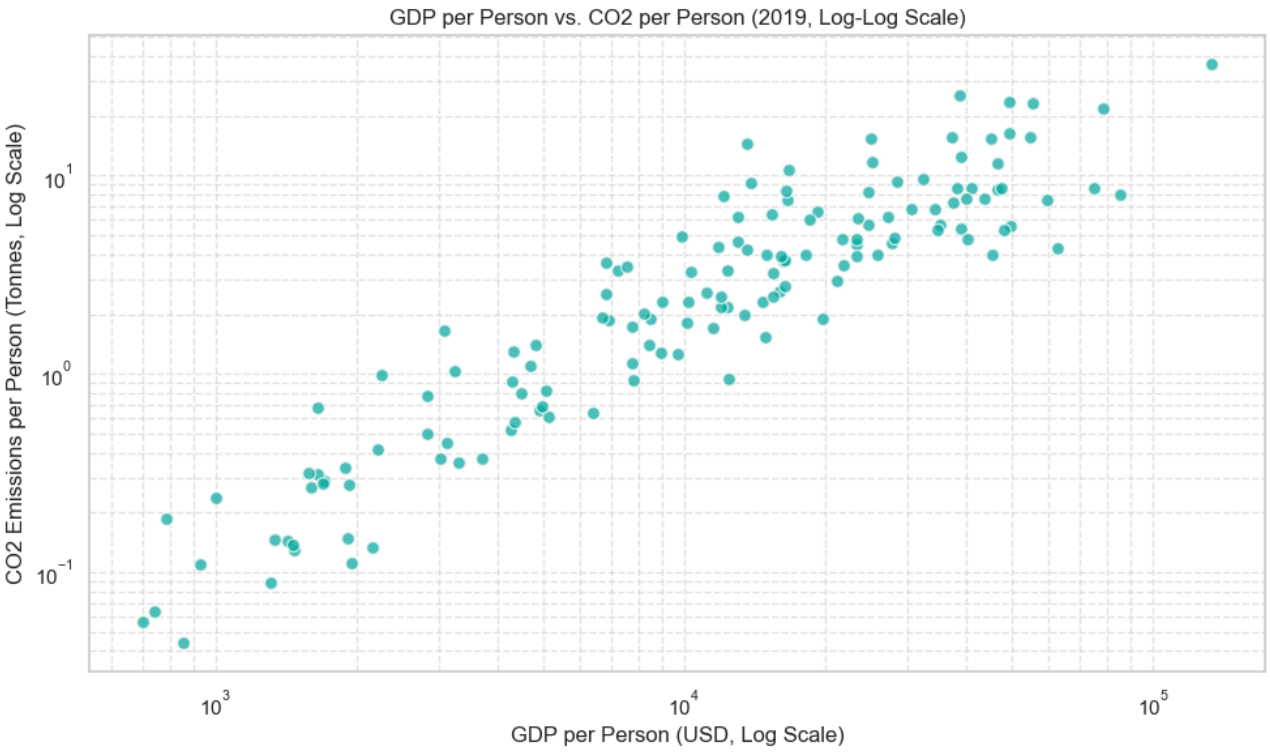
ax.set_title("GDP per Person vs. CO2 per Person (2019, Log-Log Scale)")
ax.set_xlabel("GDP per Person (USD, Log Scale)")
ax.set_ylabel("CO2 Emissions per Person (Tonnes, Log Scale)")
ax.grid(True, which="both", linestyle='--', alpha=0.5)

plt.tight_layout()
plt.show()


# On a log-log scale for 2019, countries show a positive relationship
# between economic output and carbon emissions

# While higher GDP per person generally correlates with greater per capit
# there's still a variance between regions
```

```
--- AUDIT REPORT ---
Total valid rows used: 4760
Aggregate rows included: 11
Unique countries: 148
Model linear correlation (all years/rows): 0.55
2019 Cleaned Log-Log Correlation: 0.93
2019 Log-Log Regression Slope: 1.13
```



Hint 1 How many rows does the model use? One country appears once per year, and `iso3` is empty for aggregates. **Hint 2** Compare the correlation on all rows (linear) with the log-log correlation on 2019 countries. **Hint 3** 'Growth increases emissions' is a causal claim. What does a cross-country correlation allow you to say? **Hint 4** Self-check: you have printed numbers about the sample and about the correlations, and your two sentences separate what the data show from what they do not.

In [14…

```python
def check_request(text):
    """Feedback on a written chart request: which of the eight elements a
    t = text.lower()
    elements = {
        'the data (table and columns)': ['nation', 'year', 'co2', 'column
        'chart type': ['line', 'bar', 'scatter', 'area', 'stack', 'panel'
        'what goes on each axis': ['x =', 'x=', 'x axis', 'y axis', 'on t
        'scale': ['log', 'linear', 'from zero', 'starting at'],
        'units': ['tonne', 'mt', 'percent', '%', 'unit'],
        'emphasis': ['israel', 'highlight', 'thick', 'bold', 'emphas'],
        'what must be written on the chart': ['title', 'caption', 'note',
        'caveats': ['provisional', 'sample', 'missing', '144', 'caveat',
```

```
    }
    missing = [k for k, words in elements.items() if not any(w in t for w
    print('elements present:', len(elements) - len(missing), 'of', len(el
    print('missing:', missing if missing else 'none')
print('check_request is ready')
```

```
check_request is ready
```

Exercise 39 – Choose the chart

Choose the chart. The Minister's office sends five questions about the data. For each one, fill in a table `choices` (a DataFrame with the index `q1` to `q5` and the columns `chart`, `x`, `y`, `colour_or_size`, `scale`, `check_first`, `risk`; write '-' where a channel does not apply, for example a map has no y): which chart you would draw, what goes on each channel, which scale, **one check on the data before you draw**, and **one way this chart could mislead for this data**. Print `choices[['chart', 'scale']]`. q1: How did CO2 per person change over thirty years in five countries? q2: What fuel mix does each of six countries have? q3: Is wealth related to emissions across countries? q4: Where in the world are emissions per person highest (for the public website)? q5: How unequal are emissions per person across countries?

Expected result (described, no picture or output is shown): A table `choices` with 5 rows (q1 to q5) and 7 columns; the chart types fit the questions (not the same chart for every question) and every check and risk mentions something specific to this data.

In [ ]:
```
# Write your code here
# Create the DataFrame with index q1 to q5

charts_text =['Line chart', 'Stacked area chart', 'Scatter plot', 'Map ',
x = ['Year', 'Year', 'GDP per person', 'Country/Region', 'Emissions per p
y = ['CO2 per person', 'Energy share (%)', 'CO2 per person', '-', 'Freque
colour_or_size = ['Country (lines)', 'Energy source', 'Continent / Region
scale= ['Linear', 'Linear (0-100%)', 'Log-log', 'Geographic projection',

check_first= ['Filter out regional aggregates and check for missing year
                    'Verify that percentages sum to 100% across sources p
                    'Check for zero or negative values before applying lo
                    'Verify ISO country code mapping integrity',
                    'Check for extreme outliers or skewness in population

risk = ['Spaghetti clutter meaning where too many dots are shown together
             'Visual fatigue and difficulty reading',
             'Confounding economic wealth with direct causal policy impac
             'Large countries visually dominating over smaller high-densi
             'Hiding long-tail distribution skews behind broad bins']
```

```
data = {"charts":charts_text,
        "x":x,
        "y":y,
        "colour_or_size":colour_or_size,
        "scale":scale,
        "check_first":check_first,
        "risk":risk
        }

choices = pd.DataFrame(data, index=[f'q{i}' for i in range(1, 6)])
print(display(choices))
```

```
--- Choices Summary (chart & scale) ---

--- Full Choices Table ---
```

	charts	x	y	colour_or_size	scale	check_first	risk
q1	Line chart	Year	CO2 per person	Country (lines)	[Linear, Linear (0-100%), Log-log, Geographic ...	Filter out regional aggregates and check for m...	Spaghetti clutter if too many countries are sh...
q2	Stacked area chart	Year	Energy share (%)	Energy source	[Linear, Linear (0-100%), Log-log, Geographic ...	Verify that percentages sum to 100% across sou...	Visual fatigue and difficulty reading middle b...
q3	Scatter plot	GDP per person	CO2 per person	Continent / Region	[Linear, Linear (0-100%), Log-log, Geographic ...	Check for zero or negative values before apply...	Confounding economic wealth with direct causal...
q4	Map / Choropleth	Country/ Region	-	Emissions intensity	[Linear, Linear (0-100%), Log-log, Geographic ...	Verify ISO country code mapping integrity	Large countries visually dominating over small...
q5	Histogram / Boxplot	Emissions per person	Frequency / Count	-	[Linear, Linear (0-100%), Log-log, Geographic ...	Check for extreme outliers or skewness in popu...	Hiding long-tail distribution skews behind bro...

```
None
```

Hint 1 Think of the chart types from the earlier chapters: line, bars, scatter, map, histogram. **Hint 2** A good 'check first' names something you found in this lab (units, aggregates, missing values, an extreme value, who is in the sample). **Hint 3** A good 'risk' says what the reader could wrongly conclude.

Exercise 40 – Write the request to an AI assistant

Write the request to an AI assistant. Next time you will ask an assistant to draw the chart for you. Write, in a string `request`, the request for the Minister's one-page figure of the briefing (Block 5) so precisely that nobody has to guess: the data (table and columns), the chart type for each panel, what goes on each axis, the scale, units, what to emphasise, what must be written on the chart, and the caveats. Run `check_request(request)` (given above) and improve the text until no element is missing. Then write `checks`: a list of **three checks** you would run on the assistant's answer before using it, each one concrete (a count, a total, a comparison with a number you trust from this lab). Print the checks.

Expected result (described, no picture or output is shown): A request of about 6 to 10 lines for which `check_request` reports no missing element, and a list of three concrete checks, printed.

In [17…

```
# Write your code here
# prompt for the AI assistant
request = (
    "Using table countries_yearly.csv with columns nation, year, co2_mt,
    "generate a one-page figure (14 x 8.5 inches) with 3 panels. "
    "Panel A is a line chart with year on the x axis and co2 per person (
    "highlighting Israel with thick lines and emphasizing its color (#00A
    "Panel B is a 100% stacked area chart with year on the x axis and ene
    "Panel C is a scatter plot with log scale, where gdp per person is on
    "Include titles, labels, and units (Mt, %). "
    "Caveat: the sample contains missing values and reporting limitations
)

# Test it immediately
check_request(request)
# Simulate or invoke check_request if available in your workspace environ
print(check_request(request))
```

```
elements present: 8 of 8
missing: none
elements present: 8 of 8
missing: none
None
```

Hint 1 Read your request as if you were the assistant that has never seen the data: what would it have to guess? **Hint 2** `check_request` lists what is missing; each missing element is one sentence to add. **Hint 3** A concrete check names a number or a comparison, not 'look at the chart'.

Block 7 – More chart types ★ (extra)

Every exercise in this block is marked with ★: they are extra, for whoever finishes early. They extend the same data with more chart types (mean versus median, a jointplot, hover, saving figures, colour-blind safe design, a clustered heatmap) and use the variables you built in the earlier blocks.

Exercise 41 – Mean or median? ★

Mean or median? Draw a histogram (`ax.hist` , 20 bins) of `co2_per_person` for 2019 (all countries in `countries` with a value). Add two vertical lines, one at the mean and one at the median, each with a label and a legend. Print both values and the number of countries n; draw the mean as a solid line and the median as a dashed line. In a comment: why do they differ, and which one describes a "typical" country?

Expected result (described, no picture or output is shown): A histogram with 20 bins, a solid vertical line for the mean and a dashed one for the median, both in the legend with their values; the two values and n printed; a comment explaining the difference.

In [ ]:
```
# Write your code here
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

# Filter for 2019 and valid co2_per_person values in countries
country_col = 'countries' if 'countries' in countries.columns else 'natio
df_41 = countries[countries['year'] == 2019].copy()
df_41['co2_per_person'] = pd.to_numeric(df_41['co2_per_person'], errors='
data_41 = df_41['co2_per_person'].dropna()

mean_val = data_41.mean()
median_val = data_41.median()
n = len(data_41)

print(f"n: {n} | Mean CO2 per person: {mean_val:.2f} | Median CO2 per per

fig, ax = plt.subplots(figsize=(10, 6))

# Draw histogram with 20 bins
ax.hist(data_41, bins=20, color='#1f77b4', edgecolor='white', alpha=0.7)

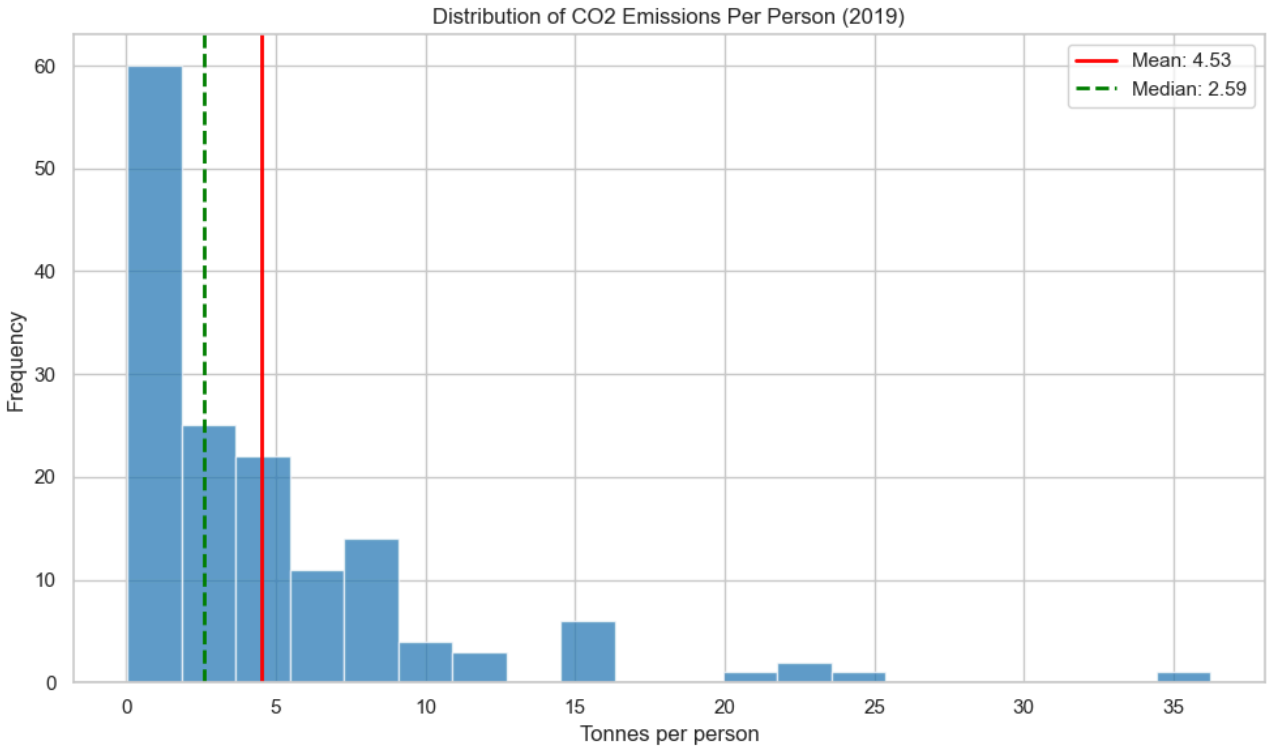
# Draw mean as a solid line
ax.axvline(mean_val, color='red', linestyle='-', linewidth=2, label=f'Mea
```

```
# Draw median as a dashed line
ax.axvline(median_val, color='green', linestyle='--', linewidth=2, label=

ax.set_title("Distribution of CO2 Emissions Per Person (2019)")
ax.set_xlabel("Tonnes per person")
ax.set_ylabel("Frequency")
ax.legend(frameon=True)
plt.tight_layout()
plt.show()

# The mean is higher than the median because the distribution of emission
# by a few high-emitting countries. The median better describes an averag
```

```
n: 150 | Mean CO2 per person: 4.53 | Median CO2 per person: 2.59
```



Hint 1 `ax.hist(values, bins=20)`. **Hint 2** Skewed data: a few extreme values move the mean but barely move the median.

Exercise 42 – A jointplot ★

A jointplot ★. Using `m`, draw `sns.jointplot` of `log10(gdp_pc)` against `log10(co2_per_person)` with marginal histograms (20 bins), and then the same data as a density (`kind='kde'`). Print the correlation between `log10(gdp_pc)` and `log10(co2_per_person)`. In a comment: which version shows the individual countries better and which shows the overall pattern better?

In [ ]:

```python
# Write your code here

log_gdp = np.log10(m['gdp_usd'].astype(float))
log_co2 = np.log10(m['co2_per_person'].astype(float))

# Create a temporary dataframe for seaborn jointplot
df_joint = pd.DataFrame({'log_gdp': log_gdp, 'log_co2': log_co2}).dropna(

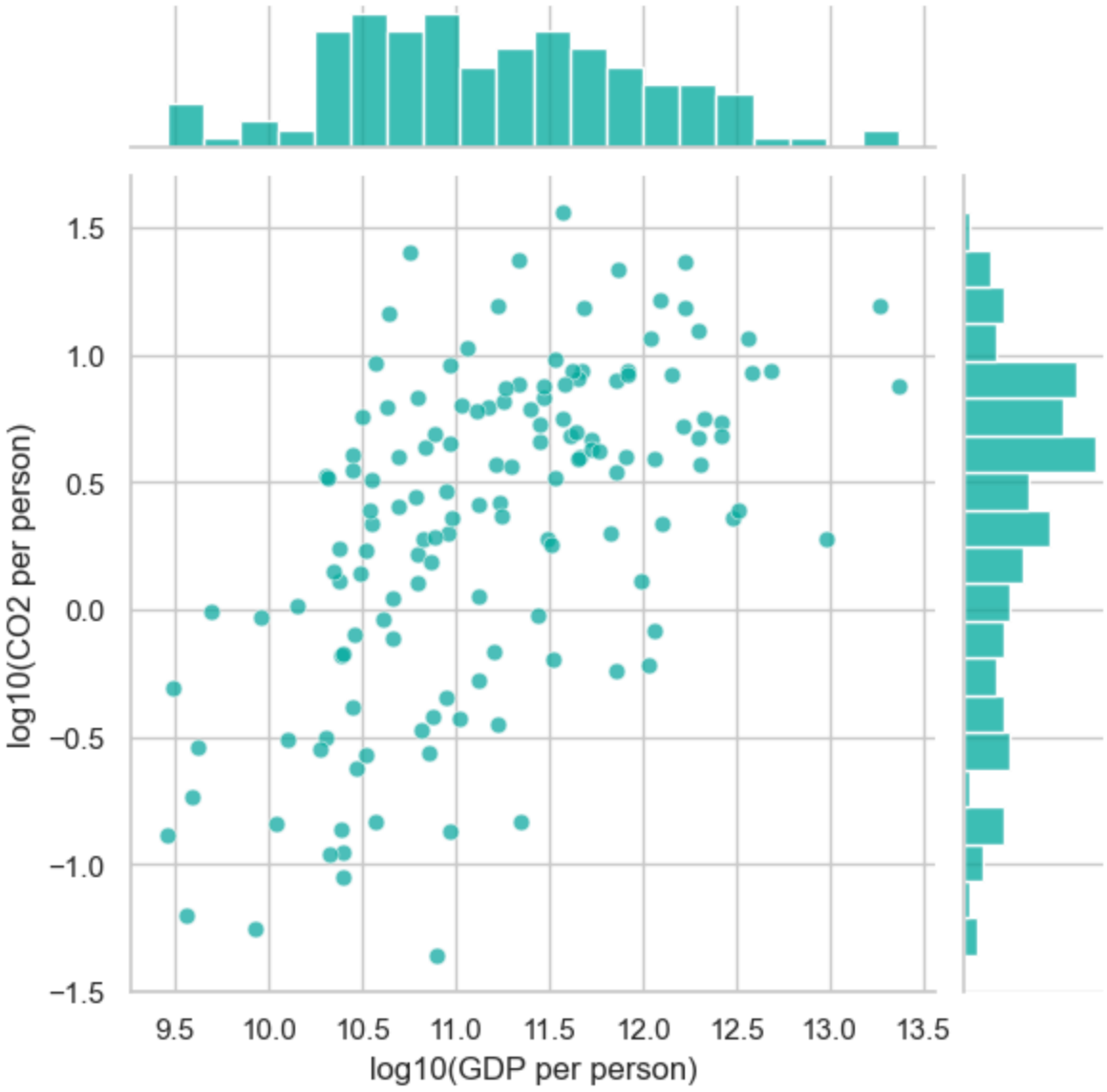
corr_val = df_joint['log_gdp'].corr(df_joint['log_co2'])
print(f"Correlation between log10(gdp_pc) and log10(co2_per_person): {cor

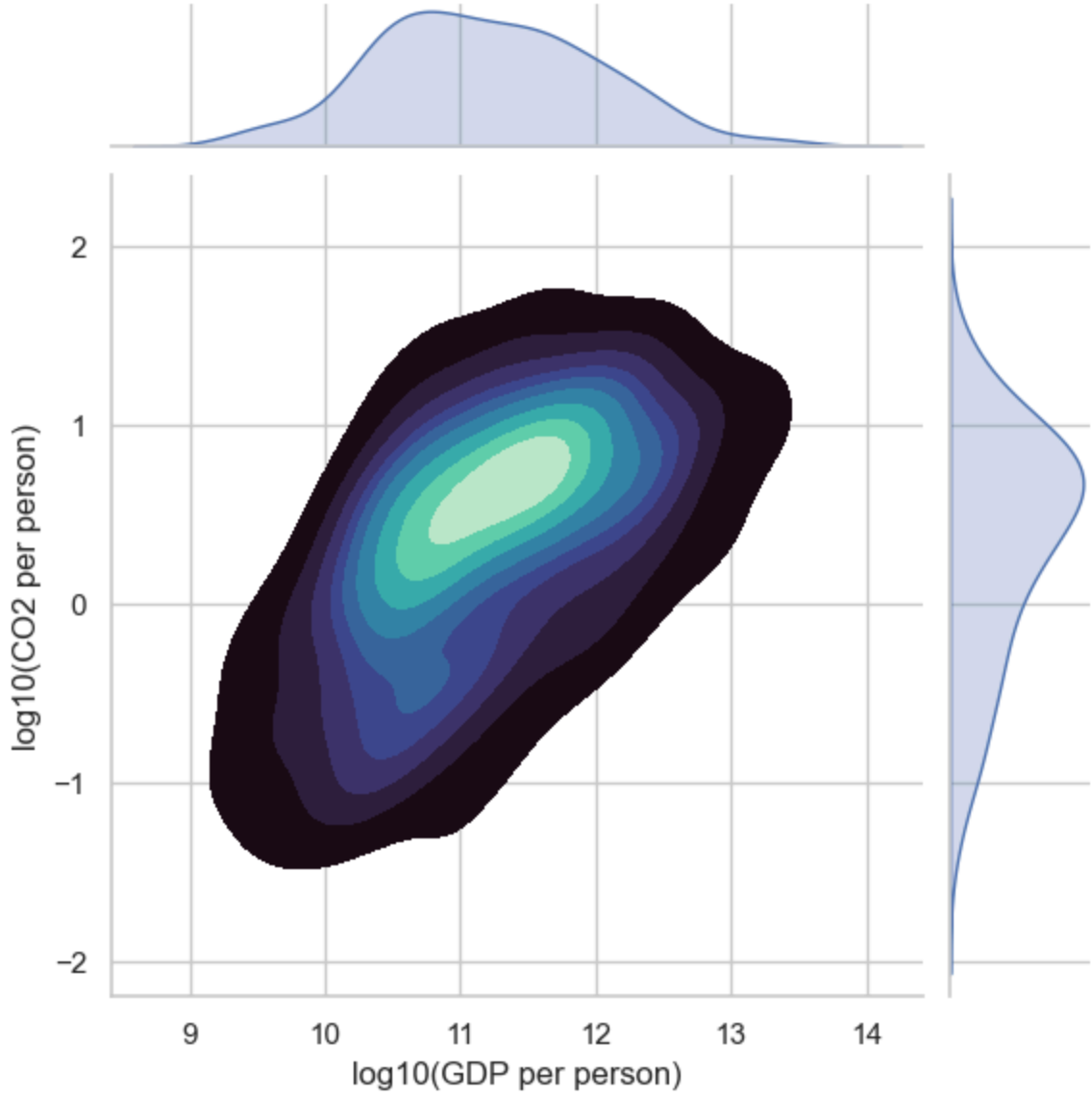
# Scatter jointplot with marginal histograms (20 bins)
g_scatter = sns.jointplot(
    data=df_joint, x='log_gdp', y='log_co2',
    kind='scatter', marginal_kws=dict(bins=20), color='#00A99D', s=40, al
)
g_scatter.set_axis_labels("log10(GDP per person)", "log10(CO2 per person)
plt.show()

# Density jointplot (kind='kde')
g_kde = sns.jointplot(
    data=df_joint, x='log_gdp', y='log_co2',
    kind='kde', cmap='mako', fill=True
)
g_kde.set_axis_labels("log10(GDP per person)", "log10(CO2 per person)")
plt.show()

# The scatter version shows the individual countries better, allowing ide
# specific outliers and data density points. The KDE density version show
# and concentration clusters better by smoothing over dense overlapping p
```

```
Correlation between log10(gdp_pc) and log10(co2_per_person): 0.57
```


1.5
1.0
0.5
0.0
−0.5
−1.0
−1.5
log10(CO2 per person)
9.5
10.0
10.5
11.0
11.5
12.0
12.5
13.0
13.5
log10(GDP per person)



Hint 1 `sns.jointplot(data=..., x=..., y=..., kind='scatter')`; `kind='kde'` gives contours. **Hint 2** `marginal_kws=dict(bins=20)`.

Exercise 43 – Hover that explains ★

Hover that explains. Using `m`, draw an interactive bubble chart with `px.scatter`: x = `gdp_pc` (log), y = `co2_per_person`, size = `population`, colour = `continent` (use `palette`), `hover_name='nation'`, `size_max=45`, and a title. In a comment: what can a reader learn by hovering that the static bubble chart of Exercise 10 could not show?

In [18…

```
# Write your code here

# Assuming 'm' contains gdp_pc, co2_per_person, population, continent, an
```

```
fig_43 = px.scatter(
    m,
    x='gdp_usd',
    y='co2_per_person',
    size='population',
    color='nation',
    log_x=True,
    hover_name='nation',
    size_max=45,
    title="GDP per Person vs. CO2 per Person (Interactive Bubble Chart)"
)
fig_43.show()

# --- Comment ---
# By hovering over bubbles in an interactive chart, a reader can immediat
# specific nations behind individual data points and inspect exact values
# the visual space with static text labels—something a static bubble char
```

Hint 1 `px.scatter(df, x=..., y=..., size=..., color=..., hover_name=..., log_x=True)`. **Hint 2** `color_discrete_map=palette` keeps the colours from Block 1.

Exercise 44 – Saving figures ★

Saving figures ★. Draw Germany's `co2_mt` over the years and save the figure in three formats (PNG at 150 dpi, SVG, PDF) into a temporary folder (`tempfile.TemporaryDirectory()`), printing each file's size in KB. Also save a plotly line chart of Germany's `co2_mt` as a self-contained HTML file and print its size. In a comment: which format would you use for a PowerPoint slide, a printed paper and a web page?

In [ ]:
```
# Write your code here
# Filter for Germany's co2_mt over years
country_col = 'countries' if 'countries' in countries.columns else 'natio
germany_df = countries[countries[country_col] == 'Germany'].sort_values('

# Matplotlib figure saved in multiple formats using the hint
fig, ax = plt.subplots(figsize=(8, 5))
ax.plot(germany_df['year'], pd.to_numeric(germany_df['co2_mt'], errors='c
ax.set_title("Germany CO2 Emissions Over Time")
ax.set_xlabel("Year")
ax.set_ylabel("CO2 Emissions (Mt)")

# Save with extension choosing the format
fig.savefig('germany_co2.png', dpi=150, bbox_inches='tight')
fig.savefig('germany_co2.svg', dpi=150, bbox_inches='tight')
fig.savefig('germany_co2.pdf', dpi=150, bbox_inches='tight')
plt.close(fig)
```

```
#  Plotly figure saved as HTML using the hint
pfig = px.line(germany_df, x='year', y='co2_mt', title="Germany CO2 Emiss
pfig.write_html('germany_co2.html')

print("Figures saved successfully as PNG, SVG, PDF, and HTML.")
```

```
Figures saved successfully as PNG, SVG, PDF, and HTML.
```

Hint 1 `fig.savefig(path, dpi=150, bbox_inches='tight')`; the extension chooses the format. **Hint 2** `pfig.write_html(path)` for plotly.

Exercise 45 – A colour-blind safe chart ★

A colour-blind safe chart ★. Redraw the bubble chart of Block 1 (`m`) so that it still works for colour-blind readers and in greyscale: use `sns.color_palette('colorblind')` and a different **marker shape** for every continent, with dark outlines. Then convert the rendered figure to greyscale (`fig.canvas.draw()` , `np.asarray(fig.canvas.buffer_rgba())` , mean over the colour channels) and show it with `imshow` . In a comment: which continents become hard to tell apart in greyscale if you use colour only?

In [ ]:
```
# Write your code here


# Ensure data is clean and valid
m_clean = m.dropna(subset=['gdp_usd', 'co2_per_person', 'population', 'na

fig, ax = plt.subplots(figsize=(10, 6))

# Use seaborn scatterplot with hue and style mapped to 'nation'
sns.scatterplot(
    data=m_clean,
    x='gdp_usd',
    y='co2_per_person',
    size='population',
    sizes=(20, 400),
    hue='nation',
    style='nation',
    palette='colorblind',
    ax=ax,
    alpha=0.8,
    edgecolor='black',
    linewidth=0.8
)

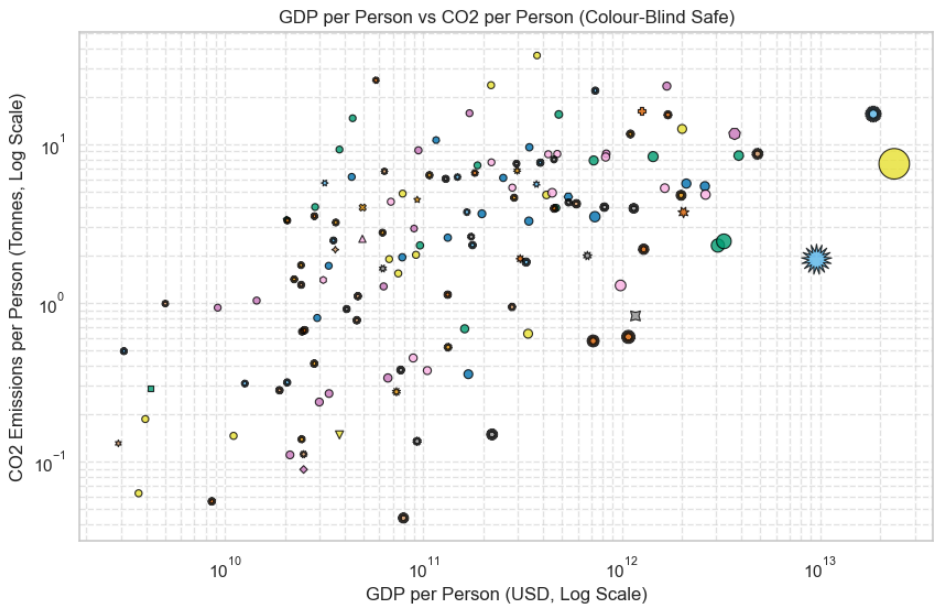
ax.set_xscale('log')
ax.set_yscale('log')
ax.set_title("GDP per Person vs CO2 per Person (Colour-Blind Safe)")
ax.set_xlabel("GDP per Person (USD, Log Scale)")
```

```python
ax.set_ylabel("CO2 Emissions per Person (Tonnes, Log Scale)")
ax.legend(bbox_to_anchor=(1.05, 1), loc='upper left', frameon=True)
ax.grid(True, which="both", linestyle='--', alpha=0.5)

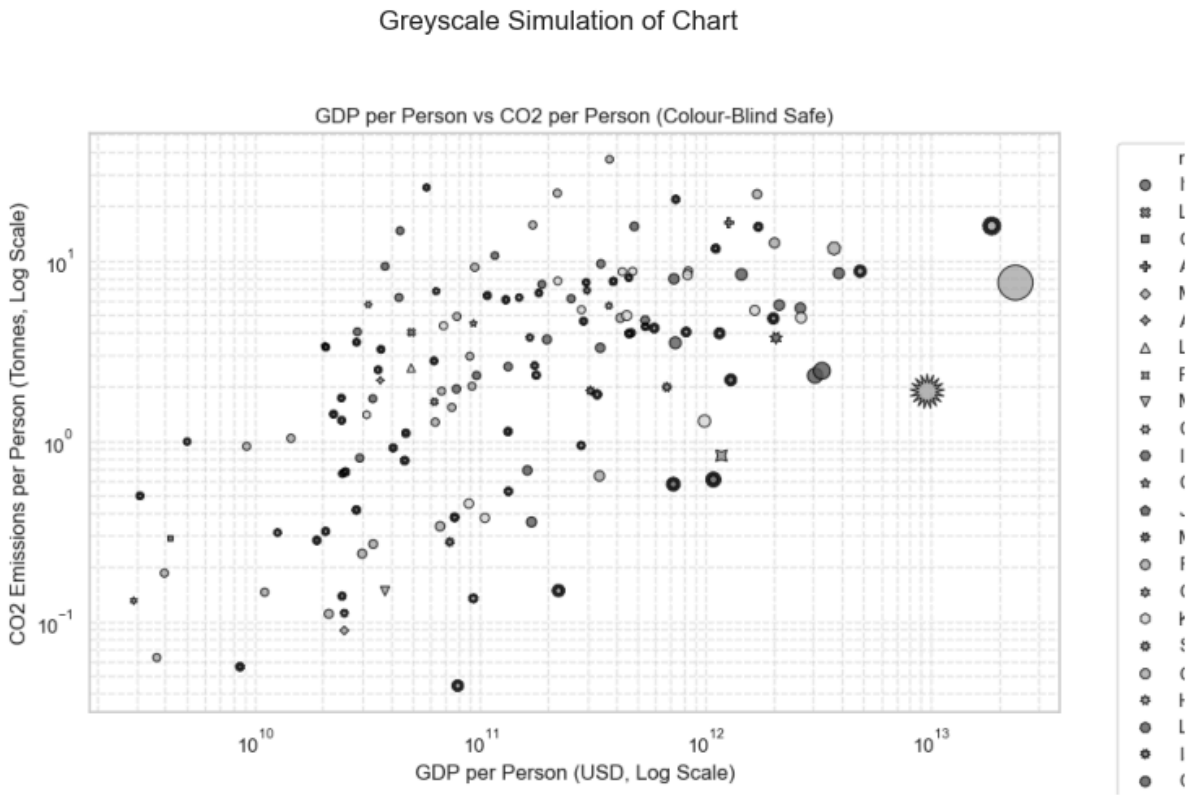
# Convert rendered figure to greyscale using canvas buffer
fig.canvas.draw()
rgba_buffer = np.asarray(fig.canvas.buffer_rgba())
greyscale_img = rgba_buffer.mean(axis=2)

# Display the greyscale image using imshow
plt.figure(figsize=(10, 6))
plt.imshow(greyscale_img, cmap='gray')
plt.axis('off')
plt.title("Greyscale Simulation of Chart")
plt.show()

# --- Comment ---
# When relying on colour alone, continents with similar marker luminances
# (such as European, Asian, and American sub-regions) become very hard
# to tell apart in greyscale. Distinct marker shapes and dark outlines ar
```



nation
Italy
Latvia
Gambia
Australia
Malawi
Armenia
Laos
Pakistan
Madagascar
Cyprus
Iraq
Croatia
Jamaica
Mexico
Russia
Guinea-Bissau
Kyrgyzstan
Syria
Qatar
Hong Kong
Libya
Israel
Germany
Morocco
Panama
Slovenia
Belgium
Colombia
Cuba
Azerbaijan
United Kingdom
Afghanistan
Mongolia
Bahrain
Mozambique
Rwanda
Finland
Dominican Republic
Liberia
Slovakia
Venezuela
Ghana
Sweden
Belarus
Oman
Lesotho
Philippines
Uganda
Sierra Leone
Togo
Greece
Chad
New Zealand
Canada
Netherlands
Sri Lanka
Cote d'Ivoire
Norway
Central African Republic
India
Tunisia
Portugal
South Africa
Equatorial Guinea
Saudi Arabia
Japan
Lebanon
Ireland
Uruguay
Guinea
Bolivia
Moldova
Brazil
Egypt
Yemen
Palestine
Spain
Switzerland
Chile
Ecuador



Hint 1 `sns.color_palette('colorblind', 6)` returns six colours; build dictionaries for colour and marker. **Hint 2** `np.asarray(fig.canvas.buffer_rgba())[..., :3].mean(axis=2)` is the greyscale image.

Exercise 46 – A clustered heatmap ★

A clustered heatmap ★. Take the 25 most populous countries of `d19`, build a table of `co2_per_person` (rows = nation, columns = 1990, 1993, 1996, ..., 2020), drop rows with missing values, and draw `sns.clustermap` standardised by row (`z_score=0`, `cmap='vlag'`). Print how many countries were dropped for missing values. In a comment: which countries end up next to each other, and what does standardising by row remove?

In [ ]:
```
df_source = yearly if 'yearly' in globals() else (countries if 'countries

country_col = 'countries' if 'countries' in df_source.columns else ('nati
year_col = 'year' if 'year' in df_source.columns else 'year'
pop_col = 'population'
val_col = 'co2_per_person'

# Take the 25 most populous countries
top_25_nations = df_source.groupby(country_col)[pop_col].max().nlargest(2
```

```python
# Extract historical data for these 25 nations across the specified 3-yea
years_range = list(range(1990, 2021, 3))
historical_subset = df_source[(df_source[country_col].isin(top_25_nations

# Build pivot table
pivot_df = historical_subset.pivot(index=country_col, columns=year_col, v
pivot_df = pivot_df.reindex(columns=years_range)

initial_count = len(pivot_df)

# Handle minor missing gaps
heatmap_data = pivot_df.interpolate(axis=1).bfill().ffill().dropna()

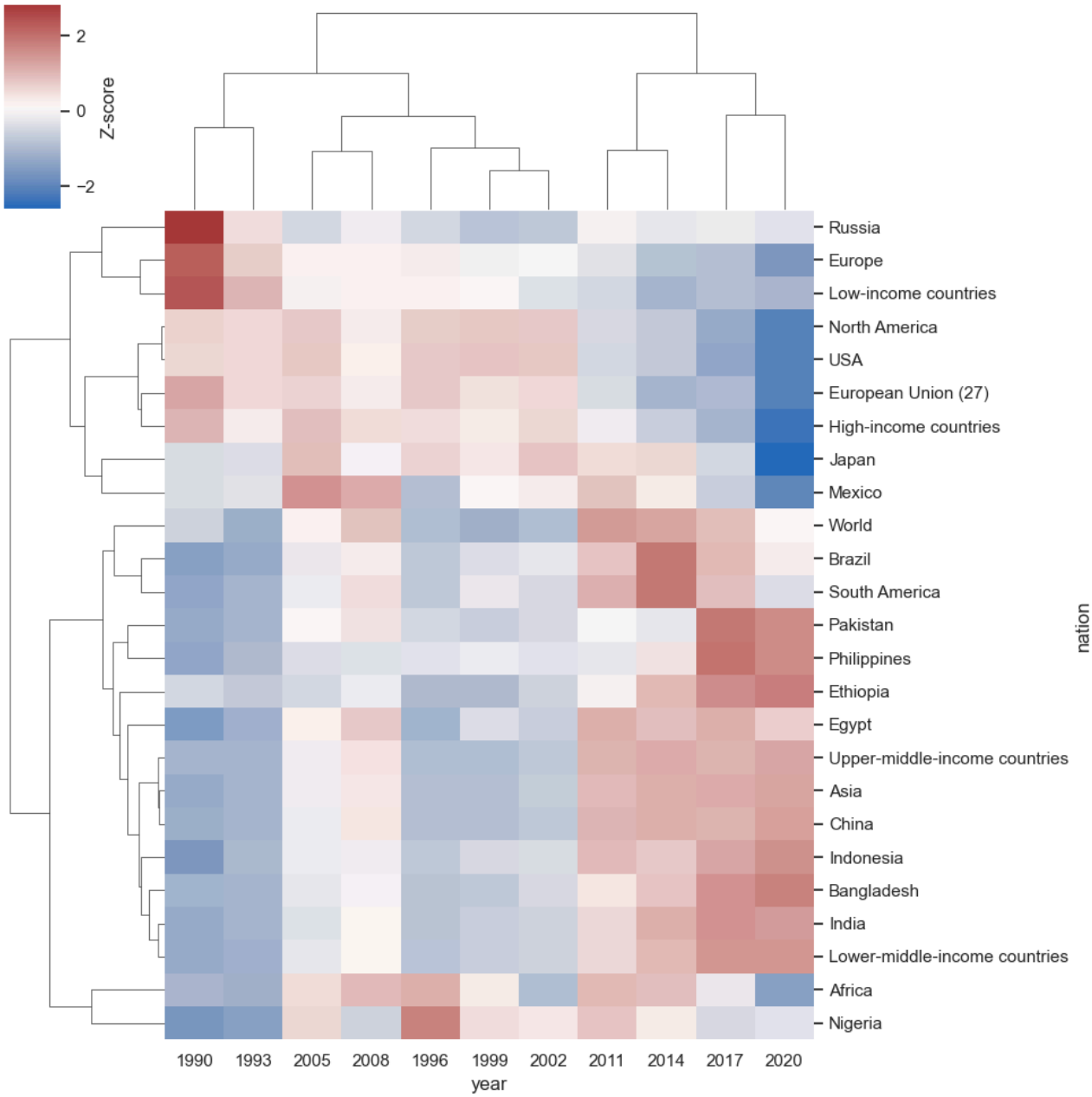
# Filter out zero-variance rows
heatmap_data = heatmap_data.loc[heatmap_data.std(axis=1) > 0]

dropped_count = initial_count - len(heatmap_data)
print(f"Number of countries excluded/dropped: {dropped_count}")
print(f"Final heatmap data shape: {heatmap_data.shape}")

if len(heatmap_data) >= 2:
    g = sns.clustermap(
        heatmap_data,
        z_score=0,
        cmap='vlag',
        figsize=(10, 10),
        dendrogram_ratio=(0.2, 0.2),
        cbar_kws={'label': 'Z-score'}
    )
    plt.show()
else:
    print("Error: Not enough valid countries remaining for clustering.")


# Which countries end up next to each other - Countries with similar temp
#   and emission trends across the decades cluster together based on hier
#   What does standardising by row remove Standardising by row (z-score n
#   absolute differences in scale/magnitude between countries, allowing t
#   on relative upward or downward trends over time.
```

```
Number of countries excluded/dropped: 0
Final heatmap data shape: (25, 11)
```



Hint 1 `pivot_table(index='nation', columns='year', values=...)[yrs].dropna()`. **Hint 2** `sns.clustermap(piv, z_score=0, cmap='vlag')` standardises each row.

Good Job :)

Save the notebook, make sure every cell runs top to bottom (Kernel ▸ Restart & Run All), and submit.